



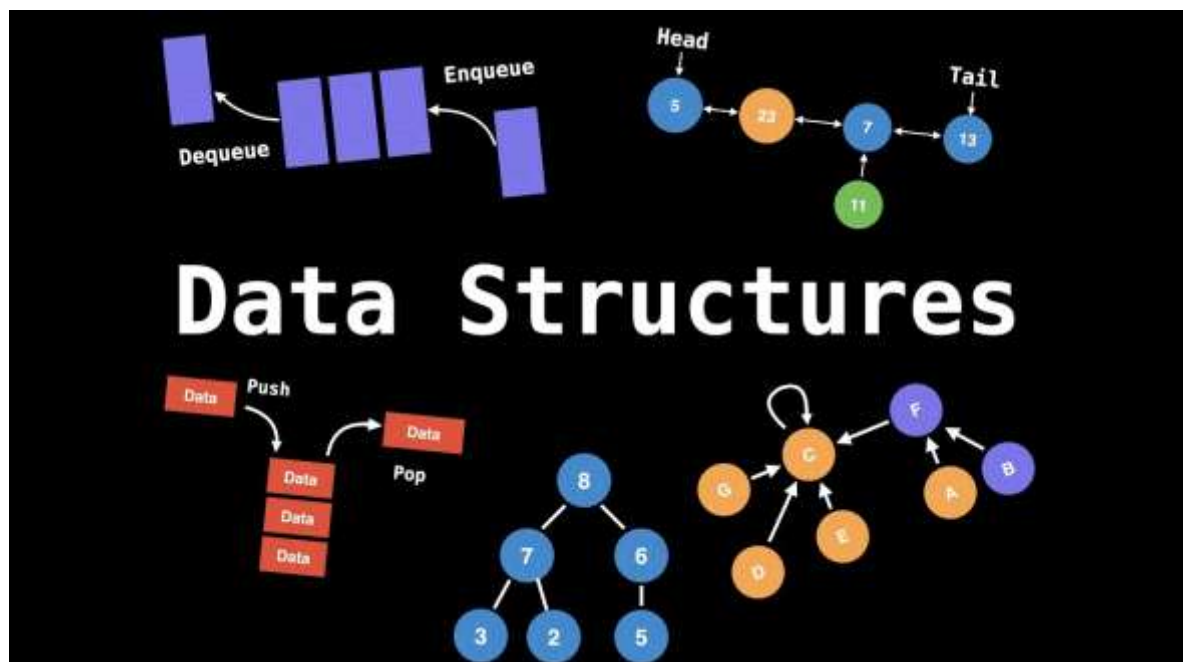
NEW L J INSTITUTE OF ENGINEERING & TECHNOLOGY

SEMESTER- 3

BRANCH- CSE/CSE(AIML)/CSE(DS)

SUBJECT NAME: DATA STRUCTURES

SUBJECT CODE: BE03000081





Course Outcomes

Sr. No.	CO Statement	Chapters	Marks % Weightage
CO-1	Define and classify various data structures, storage structures and common operations on them.	1,4	10
CO-2	Use various linear data structures with their representation and perform different operations on them.	2	20
CO-3	Use various nonlinear data structures with their representation and perform different operations on them.	3	20
CO-4	Apply various searching and sorting techniques on small and large dataset.	5	30
CO-5	Solve the given problem using an appropriate data structure to achieve optimal performance and compare its performance with other possible data structures.	1,2,3,4,5	20

Teaching and Examination Scheme

Teaching - Learning Scheme					Total Credits = TH/30	Assessment Pattern and Marks					Total Marks
L	T	P	TW/SL	TH		Theory		Tutorial / Practical			
						ESE (E)	PA (M)	PA/ (I)	TW/ SL (I)	ESE (V)	
45	0	60	15	120	04	70	30	20	30	50	200

Distribution of Theory Marks

R Level	U Level	A Level	N Level	E Level	C Level

Legends: R: Remembrance; U: Understanding; A: Application, N: Analyse and E: Evaluate C: Create and above Levels (Revised Bloom's Taxonomy)



Syllabus

Sr. No.	Topics	Teaching Hrs.	Module Weightage
1	INTRODUCTION TO DATA STRUCTURES: Basic Terminology, Classification of Data Structure- Primitive and Non Primitive Data Structure, Linear and Non-Linear Data Structures, Examples of Primitive and Non Primitive Data Structures, storage representation of Primitive and Non Primitive Data Structures	04	10
2	LINEAR DATA STRUCTURE Array: Representation of arrays, Applications of arrays, sparse matrix and its representation Stack: Stack-Definitions & Concepts, Operations On Stacks, Applications of Stacks, Polish Expression, Reverse Polish Expression And Their Compilation, Recursion, Tower of Hanoi Queue: Representation Of Queue, Operations On Queue, Circular Queue, Priority Queue, Array representation of Priority Queue, Double Ended Queue, Applications of Queue Linked List: Singly Linked List, Doubly Linked list, Circular linked list, Linked implementation of Stack, Linked implementation of Queue, Applications of linked list.	13	30
3	NON LINEAR DATA STRUCTURE : Trees: Definitions and Concepts, Representation of binary tree, Binary tree traversal (Inorder, postorder, preorder), Threaded binary tree, Binary search trees, Conversion of General Trees To Binary Trees, Applications Of Trees Graphs: Graph-Matrix Representation Of Graphs, Elementary Graph operations,(Breadth First Search, Depth First Search, Spanning Trees, Shortest path, Minimal spanning tree)	13	30
4	HASHING AND FILES TRUCTURES: Hashing: The symbol table, Hashing Functions, Collision- Resolution Techniques File Structure: Fixed and variable length record, File organizations: Sequential, Random Access, indexed and Relative/Random File Organization	07	15
5	Sorting & Searching: Sorting: Various sorting techniques: Bubble Sort, Selection Sort, Insertion Sort, Quick Sort, Merge Sort, Heap Sort Searching: Sequential Search and Binary Search	08	15
	TOTAL	45	100

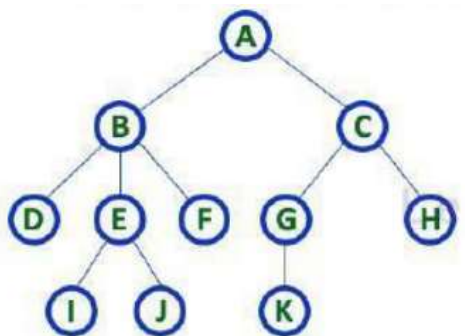
Unit 3 Non Linear Data Structure

Syllabus	Trees: Definitions and Concepts, Representation of binary tree, Binary tree traversal (Inorder, postorder, preorder), Threaded binary tree, Binary search trees, Conversion of General Trees To Binary Trees, Applications Of Trees. Graphs: Graph-Matrix Representation Of Graphs, Elementary Graph operations, (Breadth First Search, Depth First Search, Spanning Trees, Shortest path, Minimal spanning tree).
-----------------	--

Tree - Definitions and Concepts

➤ What is tree?

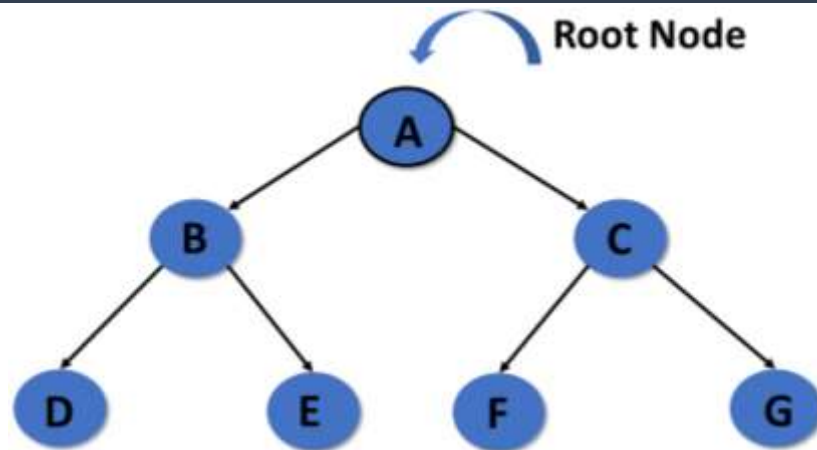
- Tree is a non-linear data structure which organizes data in hierarchical structure and this is a recursive definition.
- Tree data structure is a collection of data (Node) which is organized in hierarchical structure and this is a recursive definition.
- In tree data structure, every individual element is called as Node. Node in a tree data structure, stores the actual data of that particular element and link to next element in hierarchical structure.
- In a tree data structure, if we have N number of nodes then we can have a maximum of N-1 number of links.
- Example



- TREE with 11 nodes and 10 edges
- In any tree with 'N' nodes there will be maximum of 'N-1' edges
- In a tree every individual element is called as 'NODE'

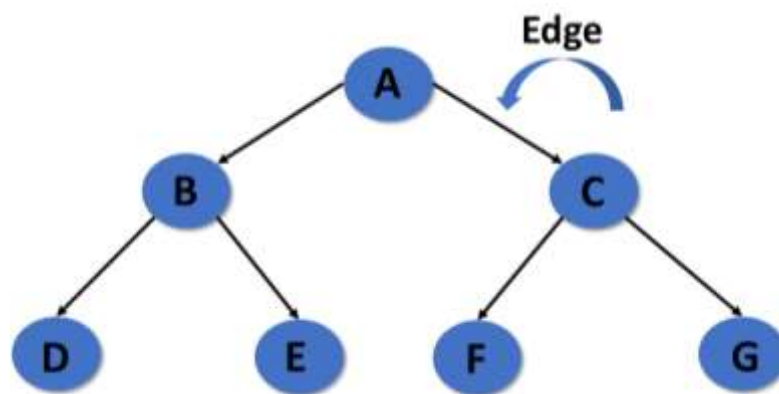
➤ Tree Terminology:

- In a tree data structure, we use the following terminology...
- **Root**
 - In a tree data structure, the root is the first node of the tree. The root node is the initial node of the tree in data structures.
 - In the tree data structure, there must be only one root node.



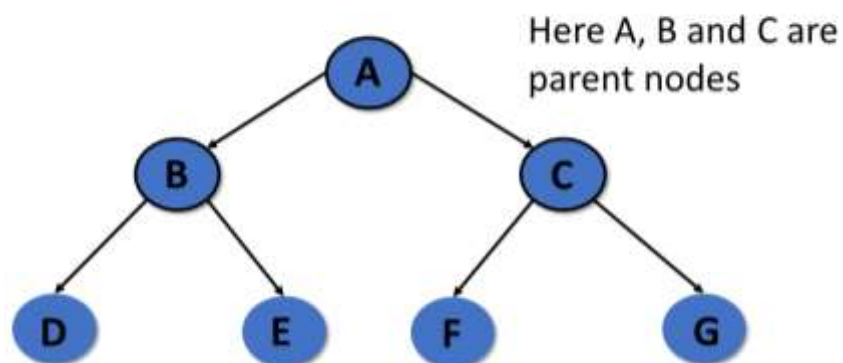
- **Edge**

- In a tree in data structures, the connecting link of any two nodes is called the edge of the tree data structure.
- In the tree data structure, N number of nodes connecting with N -1 number of edges.



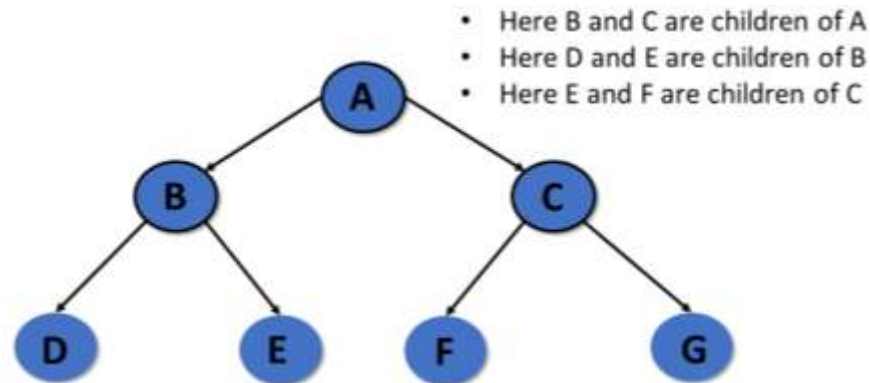
- **Parent**

- In the tree in data structures, the node that is the predecessor of any node is known as a parent node, or a node with a branch from itself to any other successive node is called the parent node.



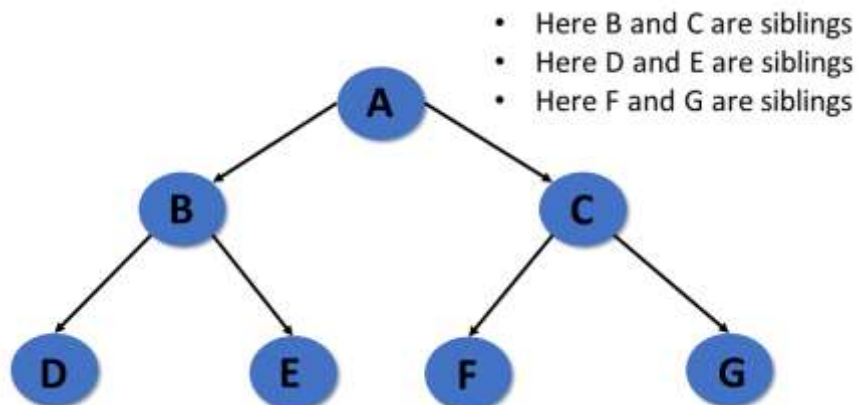
- **Child**

- The node, a descendant of any node, is known as child nodes in data structures.
- In a tree, any number of parent nodes can have any number of child nodes.
- In a tree, every node except the root node is a child node.



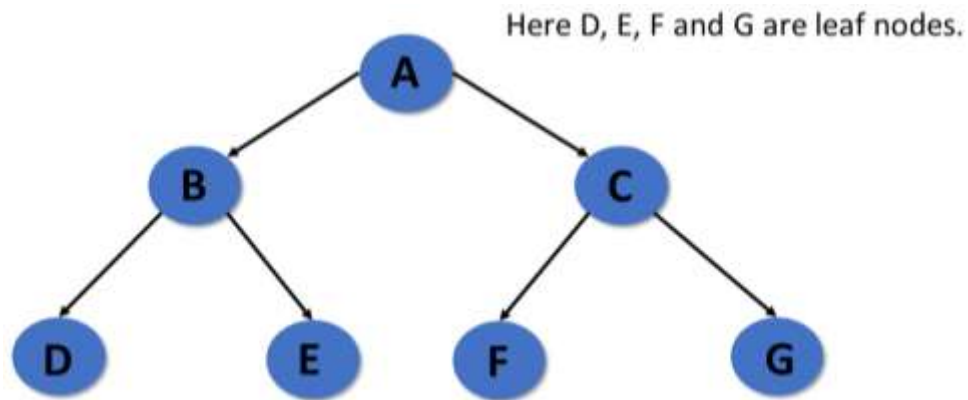
- **Siblings**

- In trees in the data structure, nodes that belong to the same parent are called siblings.



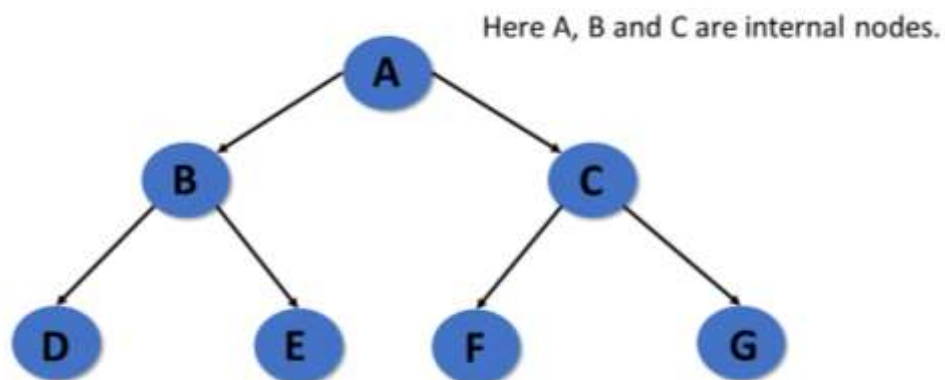
- **Leaf**

- Trees in the data structure, the node with no child, is known as a leaf node.
- In trees, leaf nodes are also called external nodes or terminal nodes.



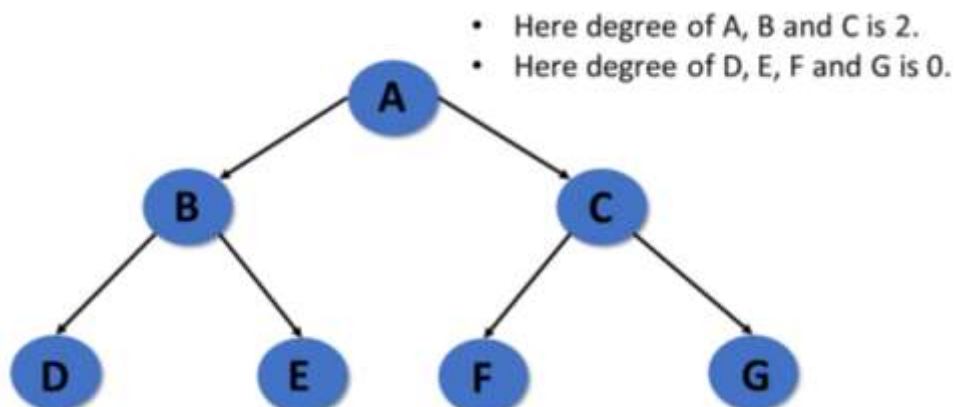
- **Internal nodes**

- Trees in the data structure have at least one child node known as internal nodes.
- In trees, nodes other than leaf nodes are internal nodes.
- Sometimes root nodes are also called internal nodes if the tree has more than one node.



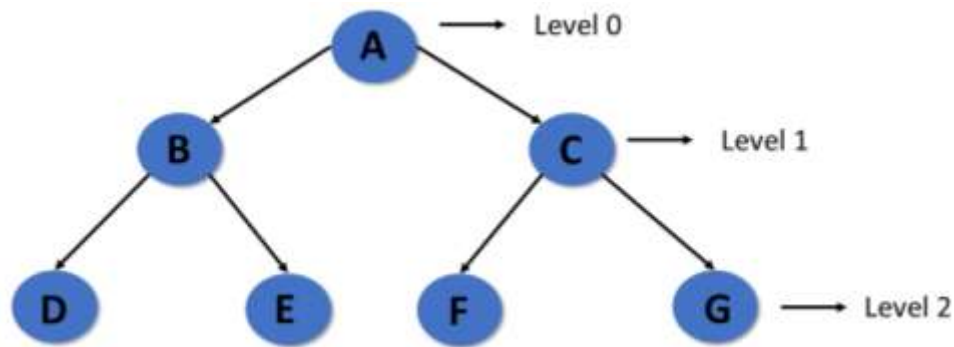
- **Degree**

- In the tree data structure, the total number of children of a node is called the degree of the node.
- The highest degree of the node among all the nodes in a tree is called the Degree of Tree.



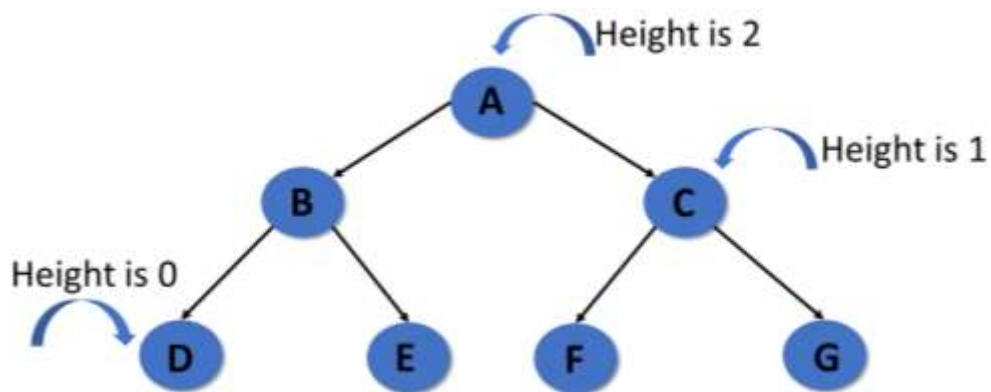
- **Level**

- In tree data structures, the root node is said to be at level 0, and the root node's children are at level 1, and the children of that node at level 1 will be level 2, and so on.



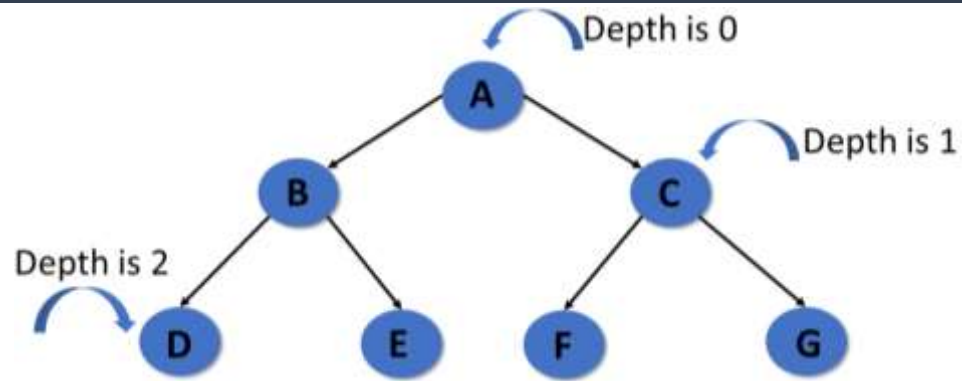
- **Height**

- In a tree data structure, the number of edges from the leaf node to the particular node in the longest path is known as the height of that node.
- In the tree, the height of the root node is called "Height of Tree".
- The tree height of all leaf nodes is 0.



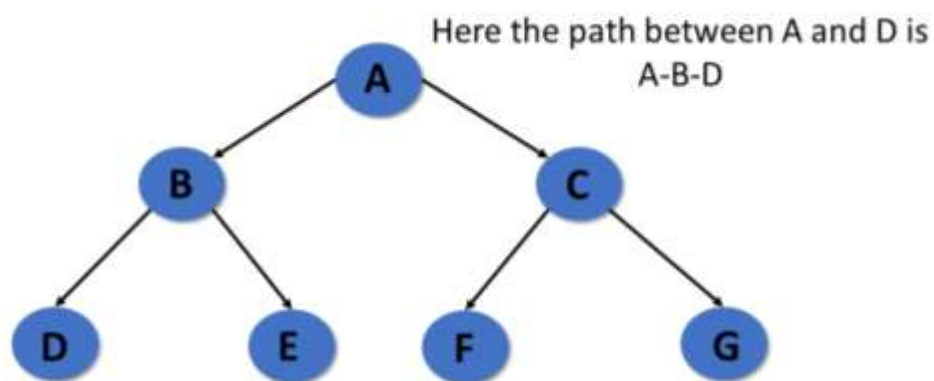
- **Depth**

- In a tree, many edges from the root node to the particular node are called the depth of the tree.
- In the tree, the total number of edges from the root node to the leaf node in the longest path is known as "Depth of Tree".
- In the tree data structures, the depth of the root node is 0.



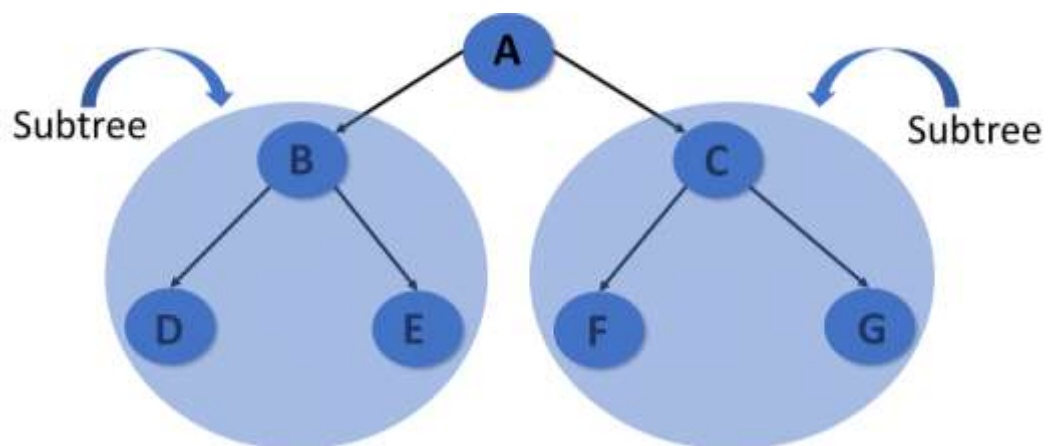
- **Path**

- In the tree in data structures, the sequence of nodes and edges from one node to another node is called the path between those two nodes.
- The length of a path is the total number of nodes in a path.



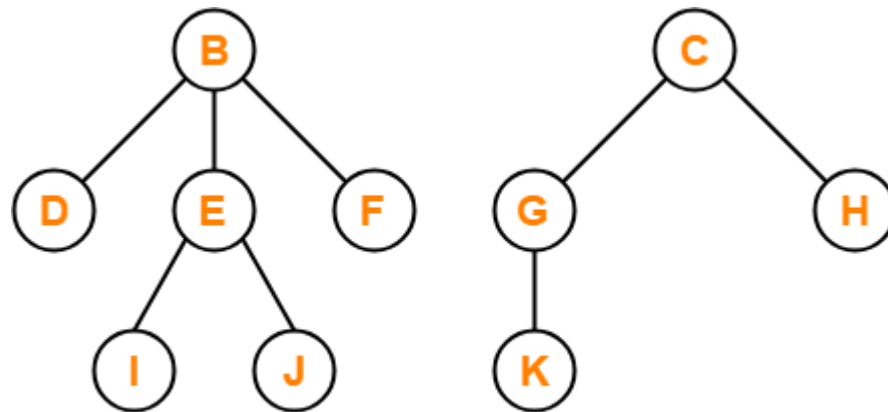
- **Subtree**

- In the tree in data structures, each child from a node shapes a subtree recursively and every child in the tree will form a sub-tree on its parent node.



- **Forest**

- A Forest is a collection of one or more disjoint trees. In graph theory, a forest is a set of trees that do not have any edges connecting them. Each tree within the forest is a separate, connected, acyclic graph.
- A forest is essentially a group of trees that share a common property of being acyclic and connected, but they are not necessarily connected to each other.

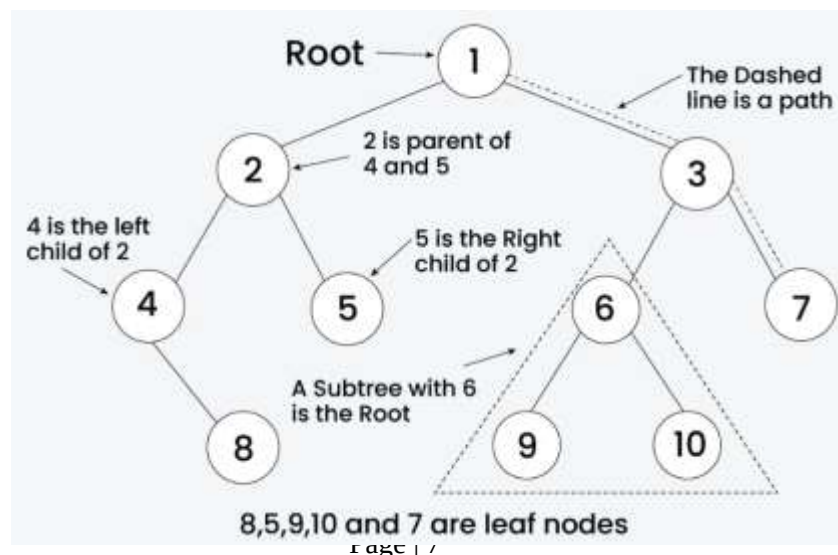


Forest

Binary Tree and its Representation

➤ What is Binary Tree:

- A Binary Tree Data Structure is a hierarchical data structure in which each node has at most two children, referred to as the left child and the right child.
- It is commonly used in computer science for efficient storage and retrieval of data, with various operations such as insertion, deletion, and traversal.



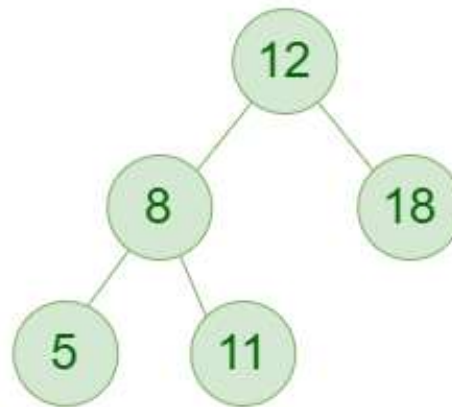
➤ **Types of Binary Tree:**

1. Full Binary Tree (Strictly Binary Tree)
2. Skewed Binary Trees
3. Complete Binary Tree
4. Perfect Binary Tree
5. Balanced Binary Tree

1. Full Binary Tree

- A Binary Tree is a full binary tree if every node has 0 or 2 children. The following are examples of a full binary tree. We can also say a full binary tree is a binary tree in which all nodes except leaf nodes have two children.
- A full Binary tree is a special type of binary tree in which every parent node/internal node has either two or no children. It is also known as a proper binary tree.

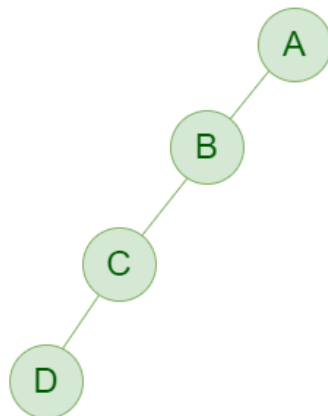
Full Binary Tree



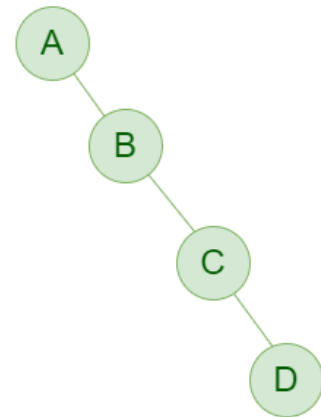
2. Skewed Binary Tree

- A skewed binary tree is a pathological/degenerate tree in which the tree is either dominated by the left nodes or the right nodes. Thus, there are two types of skewed binary tree: left-skewed binary tree and right-skewed binary tree.

Left-Skewed Binary Tree



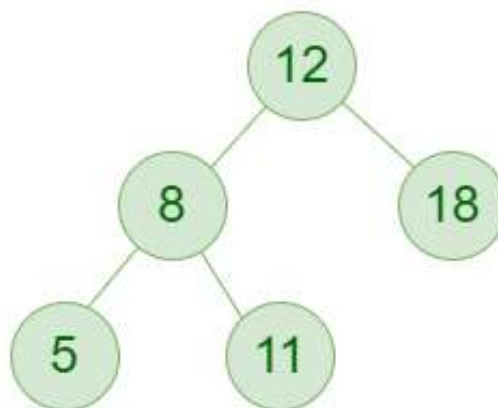
Right-Skewed Binary Tree



3. Complete Binary Tree

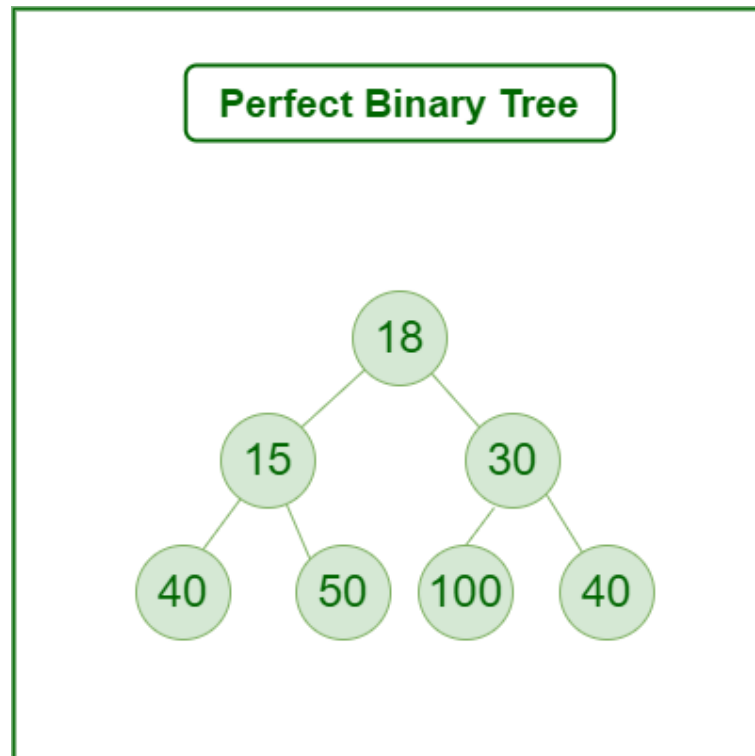
- A Binary Tree is a Complete Binary Tree if all the levels are completely filled except possibly the last level and the last level has all keys as left as possible.
- A complete binary tree is just like a full binary tree, but with two major differences:
- Every level except the last level must be completely filled.
- All the leaf elements must lean towards the left.
- The last leaf element might not have a right sibling i.e. a complete binary tree doesn't have to be a full binary tree.

Complete Binary Tree



4. Perfect Binary Tree

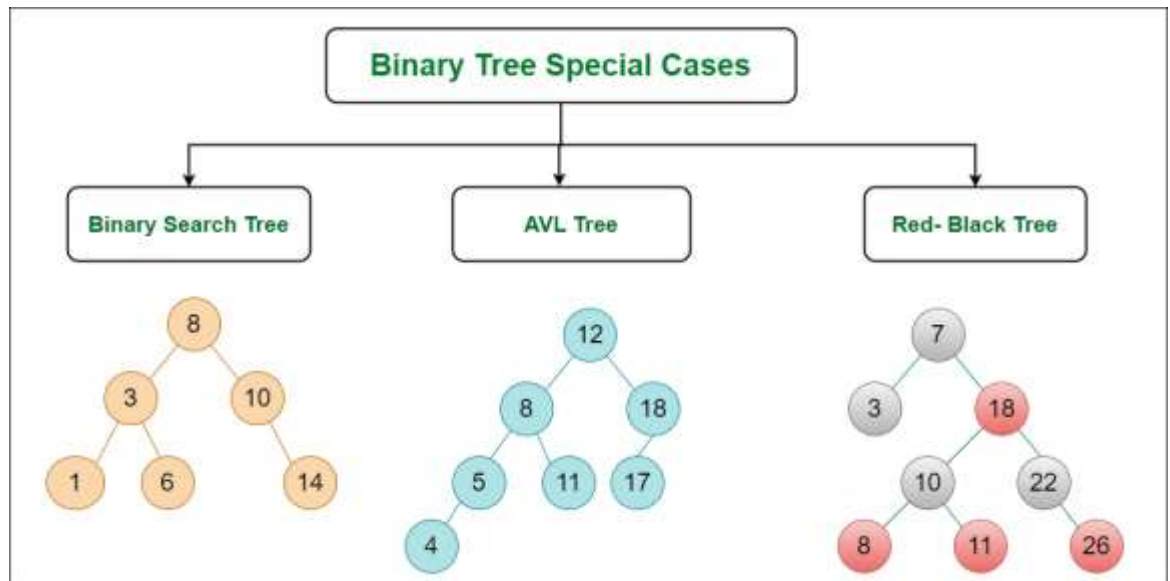
- A Binary tree is a Perfect Binary Tree in which all the internal nodes have two children and all leaf nodes are at the same level. The following are examples of Perfect Binary Trees.
- A perfect binary tree is a type of binary tree in which every internal node has exactly two child nodes and all the leaf nodes are at the same level.



- In a Perfect Binary Tree, the number of leaf nodes is the number of internal nodes plus 1
- $L = I + 1$
- Where L = Number of leaf nodes,
- I = Number of internal nodes.
- A Perfect Binary Tree of height h (where the height of the binary tree is the number of edges in the longest path from the root node to any leaf node in the tree, height of root node is 0) has $2^{h+1} - 1$ node.
- An example of a Perfect binary tree is ancestors in the family. Keep a person at root, parents as children, parents of parents as their children.

➤ **Some Special Types of Binary Trees:**

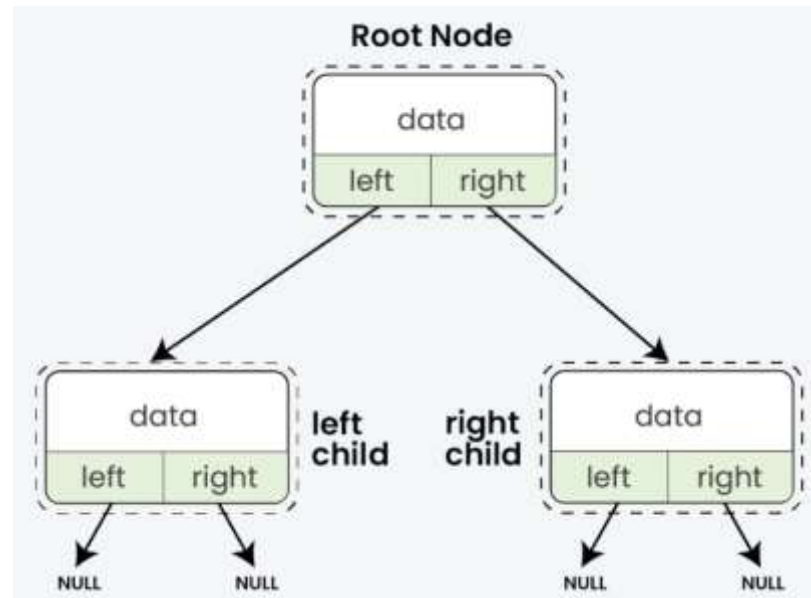
- On the basis of node values, the Binary Tree can be classified into the following special types:
 1. Binary Search Tree
 2. AVL Tree
 3. Red Black Tree
 4. B Tree



• **Binary Tree Representation:**

- Binary tree is a tree data structure (non-linear) in which each node can have at most two children which are referred to as the left child and the right child.
- The topmost node in a binary tree is called the root, and the bottom-most nodes are called leaves.
- A binary tree can be visualized as a hierarchical structure with the root at the top and the leaves at the bottom.
- Binary trees can be represented in multiple ways, each with its own advantages, depending on the use case.
- Let's explore the two common methods: linked node representation and array implementation.
- There are two primary ways to represent binary trees:
 1. **Linked Node Representation**
 2. **Array Representation**

- This is the simplest way to represent a binary tree.
- Each node contains data and pointers to its left and right children. This representation is mostly used to represent binary tree with multiple advantages.



- **Advantages:**

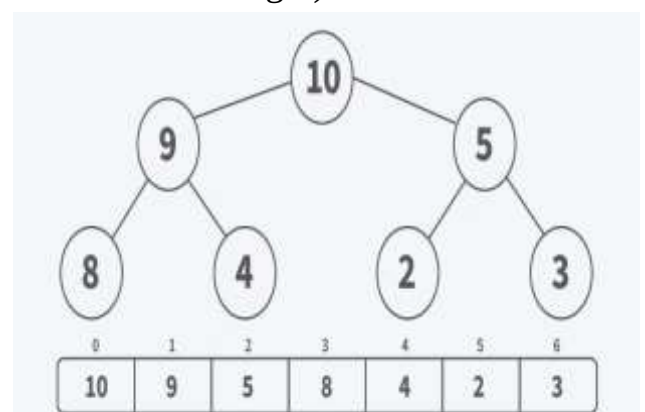
- It can easily grow or shrink as needed, so it uses only the memory it needs.
- Adding or removing nodes is straightforward and requires only pointer adjustments.
- Only uses memory for the nodes that exist, making it efficient for sparse trees.

- **Disadvantages:**

- Needs extra memory for pointers.
- Finding a node can take longer because you have to start from the root and follow pointers.

2. Array Representation

- Array Representation is another way to represent binary trees, especially useful when the tree is complete (all levels are fully filled except possibly the last, which is filled from left to right).
- In this method:
 - The tree is stored in an array.
 - For any node at index i :
 - Left Child: Located at $(2 * i) + 1$



- Right Child: Located at $(2 * i) + 2$
- Root Node: Stored at index 0
- **Advantages:**
 - Easy to navigate parent and child nodes using index calculations, which is fast
 - Easier to implement, especially for complete binary trees.
- **Disadvantages:**
 - You have to set a size in advance, which can lead to wasted space.
 - If the tree is not complete binary tree then many slots in the array might be empty, this will result in wasting memory
 - Not as flexible as linked representations for dynamic trees.
- Comparison of Linked list vs Array Representation of Binary Tree

Feature	Linked-Node Representation	Array Representation
Memory Usage	Uses extra memory for pointers	Efficient for complete trees
Ease of Implementation	Simple and intuitive	Simple but limited to complete trees
Dynamic Size	Easily grows and shrinks	Not flexible for dynamic sizes
Access Time	Slower access for specific nodes	Fast access using indices
Suitable For	Any binary tree structure	Complete binary tree

Binary Tree Traversal

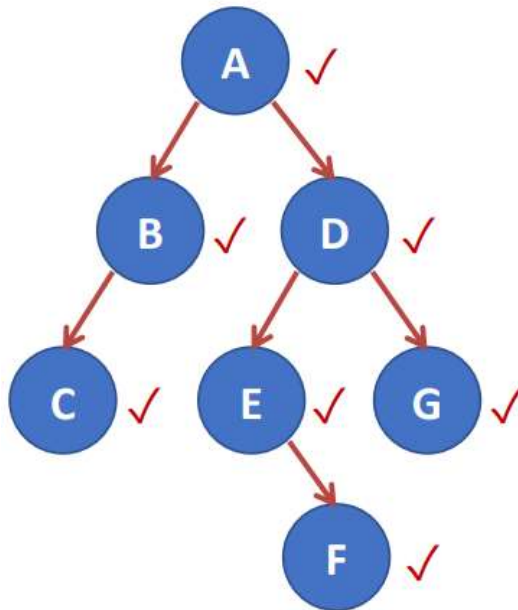
➤ Tree Traversal

- The process of visiting the nodes is known as tree traversal.
- This is a procedure by which each node in the tree is processed exactly once in a systematic manner
- There are three types traversals used to visit a node:
 - Inorder traversal
 - Preorder traversal
 - Postorder traversal

➤ **Preorder Traversal**

- Preorder traversal of a binary tree is defined as follow

- 1. Process the root node
- 2. Traverse the left subtree in preorder
- 3. Traverse the right subtree in preorder
- If particular subtree is empty (i.e., node has no left or right descendant) the traversal is performed by doing nothing.
- In other words, a null subtree is considered to be fully traversed when it is encountered



Preorder traversal of a given tree as

A B C D E F G

➤ **Procedure: RPREORDER(T) (Algorithm for preorder)**

- This procedure traverses the tree in preorder, in a recursive manner.
- T is root node address of given binary tree.
- Node structure of binary tree is described as below



Typical node of Binary Tree

• **Algorithm:**

RPREORDER(T)

1. [Check for Empty Tree]
 - IF T = NULL
 - THEN write ('Empty Tree')
 - return
 - ELSE write (DATA(T))
2. [Process the Left Sub Tree]
 - IF LPTR(T) ≠ NULL

THEN RPREORDER(LPTR(T))

3. [Process the Right Sub Tree]

IF RPTR(T) $\neq$ NULL

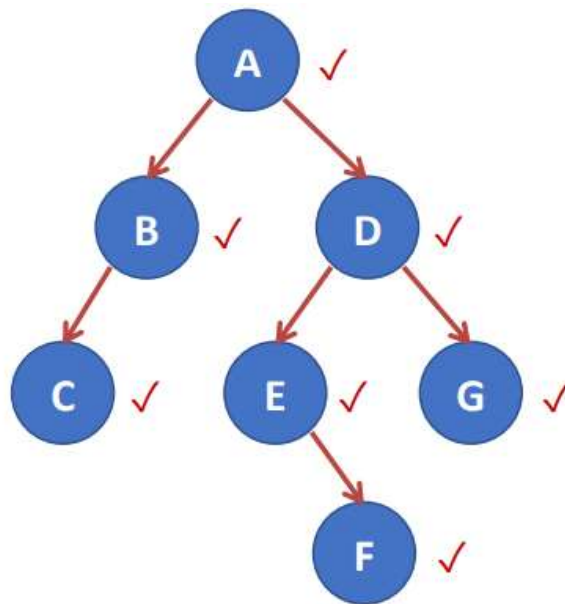
THEN RPREORDER(RPTR(T))

4. [Finished]

Return

➤ Inorder Traversal

- Inorder traversal of a binary tree is defined as follow
 - 1. Traverse the left subtree in Inorder
 - 2. Process the root node
 - 3. Traverse the right subtree in Inorder



Inorder traversal of a given tree as

C B A E F D G

➤ Procedure: RINORDER (T) (Algorithm for Inorder)

- This procedure traverses the tree in InOrder, in a recursive manner.
- T is root node address of given binary tree.
- **Algorithm:**

RINORDER(T)

1. [Check for Empty Tree]

IF T = NULL

THEN write ('Empty Tree')

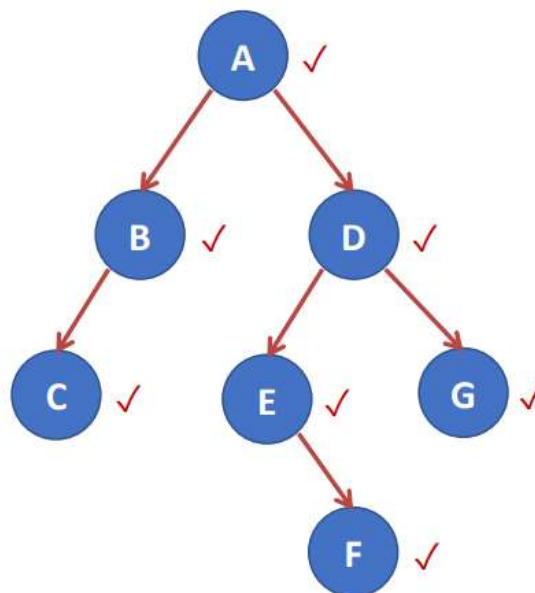
return

2. [Process the Left Sub Tree]

```
IF LPTR(T) ≠ NULL
    THEN RINORDER(LPTR(T))
3. [Process the Root Node]
   write (DATA(T))
4. [Process the Right Sub Tree]
   IF RPTR(T) ≠ NULL
       THEN RINORDER(RPTR(T))
5. [Finished]
   Return
```

➤ **Postorder Traversal**

- Postorder traversal of a binary tree is defined as follow
 - 1. Traverse the left subtree in Postorder
 - 2. Traverse the right subtree in Postorder
 - 3. Process the root node



Postorder traversal of a given tree as

C B F E G D A

➤ **Procedure: RPOSTORDER (T) (Algorithm for Postorder)**

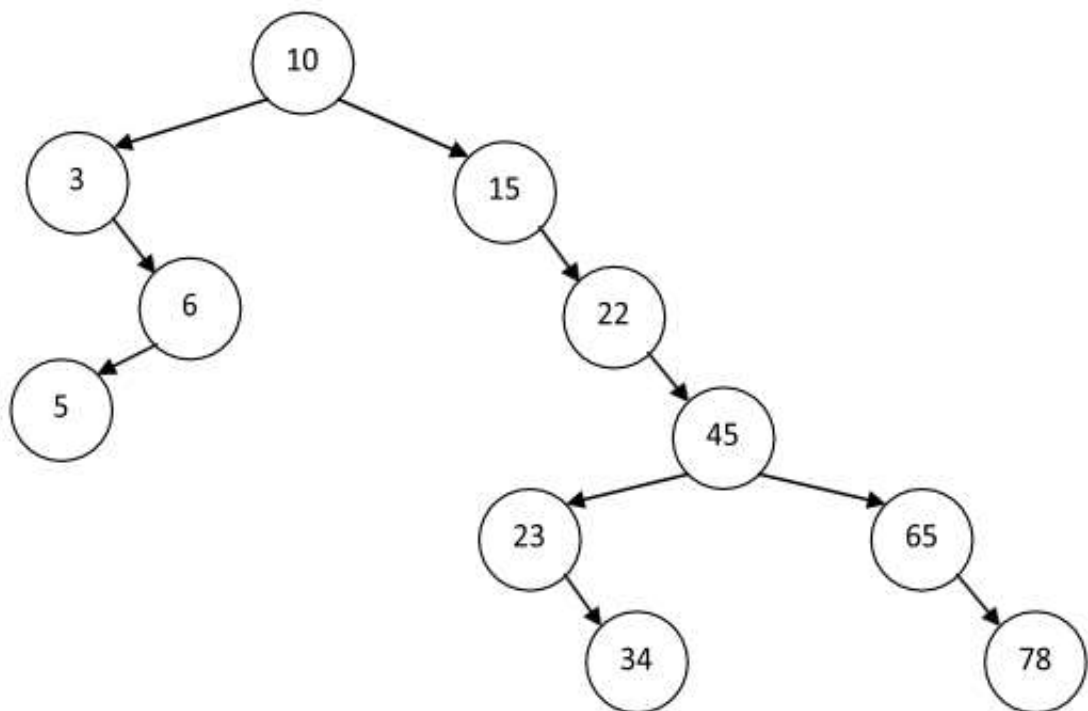
- This procedure traverses the tree in PostOrder, in a recursive manner.
- T is root node address of given binary tree.
- **Algorithm:**

RPOSTORDER(T)

1. [Check for Empty Tree]
IF T = NULL

```
    THEN write ('Empty Tree')
    return
2. [Process the Left Sub Tree]
   IF LPTR(T)  $\neq$  NULL
       THEN RPOSTORDER(LPTR(T))
3. [Process the Right Sub Tree]
   IF RPTR(T)  $\neq$  NULL
       THEN RPOSTORDER(RPTR(T))
4. [Process the Root Node]
   write (DATA(T))
5. [Finished]
   Return
```

- **Example:** Do Inorder, Preorder and Postorder traversal for following tree.



Preorder :10,3,6,5,15,22,45,23,34,65,78

Inorder :3,5,6,10,15,22,23,34,45,65,78

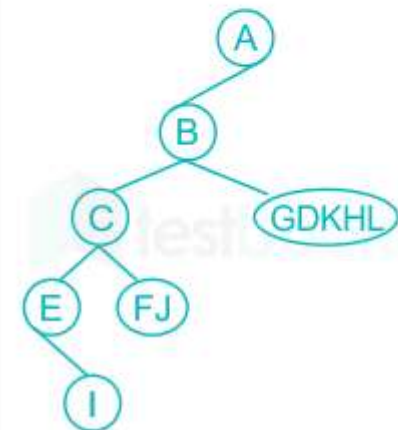
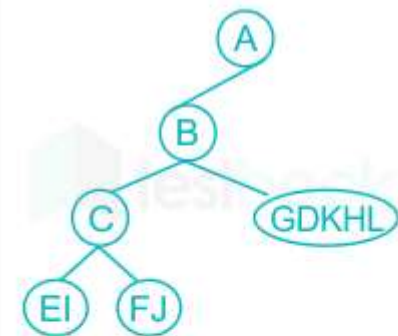
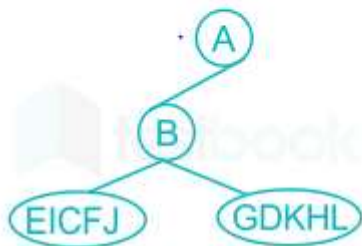
Postorder :5,6,3,34,23,78,65,45,22,15,10

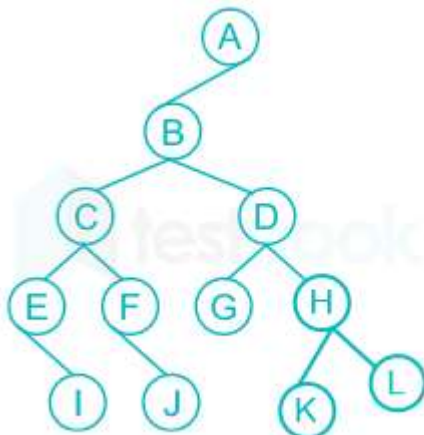
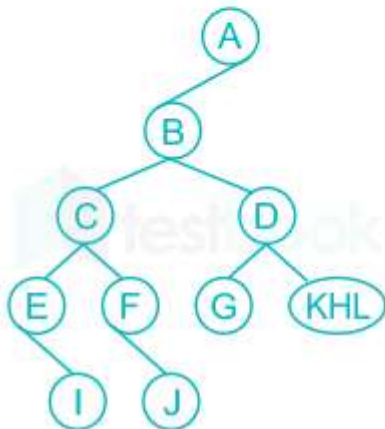
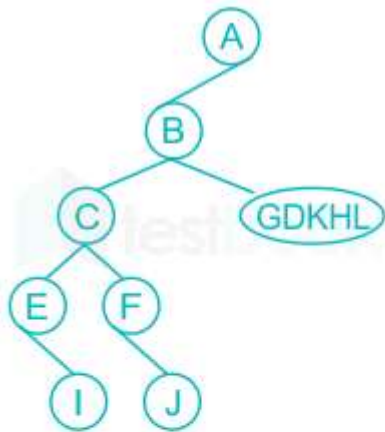
Traversal to Binary Tree

Example – 1: Construct Binary Tree from Given Traversal.

Preorder : A,B,C,E,I,F,J,D,G,H,K,L

Inorder : E,I,C,F,J,B,G,D,K,H,L,A



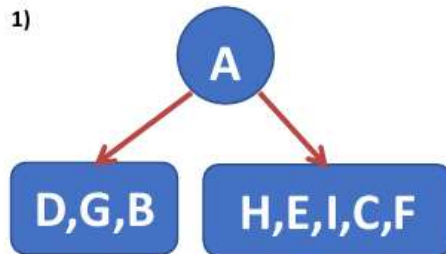


Example 2: Construct Binary Tree from given Traversals

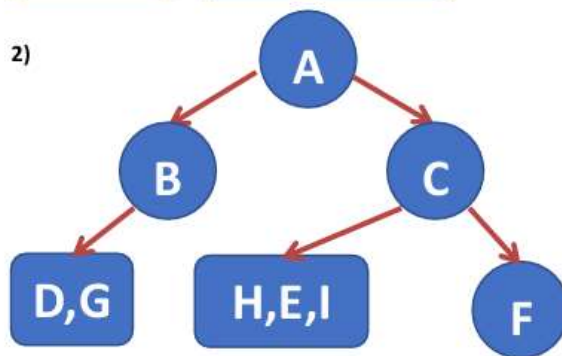
Inorder : D G B A H E I C F

Postorder : G D B H I E F C A

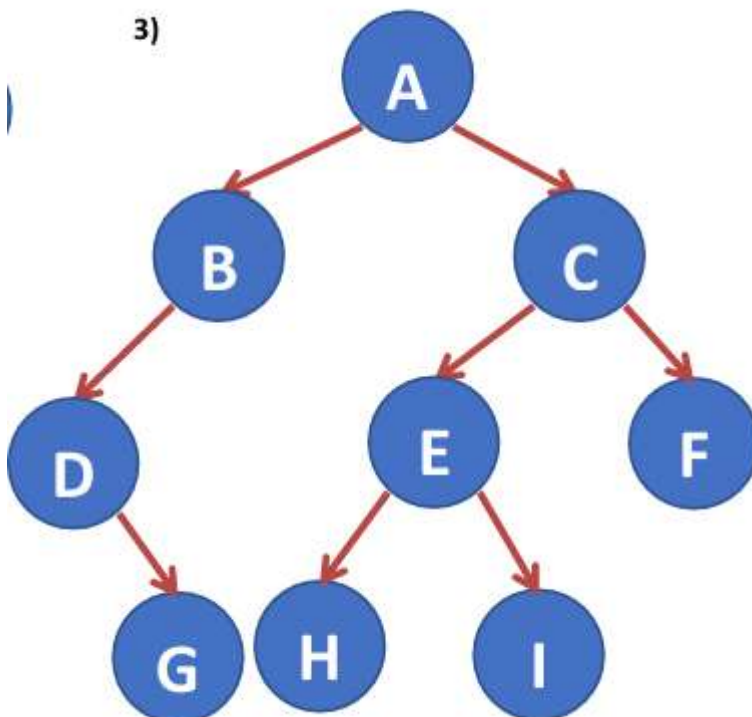
1)



2)



3)

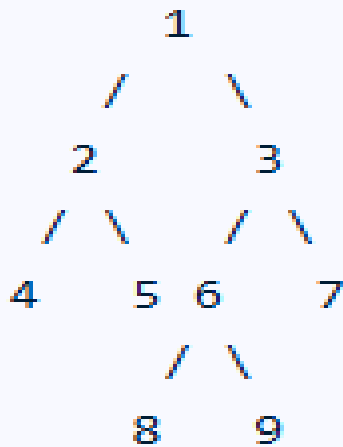


Example 3: Construct Tree from given Traversal

Preorder : 1,2,4,5,3,6,8,9,7

Postorder: 4,5,2,8,9,6,7,3,1

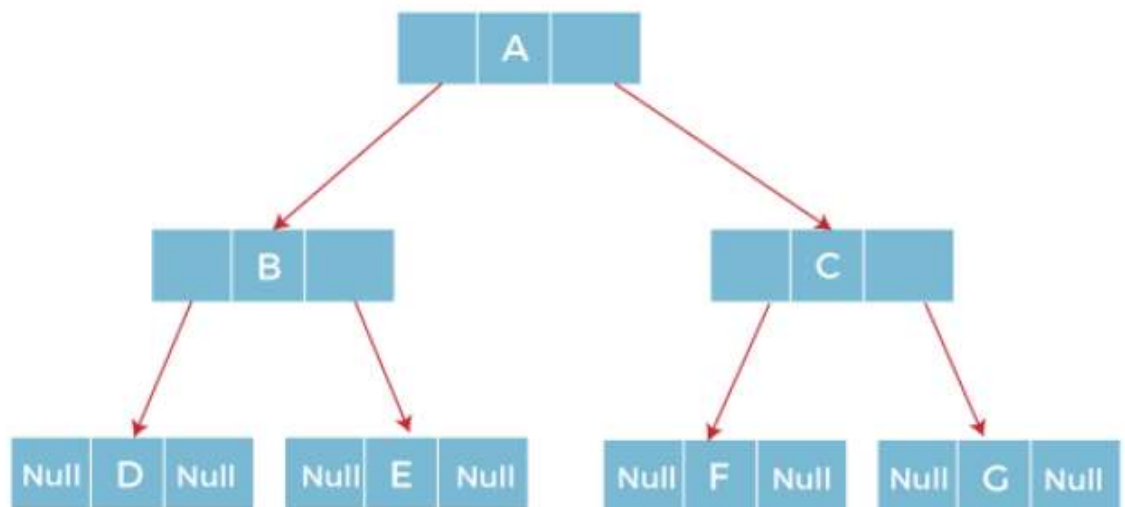
1. Identify the root: The root of the entire tree is the first element of the preorder traversal: 1.
2. Locate subtrees: The element after the root in the preorder traversal, 2, is the root of the left subtree. Find 2 in the postorder traversal to determine the number of nodes in the left subtree.
Preorder: 1, [2, 4, 5], [3, 6, 8, 9, 7]
Postorder: [4, 5, 2], [8, 9, 6, 7, 3], 1
3. Recursive calls: Recursively apply the same logic to the subarrays for the left and right subtrees.
 - **Left Subtree:** The elements for the left subtree are [2, 4, 5] in preorder and [4, 5, 2] in postorder.
 - The root is 2 (first in preorder).
 - The left child is 4 (first after 2 in preorder).
 - 4 is the first element in the postorder array [4, 5, 2]. This means the left subtree of 2 has one node.
 - The left subtree is 4 (preorder: [4], postorder: [4]).
 - The right subtree consists of the remaining elements: 5 (preorder: [5], postorder: [5]).
 - **Right Subtree:** The elements for the right subtree are [3, 6, 8, 9, 7] in preorder and [8, 9, 6, 7, 3] in postorder.
 - The root is 3 (first in preorder [3, ...]).
 - The left child is 6 (first after 3 in preorder).
 - Find 6 in the postorder subarray [8, 9, 6, 7, 3]. It is at index 2.
 - The left subtree of 3 has three nodes (preorder: [6, 8, 9], postorder: [8, 9, 6]). The right subtree of 3 has two nodes (preorder: [7], postorder: [7]).
4. Repeat: Continue this process for all subtrees until you have only single-node trees.



Threaded Binary Tree

➤ Definition

- In the linked representation of binary trees, more than one half of the link fields contain NULL values which results in wastage of storage space. If a binary tree consists of n nodes, then $n+1$ link fields contain NULL values.
- So, in order to effectively manage the space, a method was devised by Perlis and Thornton in which the NULL links are replaced with special links known as threads. Such binary trees with threads are known as threaded binary trees.
- Each node in a threaded binary tree either contains a link to its child node or thread to other nodes in the tree.



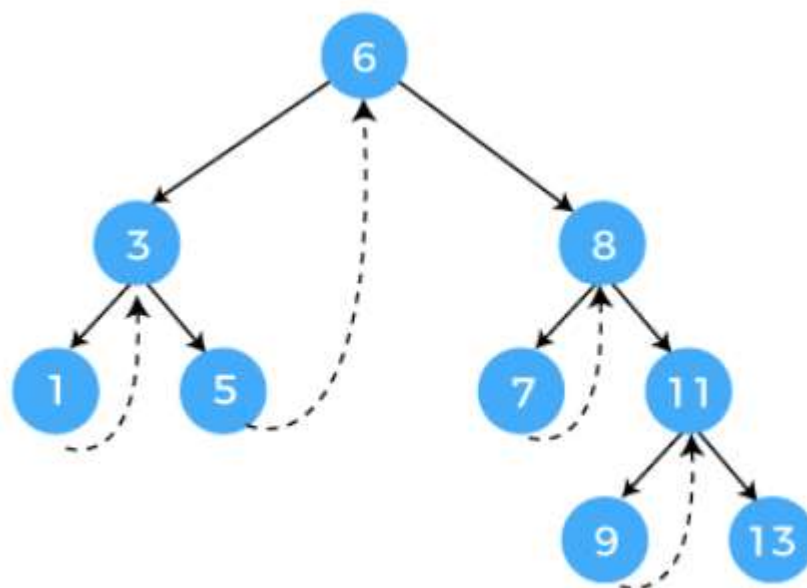
Threaded binary tree

- **Types of Threaded Binary Tree**

1. One-way threaded Binary Tree
2. Two-way threaded Binary Tree

- **1. One-way threaded Binary trees**

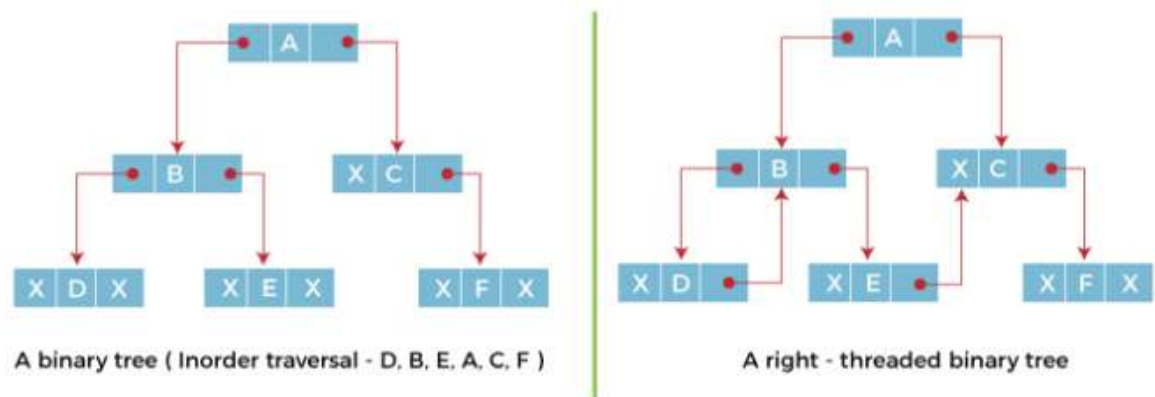
- In one-way threaded binary trees, a thread will appear either in the right or left link field of a node.
- If it appears in the right link field of a node then it will point to the next node that will appear on performing in order traversal. Such trees are called Right threaded binary trees.
- If thread appears in the left field of a node then it will point to the nodes inorder predecessor. Such trees are called Left threaded binary trees. Left threaded binary trees are used less often as they don't yield the last advantages of right threaded binary trees.
- In one-way threaded binary trees, the right link field of last node and left link field of first node contains a NULL.
- In order to distinguish threads from normal links they are represented by dotted lines.



Single Threaded Binary Tree

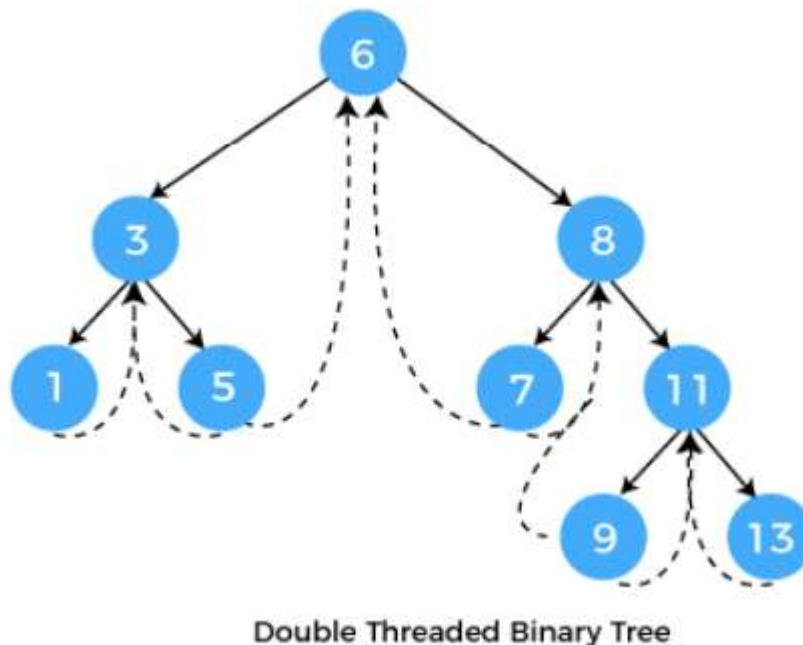
- The below figure shows the inorder traversal of this binary tree yields D, B, E, A, C, F. When this tree is represented as a right threaded binary tree, the right link field of leaf node D which contains a NULL value is replaced with a thread that points to node B which is the inorder successor of a node D. In

the same way other nodes containing values in the right link field will contain NULL value.



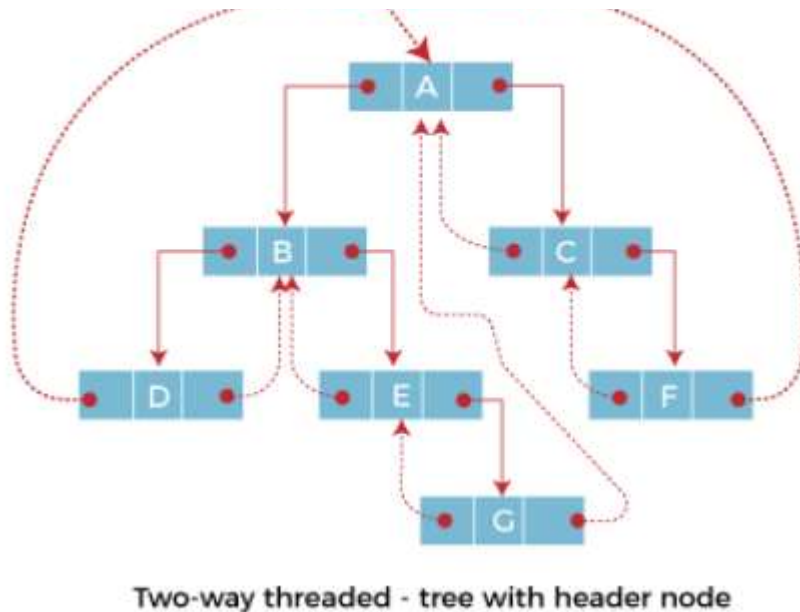
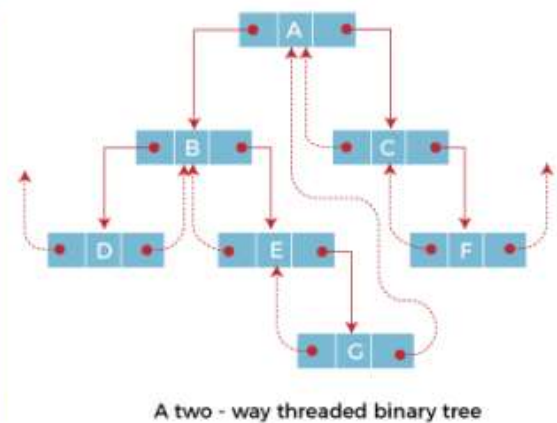
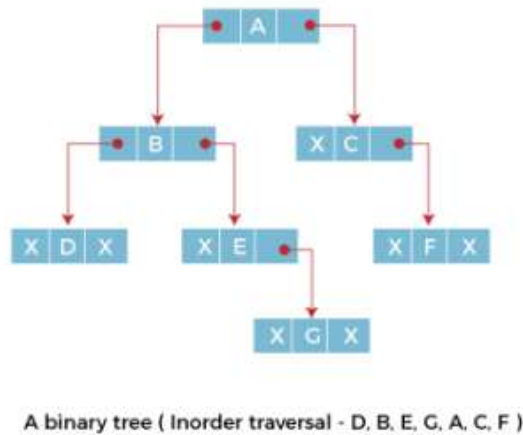
➤ Two-way threaded Binary Trees:

- In two-way threaded Binary trees, the right link field of a node containing NULL values is replaced by a thread that points to nodes inorder successor and left field of a node containing NULL values is replaced by a thread that points to nodes inorder predecessor.



- The below figure shows the inorder traversal of this binary tree yields D, B, E, G, A, C, F. If we consider the two-way threaded Binary tree, the node E whose left field contains NULL is replaced by a thread pointing to its inorder predecessor i.e. node B. Similarly, for node G whose right and left linked fields contain NULL values are replaced by threads such that right link field

points to its inorder successor and left link field points to its inorder predecessor. In the same way, other nodes containing NULL values in their link fields are filled with threads.



- In the above figure of two-way threaded Binary tree, we noticed that no left thread is possible for the first node and no right thread is possible for the last node. This is because they don't have any inorder predecessor and successor respectively.
- This is indicated by threads pointing nowhere. So in order to maintain the uniformity of threads, we maintain a special node called the header node. The header node does not contain any data part and its left link field points to the root node and its right link field points to itself.

- If this header node is included in the two-way threaded Binary tree then this node becomes the inorder predecessor of the first node and inorder successor of the last node.
- Now threads of left link fields of the first node and right link fields of the last node will point to the header node.

Binary Search Tree

➤ Binary Search tree exhibits a special behavior:

- A node's left child must have a value less than its parent's value.
- The node's right child must have a value greater than its parent value.

➤ Insertion in Binary Search Tree (BST):

- Given a BST, the task is to insert a new node in this BST.
- A new key is always inserted at the leaf by maintaining the property of the binary search tree. We start searching for a key from the root until we hit a leaf node. Once a leaf node is found, the new node is added as a child of the leaf node.
- **The below steps are followed while we try to insert a node into a binary search tree:**
 - Initialize the current node (say, currNode or node) with root node
 - Compare the key with the current node.
 - Move left if the key is less than or equal to the current node value.
 - Move right if the key is greater than current node value.
 - Repeat steps 2 and 3 until you reach a leaf node.
 - Attach the new key as a left or right child based on the comparison with the leaf node's value.

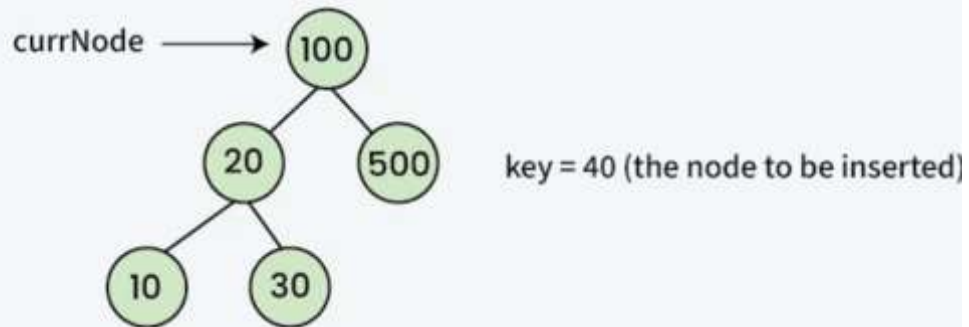
➤ **Algorithm :**

1. Read the value for the node which is to be created and store it in a node called NEW.
2. Initially if (Root=NULL) then root=NEW.
3. Again read next value of node created in NEW
4. If $\text{Info}(\text{NEW}) < \text{Info}(\text{Root})$
Then Attach NEW node as Left Child of the Root node
Else
Attach NEW node as Right Child of Root.
5. Repeat step 3 & 4 for constructing required binary search tree completely.

➤ Example :

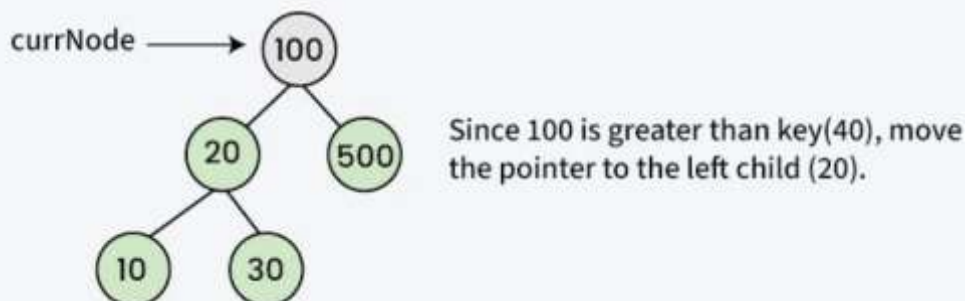
01
Step

Consider the following BST



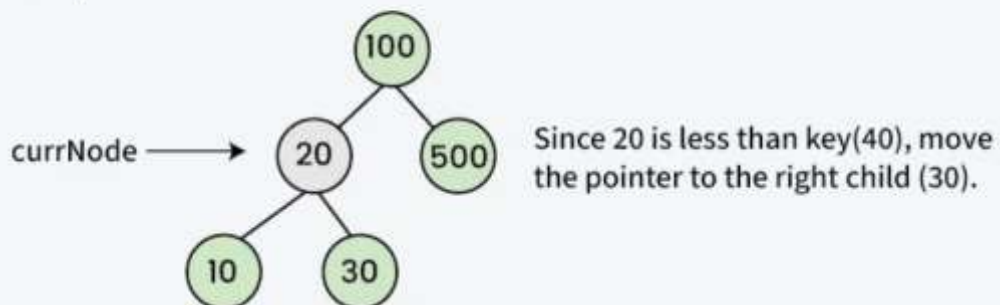
02
Step

Comparing key with root node



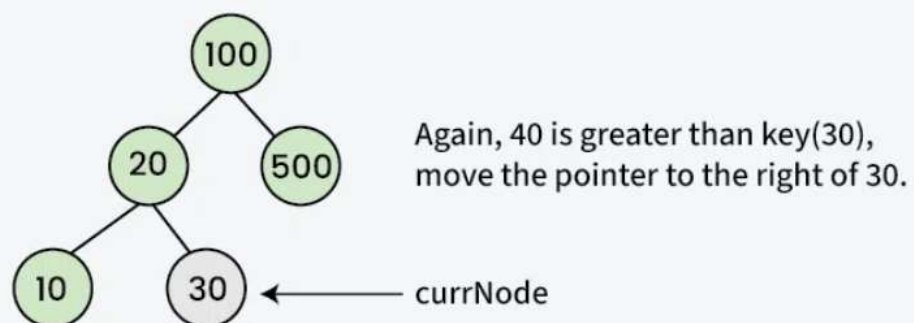
03
Step

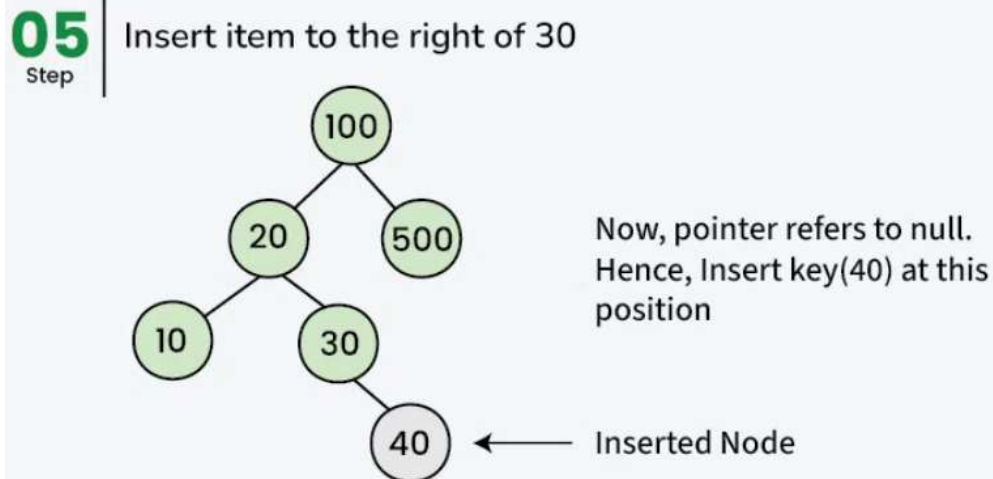
Comparing key with left child root node



04
Step

Comparing key with the right child of 20





➤ **Searching in Binary Search Tree (BST)**

- Given a BST, the task is to search a node in this BST. For searching a value in BST, consider it as a sorted array. Now we can easily perform search operation in BST using Binary Search Algorithm.

- Algorithm:**

Tree= Root Node

SearchElement(Tree,Val)

5. If Info(Tree)=Val OR Tree = NULL

Return Tree

Else

If Val < Info(Tree)

Return SearchElement(LPTR(Tree),Val)

Else

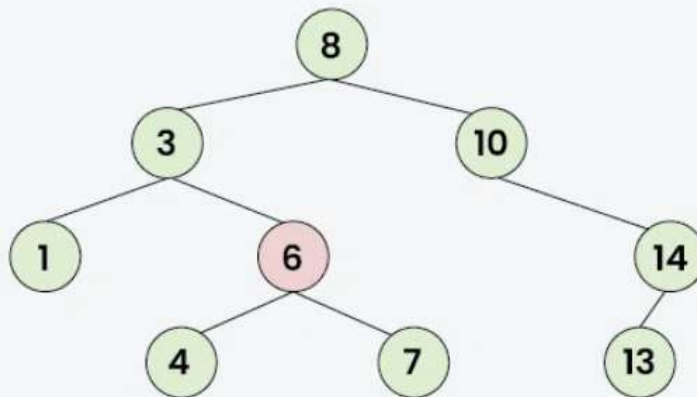
Return SearchElement(RPTR(Tree),Val)

[End If]

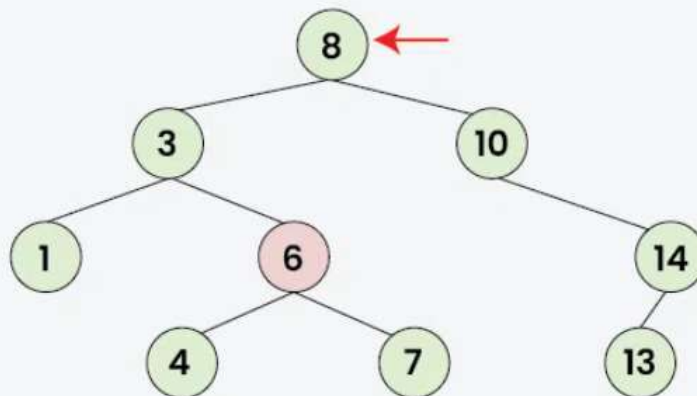
[End If]

6. End

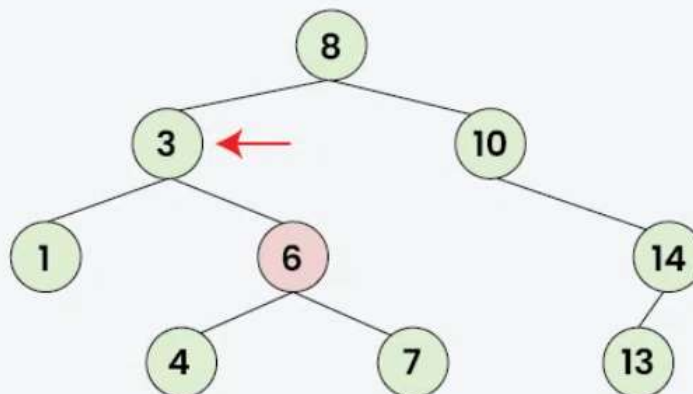
01 Step | Consider the Following BST
Key = 6



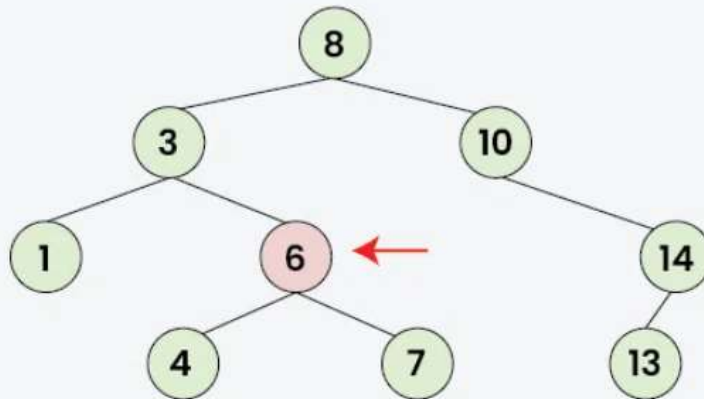
02 Step | Compare Key with Root, i.e 8
As $6 < 8$, Search in left subtree of 8



03 Step | As Key (6) Is Greater than 3,
Search in the Right Subtree of 3

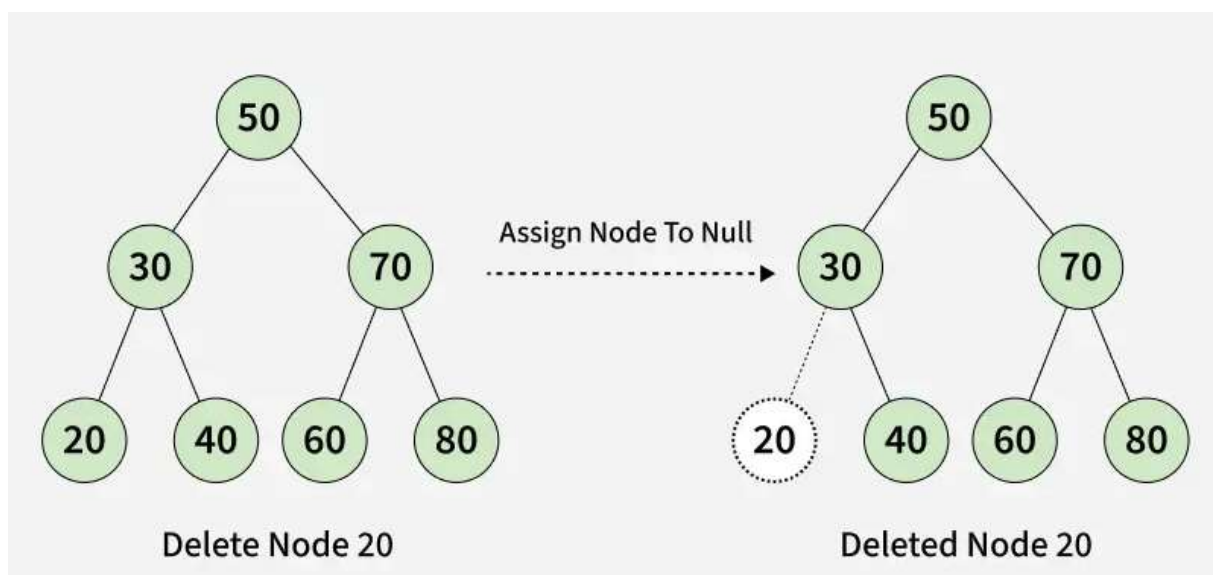


04 Step | As 6 Is Equal To Key (6),
So we have found the Key



➤ Deletion in Binary Search Tree (BST)

- Given a BST, the task is to delete a node in this BST, which can be broken down into 3 scenarios:
- Case 1.** Delete a Leaf Node in BST

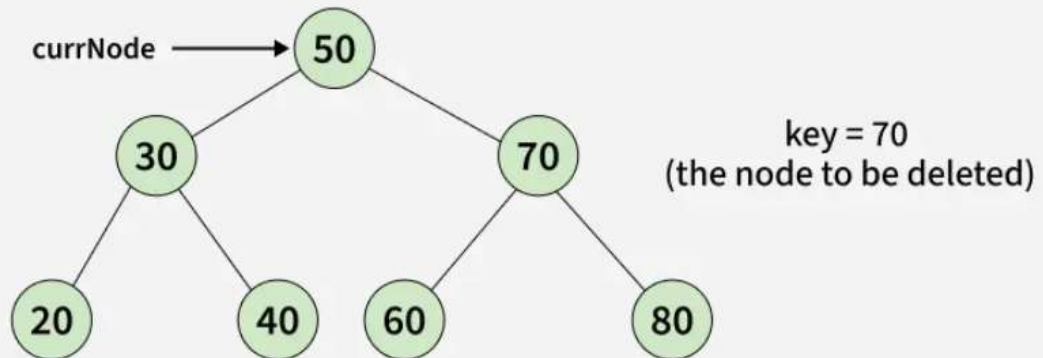


- Case 2.** Delete a Node with child in BST
 - Deleting a single child node is also simple in BST. Copy the child to the node and delete the node.
 - Deleting a node with both children is not so simple. Here we have to delete the node in such a way, that the resulting tree follows the properties of a BST.

- Deleted node will be replaced with inorder successor.

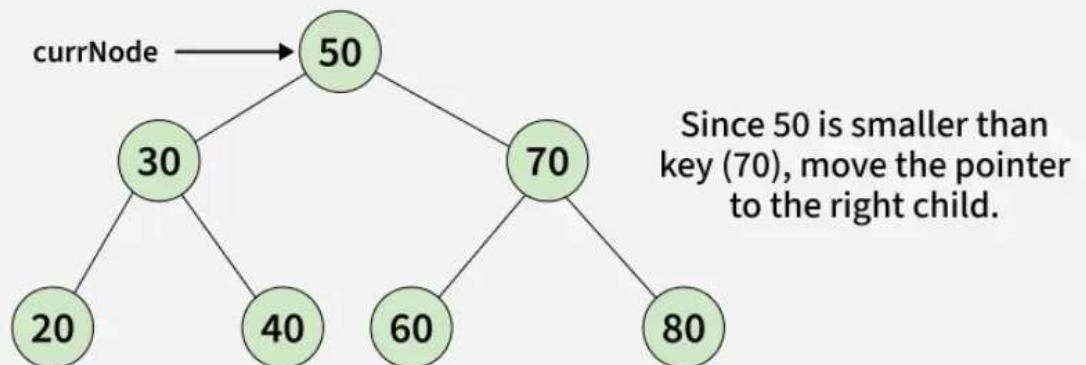
01
Step

Consider the following BST



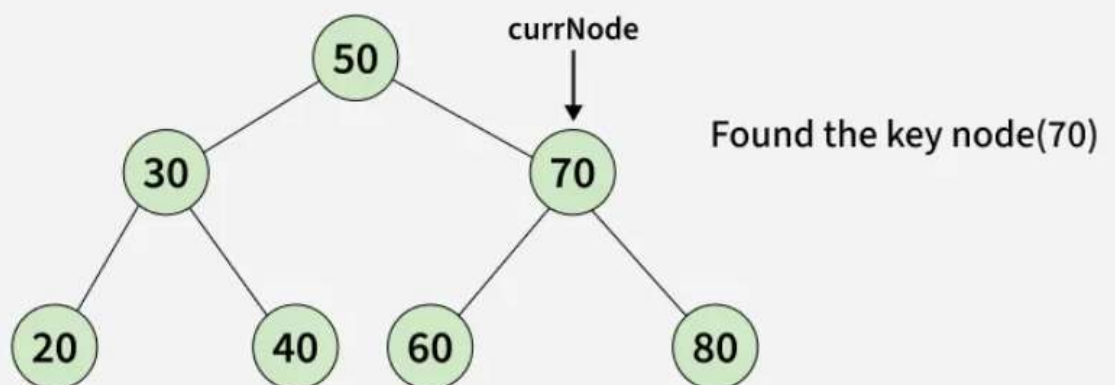
02
Step

Comparing key with root node



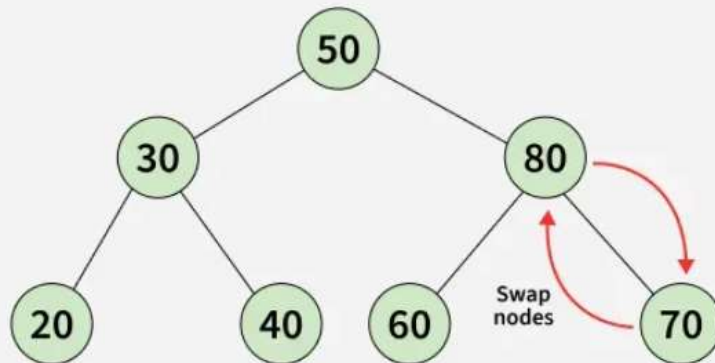
03
Step

Comparing key with right child root node



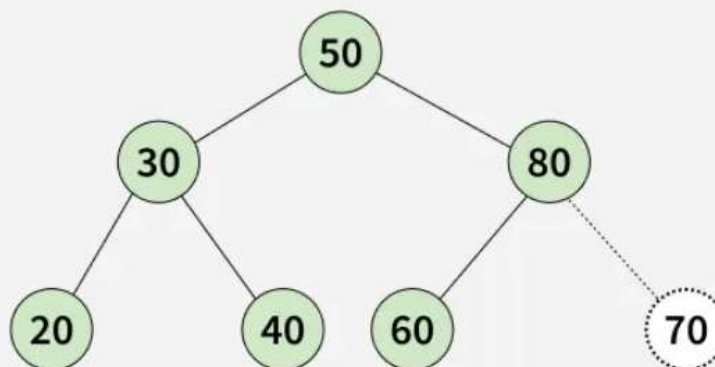
04
Step

Replace key with 80



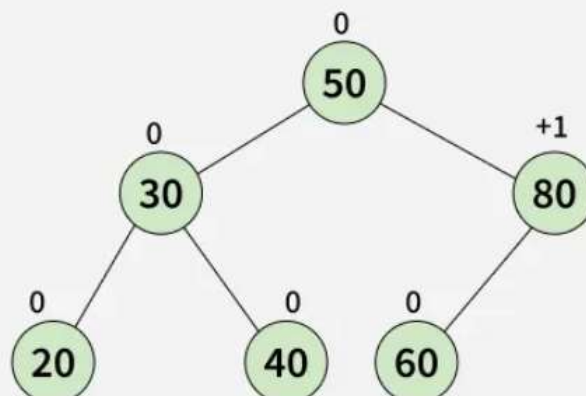
05
Step

Delete the key node (70)



06
Step

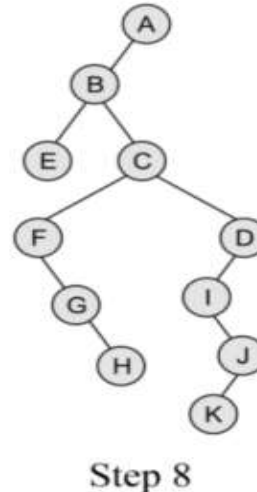
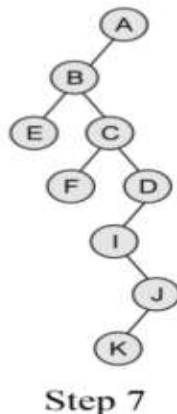
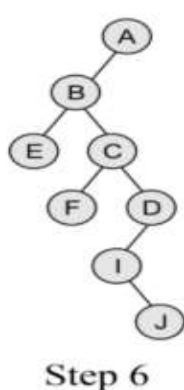
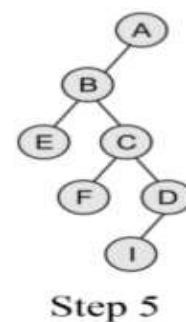
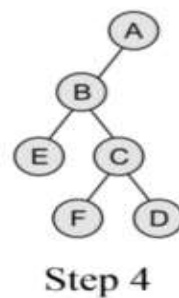
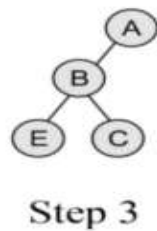
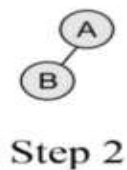
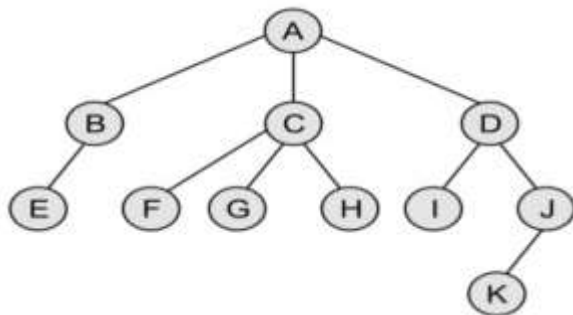
Final BST



Conversion of General Trees To Binary Trees

➤ Convert a Generic Tree(N-array Tree) to Binary Tree

- Following are the rules to convert a Generic(N-array Tree) to a Binary Tree:
- The root of the Binary Tree is the Root of the Generic Tree.
- The left child of a node in the Generic Tree is the Left child of that node in the Binary Tree.
- The right sibling of any node in the Generic Tree is the Right child of that node in the Binary Tree.
- **Example : Convert given N-array Tree to Binary Tree**

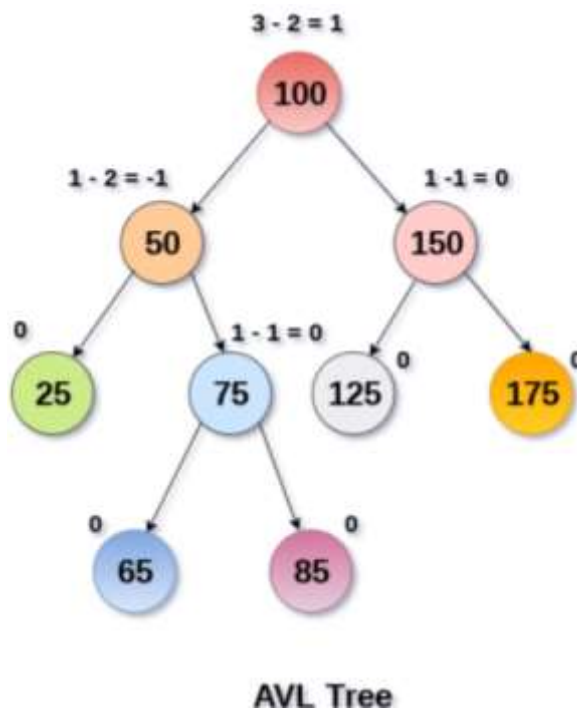


AVL Tree

- AVL Tree is invented by GM Adelson - Velsky and EM Landis in 1962. The tree is named AVL in honour of its inventors.
- AVL Tree can be defined as height balanced binary search tree in which each node is associated with a balance factor which is calculated by subtracting the height of its right sub-tree from that of its left sub-tree.
- Tree is said to be balanced if balance factor of each node is in between -1 to 1, otherwise, the tree will be unbalanced and need to be balanced.

$$\text{Balance Factor (k)} = \text{height (left(k))} - \text{height (right(k))}$$

- If balance factor of any node is 1, it means that the left sub-tree is one level higher than the right sub-tree.
- If balance factor of any node is 0, it means that the left sub-tree and right sub-tree contain equal height.
- If balance factor of any node is -1, it means that the left sub-tree is one level lower than the right sub-tree.
- An AVL tree is given in the following figure. We can see that, balance factor associated with each node is in between -1 and +1. therefore, it is an example of AVL tree.



- **Operations on AVL tree**

- Due to the fact that, AVL tree is also a binary search tree therefore, all the operations are performed in the same way as they are performed in a binary search tree. Searching and traversing do not lead to the violation in property of AVL tree. However, insertion and deletion are the operations which can violate this property and therefore, they need to be revisited.

1. Insertion: Insertion in AVL tree is performed in the same way as it is performed in a binary search tree. However, it may lead to violation in the AVL tree property and therefore the tree may need balancing. The tree can be balanced by applying rotations.

2. Deletion: Deletion can also be performed in the same way as it is performed in a binary search tree. Deletion may also disturb the balance of the tree therefore, various types of rotations are used to rebalance the tree.

- **Why AVL Tree?**

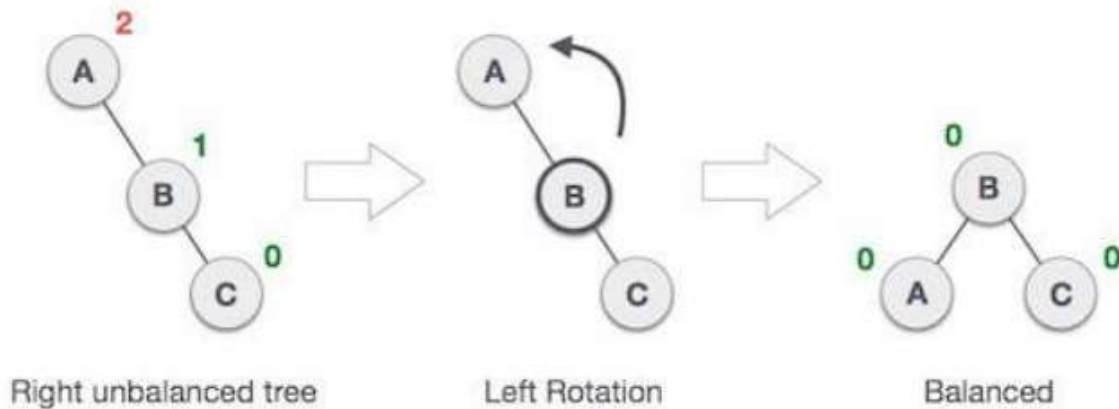
- AVL tree controls the height of the binary search tree by not letting it to be skewed. The time taken for all operations in a binary search tree of height h is $O(h)$. However, it can be extended to $O(n)$ if the BST becomes skewed (i.e. worst case). By limiting this height to $\log n$, AVL tree imposes an upper bound on each operation to be $O(\log n)$ where n is the number of nodes

- **AVL Rotations**

- We perform rotation in AVL tree only in case if Balance Factor is other than **-1, 0, and 1**. There are basically four types of rotations which are as follows:
 1. L L rotation: Inserted node is in the left subtree of left subtree of A
 2. R R rotation : Inserted node is in the right subtree of right subtree of A
 3. L R rotation : Inserted node is in the right subtree of left subtree of A
 4. R L rotation : Inserted node is in the left subtree of right subtree of A
- Where node A is the node whose balance Factor is other than -1, 0, 1.
- The first two rotations LL and RR are single rotations and the next two rotations LR and RL are double rotations. For a tree to be unbalanced, minimum height must be at least 2, Let us understand each rotation

1. RR Rotation

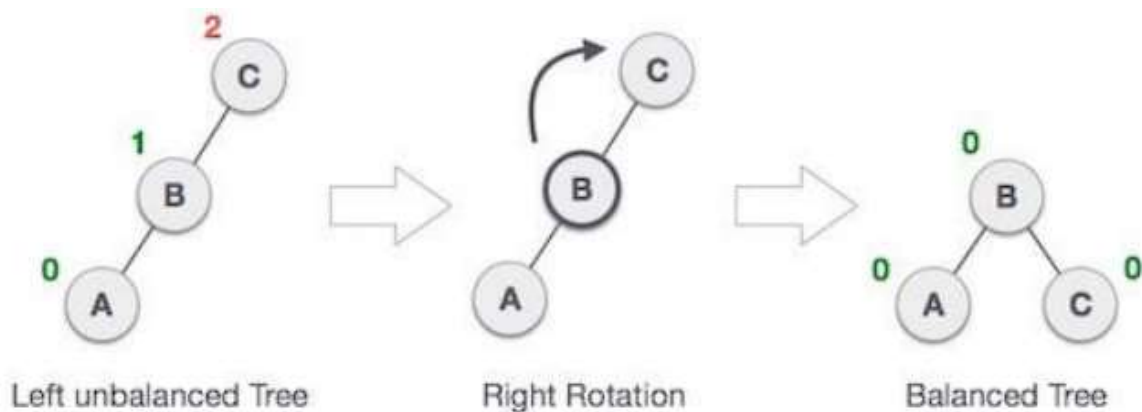
- When BST becomes unbalanced, due to a node is inserted into the right subtree of the right subtree of A, then we perform RR rotation.
- RR rotation is an anticlockwise rotation, which is applied on the edge below a node having balance factor -2



- In above example, node A has balance factor -2 because a node C is inserted in the right subtree of A right subtree. We perform the RR rotation on the edge below A.

2. LL Rotation

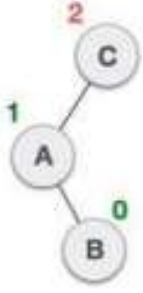
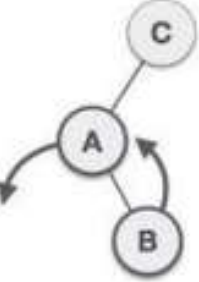
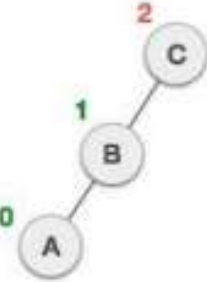
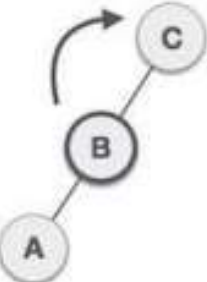
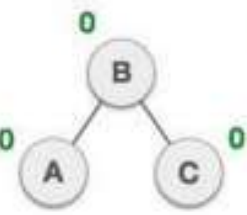
- When BST becomes unbalanced, due to a node is inserted into the left subtree of the left subtree of C, then we perform LL rotation, LL rotation is clockwise rotation, which is applied on the edge below a node having balance factor 2



- In above example, node C has balance factor 2 because a node A is inserted in the left subtree of C left subtree. We perform the LL rotation on the edge below A.

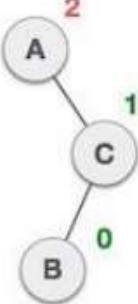
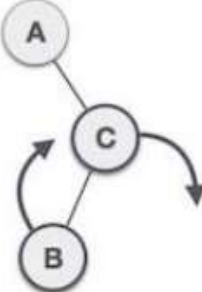
3. LR Rotation

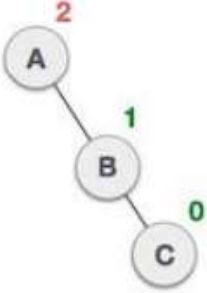
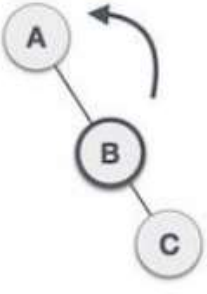
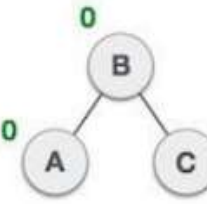
- Double rotations are bit tougher than single rotation which has already explained above. LR rotation = RR rotation + LL rotation, i.e., first RR rotation is performed on subtree and then LL rotation is performed on full tree, by full tree we mean the first node from the path of inserted node whose balance factor is other than -1, 0, or 1.
- Let us understand each and every step very clearly:

State	Action
	A node B has been inserted into the right subtree of A the left subtree of C, because of which C has become an unbalanced node having balance factor 2. This case is L R rotation where: Inserted node is in the right subtree of left subtree of C
	As LR rotation = RR + LL rotation, hence RR (anticlockwise) on subtree rooted at A is performed first. By doing RR rotation, node A, has become the left subtree of B.
	After performing RR rotation, node C is still unbalanced, i.e., having balance factor 2, as inserted node A is in the left of left of C
	Now we perform LL clockwise rotation on full tree, i.e. on node C. node C has now become the right subtree of node B, A is left subtree of B
	Balance factor of each node is now either -1, 0, or 1, i.e. BST is balanced now.

4. RL Rotation

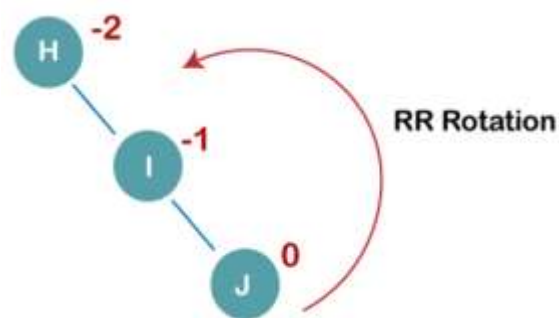
- As already discussed, that double rotations are bit tougher than single rotation which has already explained above. R L rotation = LL rotation + RR rotation, i.e., first LL rotation is performed on subtree and then RR rotation is performed on full tree, by full tree we mean the first node from the path of inserted node whose balance factor is other than -1, 0, or 1.

State	Action
	A node B has been inserted into the left subtree of C the right subtree of A, because of which A has become an unbalanced node having balance factor - 2. This case is RL rotation where: Inserted node is in the left subtree of right subtree of A
	As RL rotation = LL rotation + RR rotation, hence, LL (clockwise) on subtree rooted at C is performed first. By doing RR rotation, node C has become the right subtree of B.

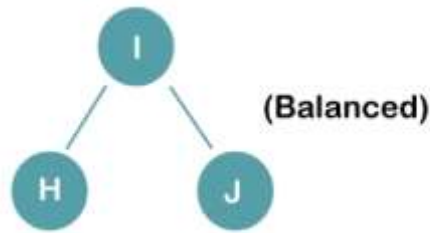
	After performing LL rotation, node A is still unbalanced, i.e. having balance factor -2, which is because of the right-subtree of the right-subtree node A.
	Now we perform RR rotation (anticlockwise rotation) on full tree, i.e. on node A. node C has now become the right subtree of node B, and node A has become the left subtree of B.
	Balance factor of each node is now either -1, 0, or 1, i.e., BST is balanced now.

- **Example :** Construct an AVL tree having the following elements
H, I, J, B, A, E, C, F, D, G, K, L

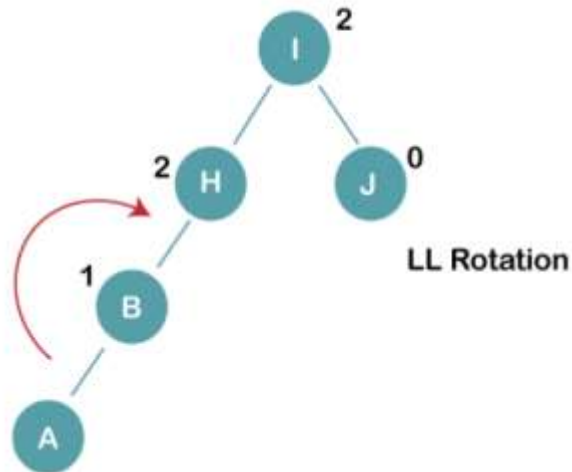
1. Insert H, I, J



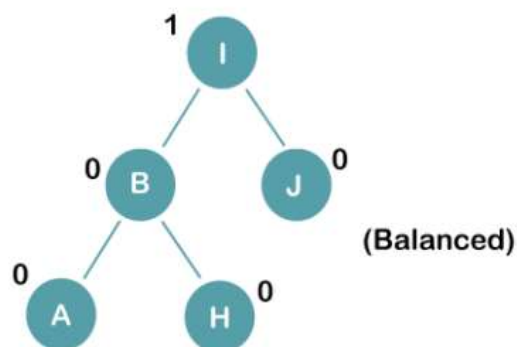
- On inserting the above elements, especially in the case of H, the BST becomes unbalanced as the Balance Factor of H is -2. Since the BST is right-skewed, we will perform RR Rotation on node H.
- The resultant balance tree is:



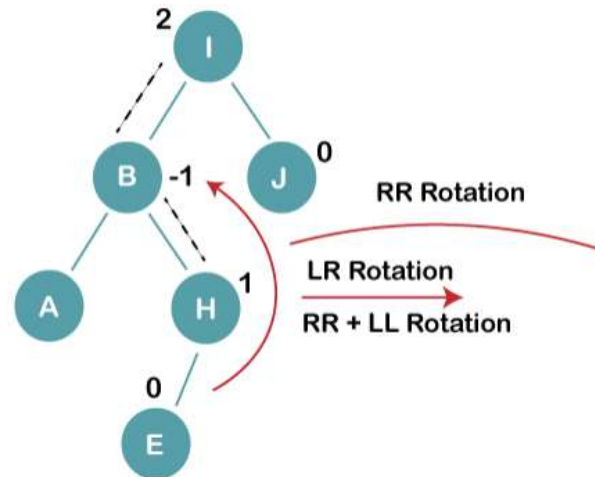
2. Insert B, A



- On inserting the above elements, especially in case of A, the BST becomes unbalanced as the Balance Factor of H and I is 2, we consider the first node from the last inserted node i.e. H. Since the BST from H is left-skewed, we will perform LL Rotation on node H.
- The resultant balance tree is:

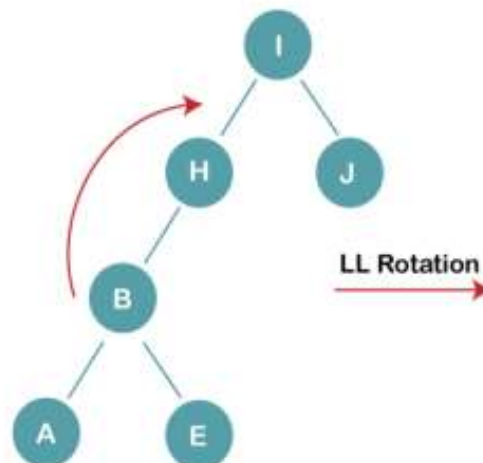


3. Insert E

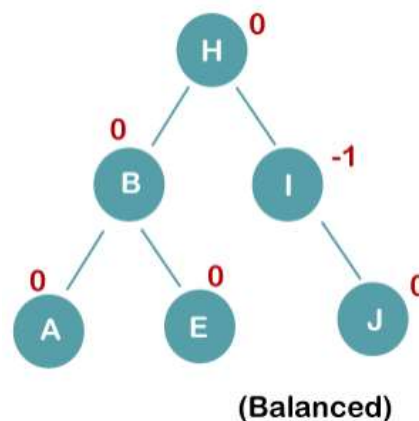


- On inserting E, BST becomes unbalanced as the Balance Factor of I is 2, since if we travel from E to I we find that it is inserted in the left subtree of right subtree of I, we will perform LR Rotation on node I. LR = RR + LL rotation.

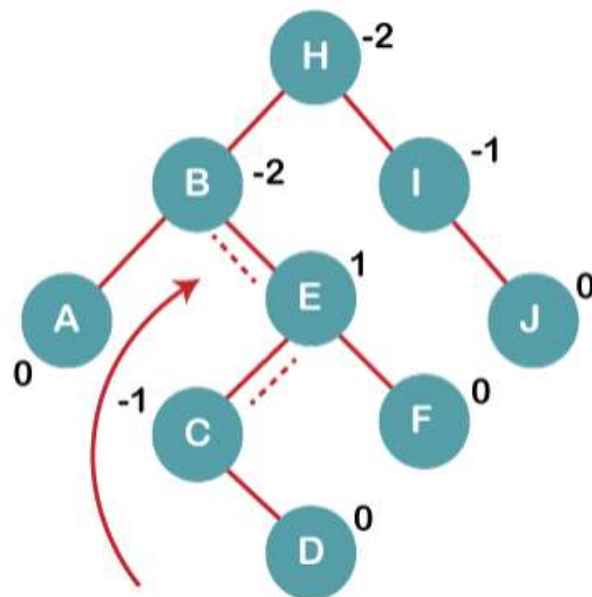
3 a) We first perform RR rotation on node B The resultant tree after RR rotation is:



3 b) We first perform LL rotation on the node I The resultant balanced tree after LL rotation is:

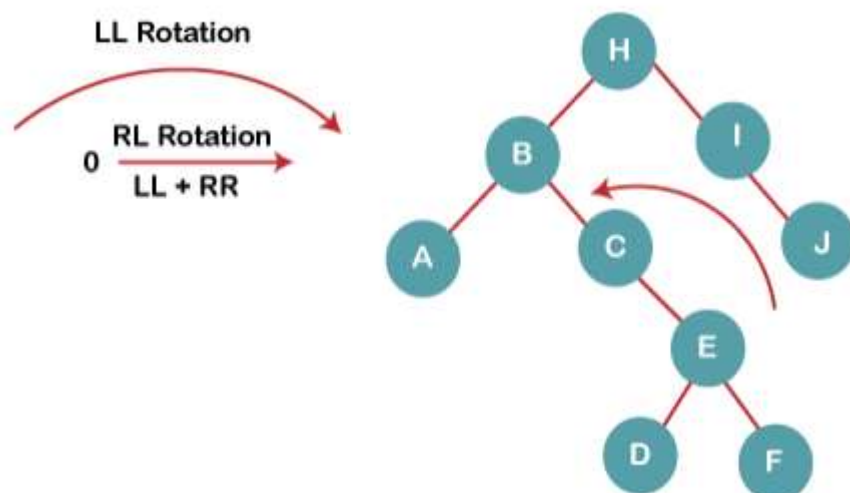


4. Insert C, F, D

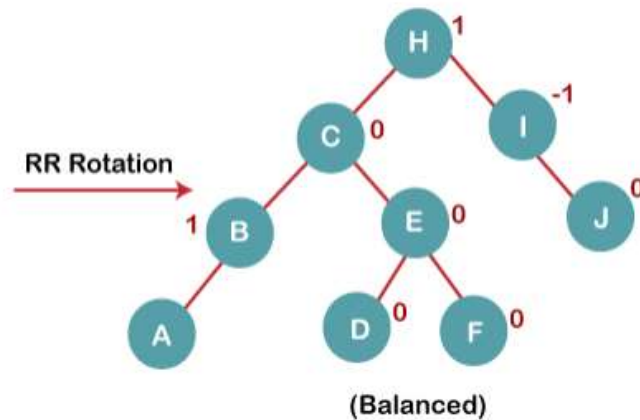


- On inserting C, F, D, BST becomes unbalanced as the Balance Factor of B and H is -2, since if we travel from D to B we find that it is inserted in the right subtree of left subtree of B, we will perform RL Rotation on node I. RL = LL + RR rotation.

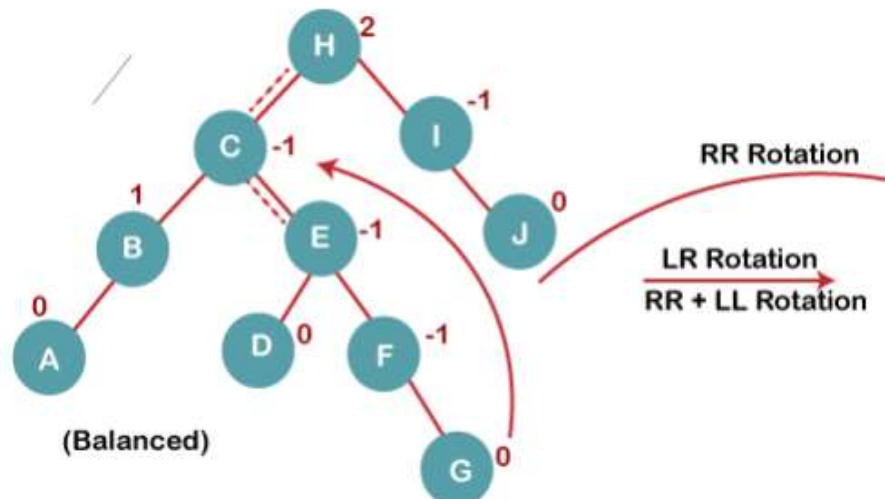
4 a) We first perform LL rotation on node E The resultant tree after LL rotation is:



4 b) We then perform RR rotation on node B The resultant balanced tree after RR rotation is:

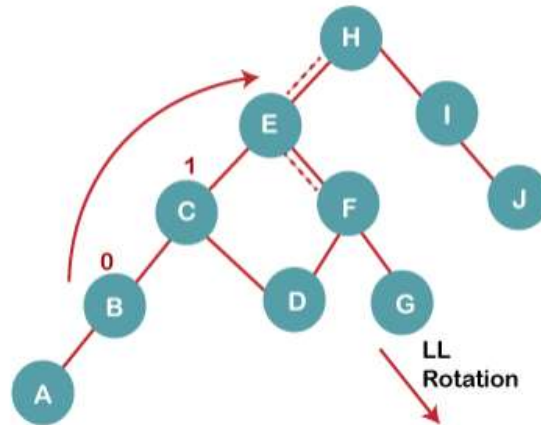


5. Insert G

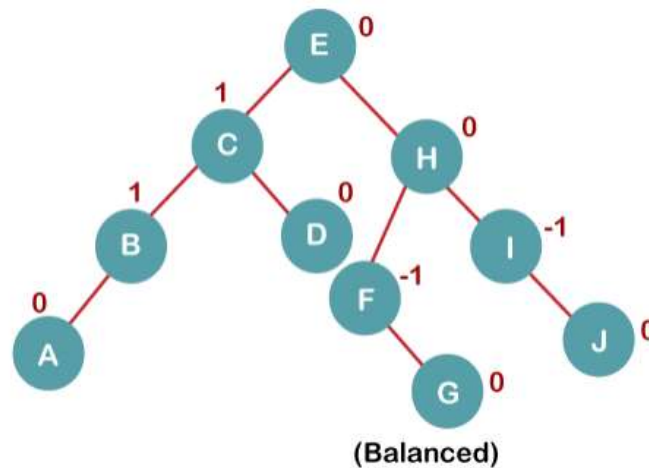


- On inserting G, BST become unbalanced as the Balance Factor of H is 2, since if we travel from G to H, we find that it is inserted in the left subtree of right subtree of H, we will perform LR Rotation on node I. LR = RR + LL rotation.

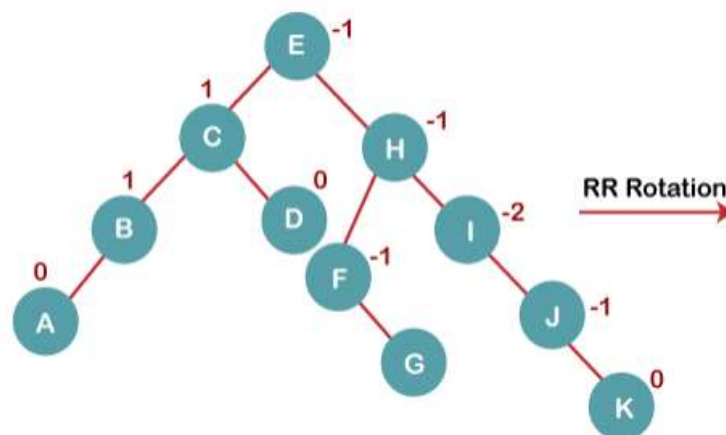
5 a) We first perform RR rotation on node C The resultant tree after RR rotation is:



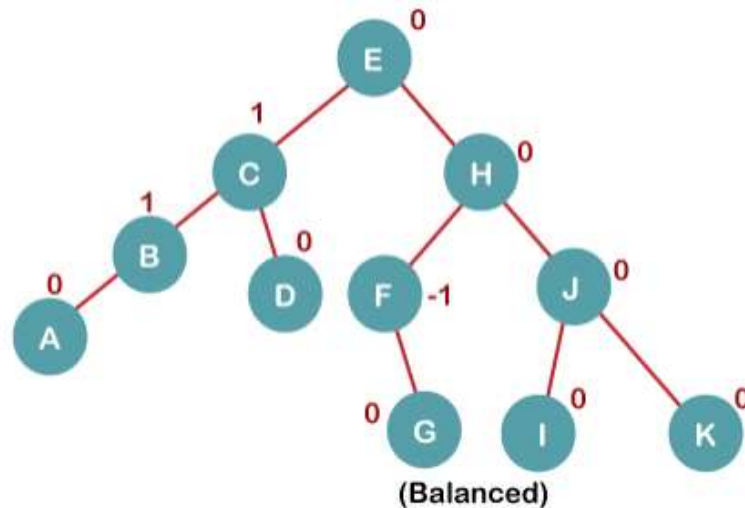
5 b) We then perform LL rotation on node H. The resultant balanced tree after LL rotation is:



6. Insert K

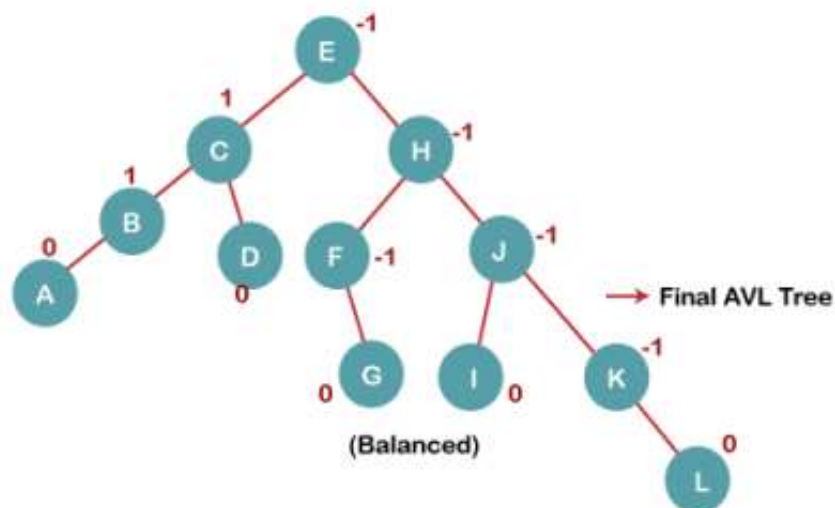


- On inserting K, BST becomes unbalanced as the Balance Factor of I is -2. Since the BST is right-skewed from I to K, hence we will perform RR Rotation on the node I.
- The resultant balanced tree after RR rotation is:



5. Insert L

- On inserting the L tree is still balanced as the Balance Factor of each node is now either, -1, 0, +1. Hence the tree is a Balanced AVL tree

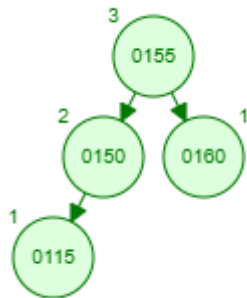


**Example 2: Construct AVL Tree on Given Data.
150,155,160,115,110,140,120,145,130,147,170,180**

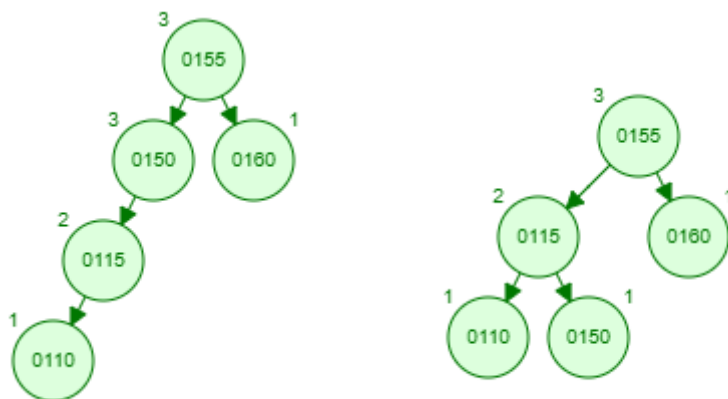
Insert 150,155,160



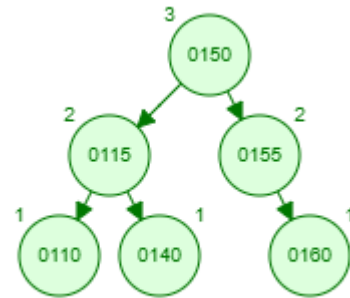
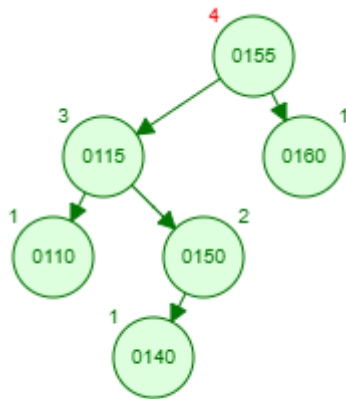
Insert 115



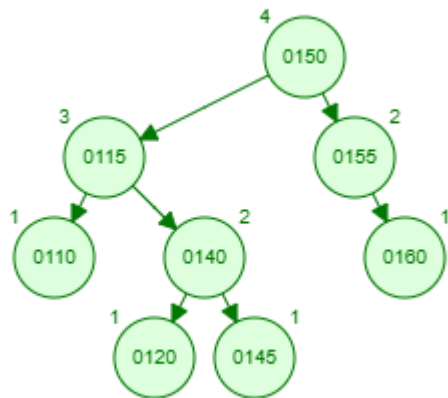
Insert 110



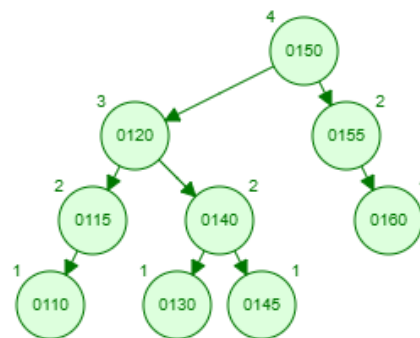
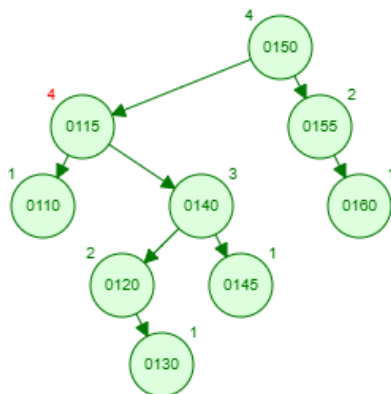
Insert 140



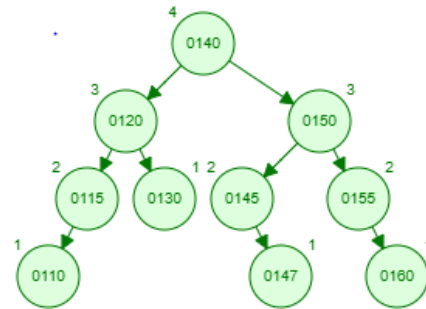
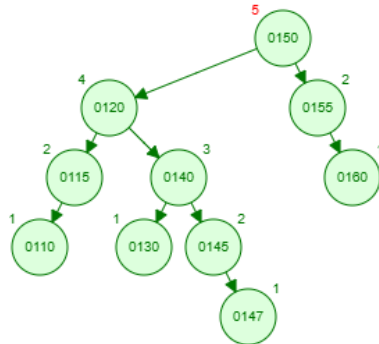
Insert 120,145



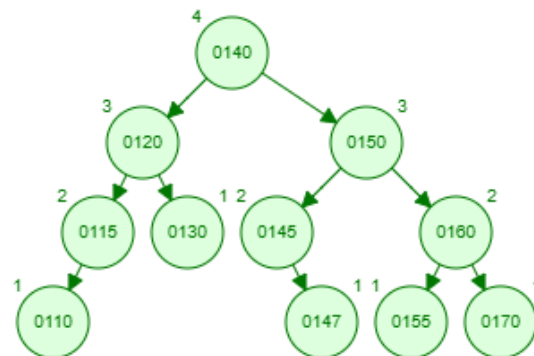
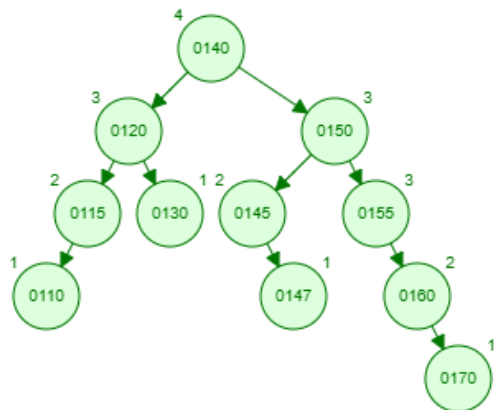
Insert 130



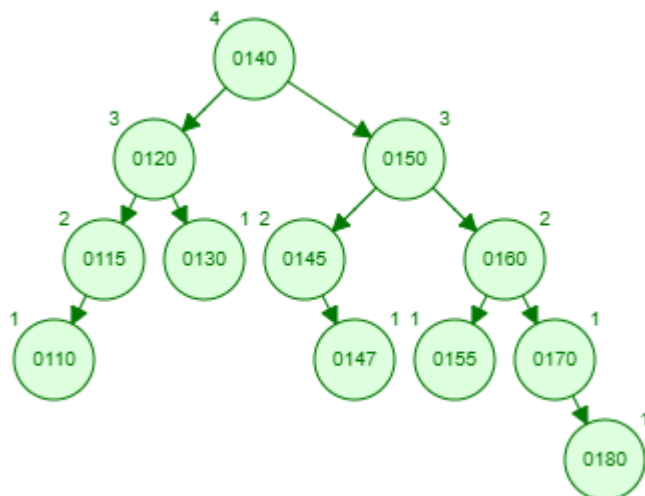
Insert 147



Insert 170



Insert 180



[Final Resultant Tree]

B-Trees/Multi way Search Tree

- A B-Tree is a self-balancing search tree in which nodes can have multiple children (not just two like in binary trees). It's especially designed to work well on storage systems like disks, where reading large blocks of data at once is efficient.
- It is widely used in Databases and in File Systems.
- One of the standout features of a B-Tree is its ability to store a significant number of keys within a single node, including large key values. It significantly reduces the tree's height, hence reducing costly disk operations.
- B Trees allow faster data retrieval and updates, making them an ideal choice for systems requiring efficient and scalable data management. By maintaining a balanced structure at all times,
- In a B-Tree of order m will be given.

➤ Properties of B-Tree :

- Each node can have up to m children and m-1 keys.
- The Key in each node are in ascending order.
- Root node should have at least two children.
- All the internal node except root node have at least $\text{ceil}(m/2)$ non-empty children.
- Each leaf node must contain at least $\text{ceil}(m/2)-1$ keys.

➤ Example: Construct B-Tree of Order 3 by Inserting numbers from 1 to 10.

insert(1)

Since '1' is the first element into the tree that is inserted into a new node. It acts as the root node.



insert(2)

Element '2' is added to existing leaf node. Here, we have only one node and that node acts as root and also leaf. This leaf node has an empty position. So, new element (2) can be inserted at that empty position.



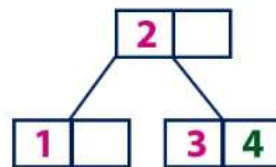
insert(3)

Element '3' is added to existing leaf node. Here, we have only one node and that node acts as root and also leaf. This leaf node doesn't have an empty position. So, we split that node by sending middle value (2) to its parent node. But here, this node doesn't have a parent. So, this middle value becomes a new root node for the tree.



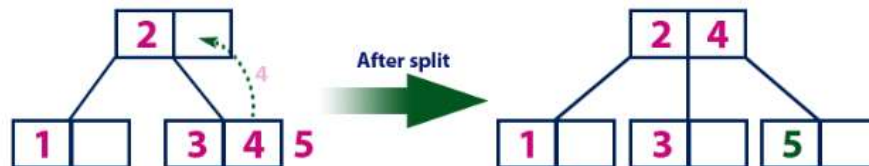
insert(4)

Element '4' is larger than root node '2' and it is not a leaf node. So, we move to the right of '2'. We reach to a leaf node with value '3' and it has an empty position. So, new element (4) can be inserted at that empty position.



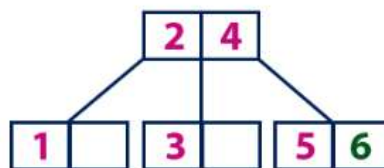
insert(5)

Element '5' is larger than root node '2' and it is not a leaf node. So, we move to the right of '2'. We reach to a leaf node and it is already full. So, we split that node by sending middle value (4) to its parent node (2). There is an empty position in its parent node. So, value '4' is added to node with value '2' and new element '5' added as new leaf node.



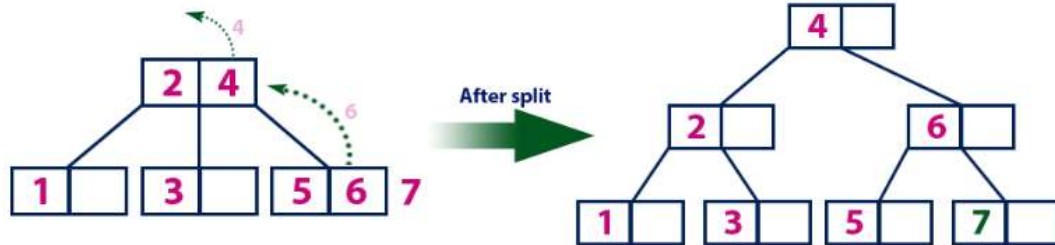
insert(6)

Element '6' is larger than root node '2' & '4' and it is not a leaf node. So, we move to the right of '4'. We reach to a leaf node with value '5' and it has an empty position. So, new element (6) can be inserted at that empty position.



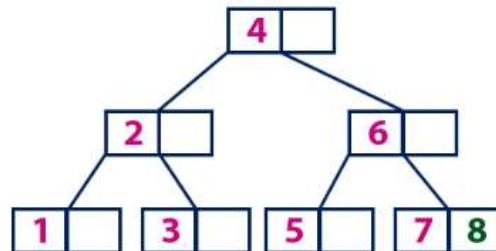
insert(7)

Element '7' is larger than root node '2' & '4' and it is not a leaf node. So, we move to the right of '4'. We reach to a leaf node and it is already full. So, we split that node by sending middle value (6) to its parent node (2&4). But the parent (2&4) is also full. So, again we split the node (2&4) by sending middle value '4' to its parent but this node doesn't have parent. So, the element '4' becomes new root node for the tree.



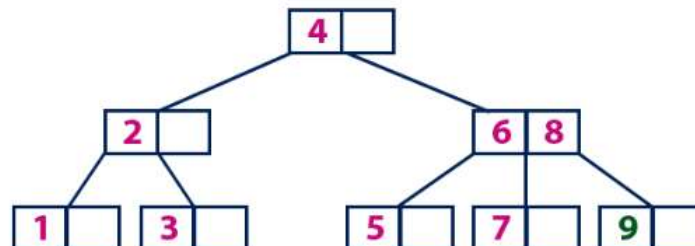
insert(8)

Element '8' is larger than root node '4' and it is not a leaf node. So, we move to the right of '4'. We reach to a node with value '6'. '8' is larger than '6' and it is also not a leaf node. So, we move to the right of '6'. We reach to a leaf node (7) and it has an empty position. So, new element (8) can be inserted at that empty position.



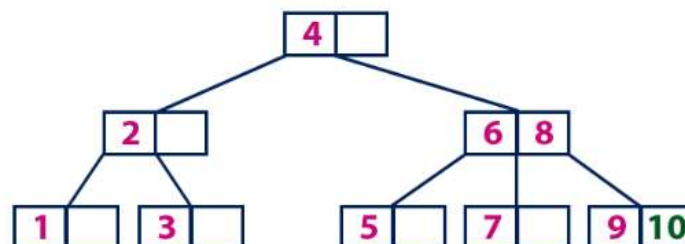
insert(9)

Element '9' is larger than root node '4' and it is not a leaf node. So, we move to the right of '4'. We reach to a node with value '6'. '9' is larger than '6' and it is also not a leaf node. So, we move to the right of '6'. We reach to a leaf node (7 & 8). This leaf node is already full. So, we split this node by sending middle value (8) to its parent node. The parent node (6) has an empty position. So, '8' is added at that position. And new element is added as a new leaf node.



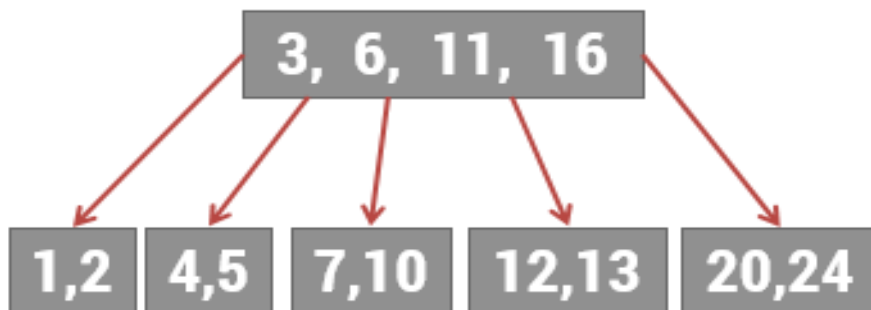
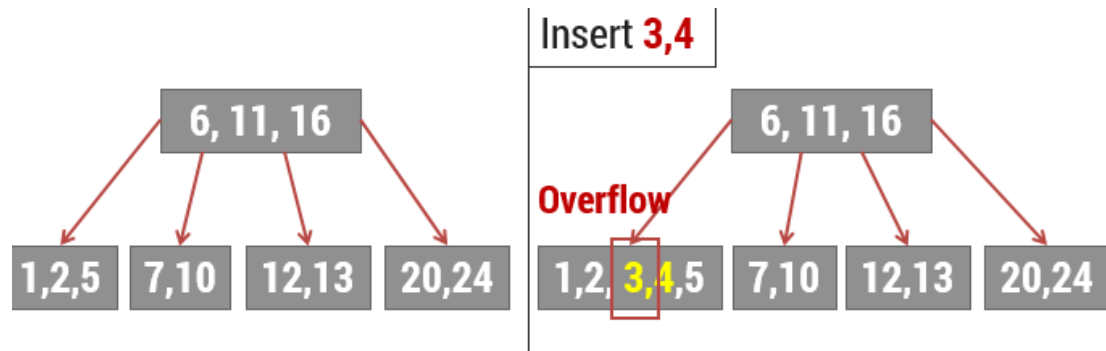
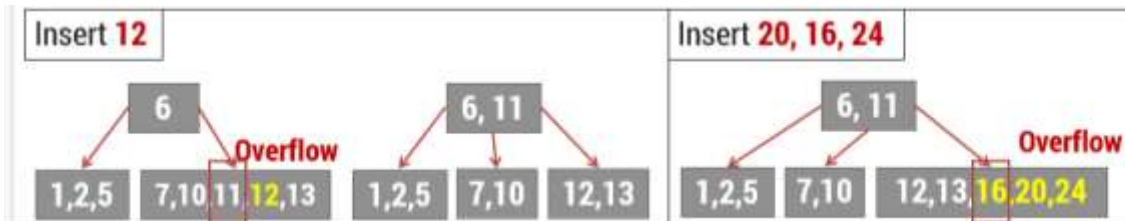
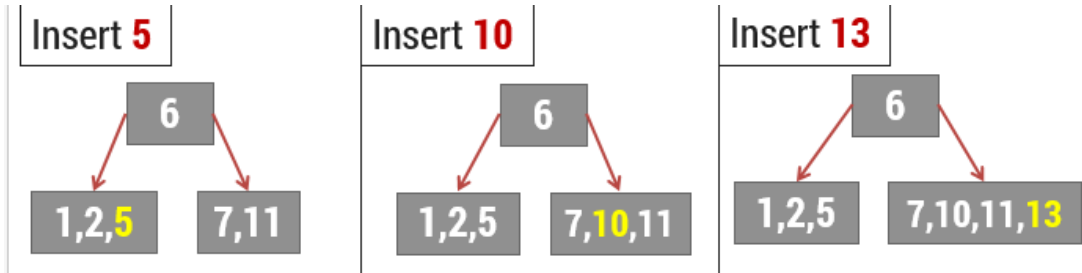
insert(10)

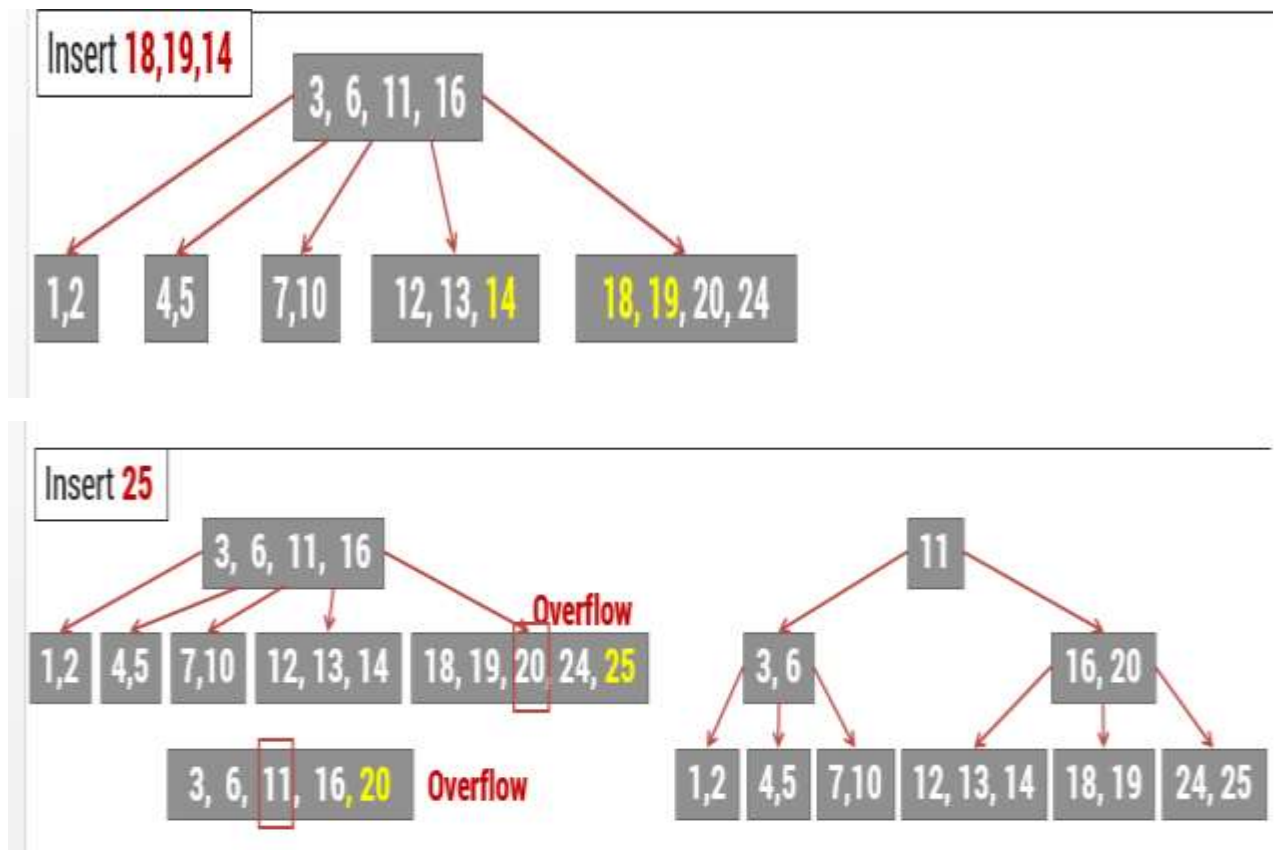
Element '10' is larger than root node '4' and it is not a leaf node. So, we move to the right of '4'. We reach to a node with values '6 & 8'. '10' is larger than '6 & 8' and it is also not a leaf node. So, we move to the right of '8'. We reach to a leaf node (9). This leaf node has an empty position. So, new element '10' is added at that empty position.



Example 2: Construct 5 order B-Tree from following Data

1,7,6,2,11,5,10,13,12,20,16,24,3,4,18,19,14,25





Deletion in B-Tress

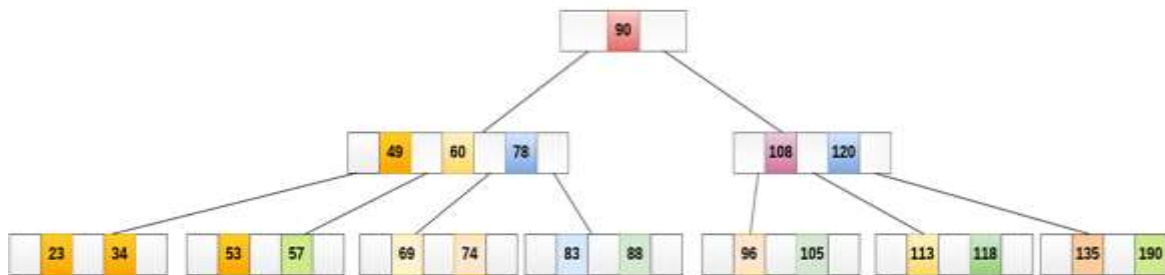
Deletion is also performed at the leaf nodes. The node which is to be deleted can either be a leaf node or an internal node. Following algorithm needs to be followed in order to delete a node from a B tree.

1. Locate the leaf node.
2. If there are more than $m/2$ keys in the leaf node then delete the desired key from the node.
3. If the leaf node doesn't contain $m/2$ keys then complete the keys by taking the element from right or left sibling.
 - If the left sibling contains more than $m/2$ elements then push its largest element up to its parent and move the intervening element down to the node where the key is deleted.
 - If the right sibling contains more than $m/2$ elements then push its smallest element up to the parent and move intervening element down to the node where the key is deleted.
4. If neither of the sibling contain more than $m/2$ elements then create a new leaf node by joining two leaf nodes and the intervening element of the parent node.

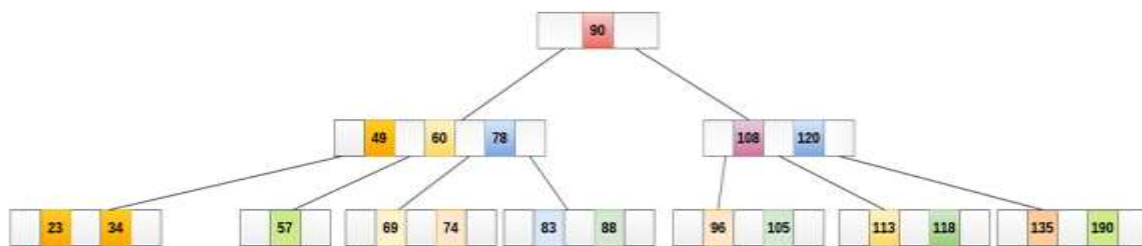
5. If parent is left with less than $m/2$ nodes then, apply the above process on the parent too.

Example:

Delete the node 53 from the B Tree of order 5 shown in the following figure.

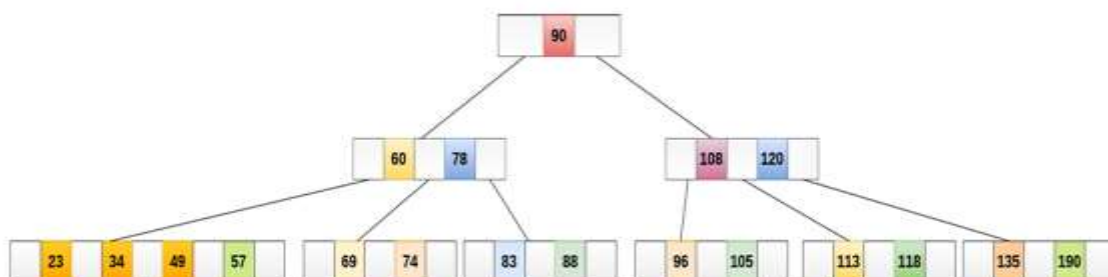


53 is present in the right child of element 49. Delete it.



Now, 57 is the only element which is left in the node, the minimum number of elements that must be present in a B tree of order 5, is 2. it is less than that, the elements in its left and right sub-tree are also not sufficient therefore, merge it with the left sibling and intervening element of parent i.e. 49.

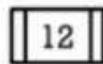
The final B tree is shown as follows.



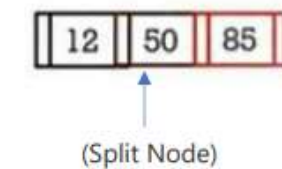
➤ 2-3 Trees:

- A 2-3 Tree is a specific form of a B tree. A 2-3 tree is a search tree. However, it is very different from a binary search tree.
- Here are the properties of a 2-3 tree:
 - each node has either one value or two value
 - a node with one value is either a leaf node or has exactly two children (non-null). Values in left subtree < value in node < values in right subtree
 - a node with two values is either a leaf node or has exactly three children (non-null). Values in left subtree < first value in node < values in middle subtree < second value in node < value in right subtree.
 - all leaf nodes are at the same level of the tree

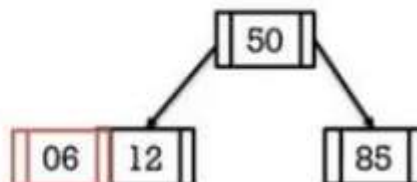
- 12, 50, 85, 06, 10, 37, 100, 120, 25, 70



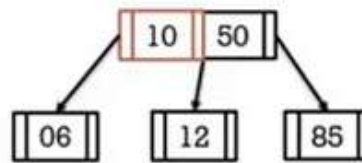
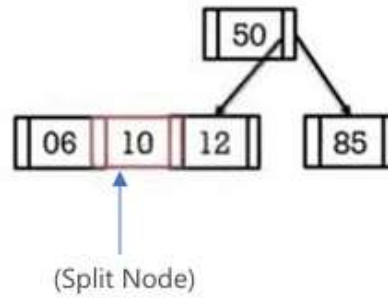
- 12, 50, 85, 06, 10, 37, 100, 120, 25, 70



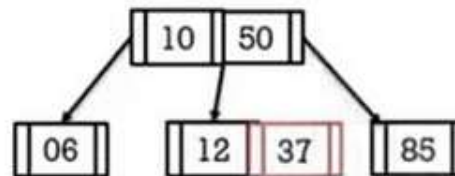
- 12, 50, 85, 06, 10, 37, 100, 120, 25, 70



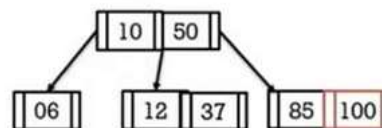
- 12, 50, 85, 06, 10, 37, 100, 120, 25, 70



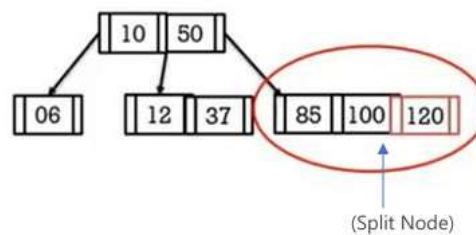
- 12, 50, 85, 06, 10, 37, 100, 120, 25, 70

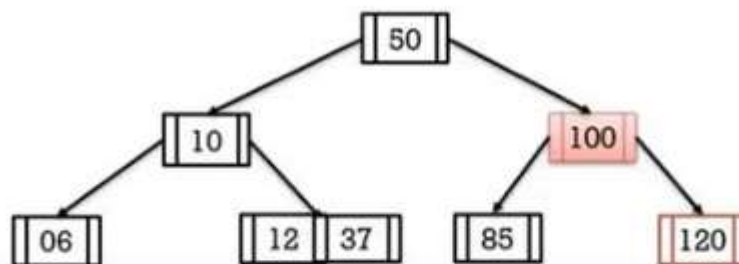
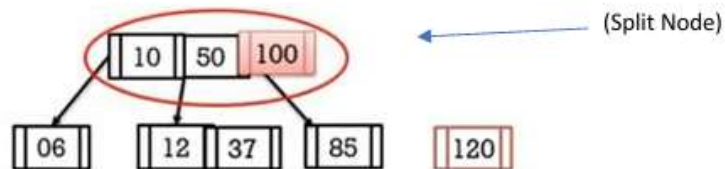
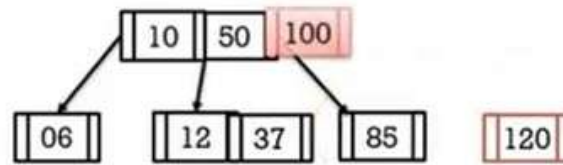


- 12, 50, 85, 06, 10, 37, 100, 120, 25, 70

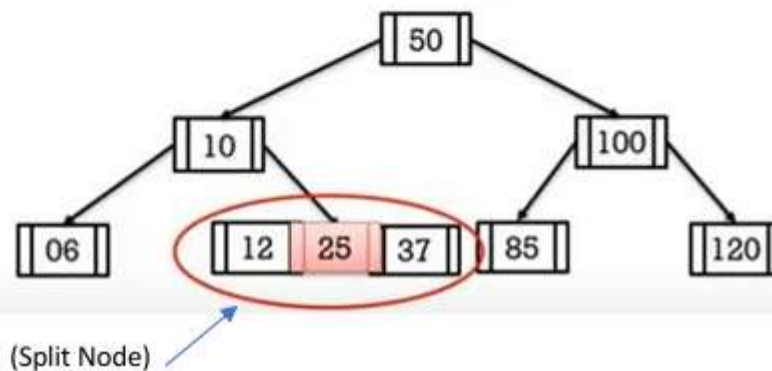


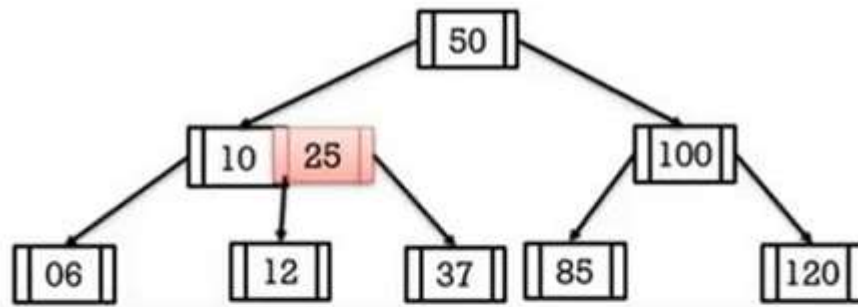
- 12, 50, 85, 06, 10, 37, 100, 120, 25, 70



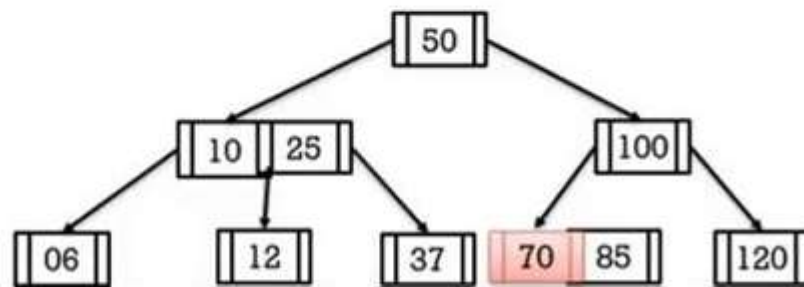


- 12, 50, 85, 06, 10, 37, 100, 120, 25, 70



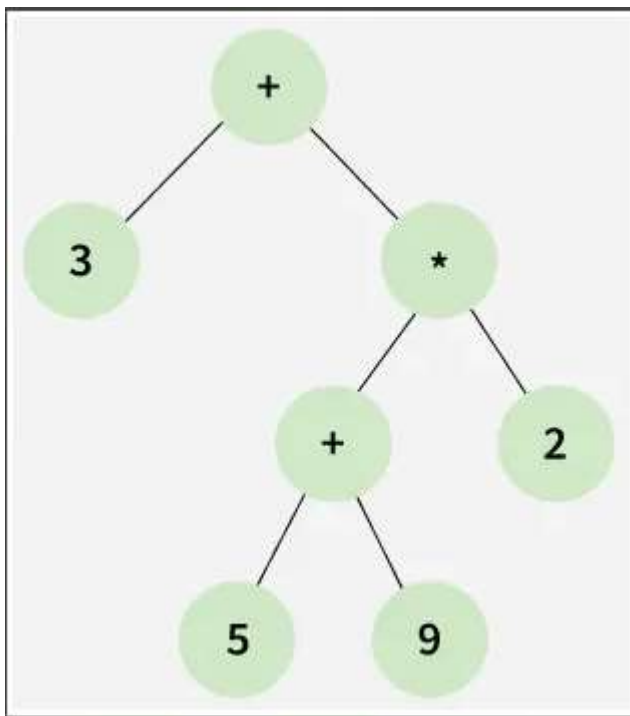


- 12, 50, 85, 06, 10, 37, 100, 120, 25, **70**

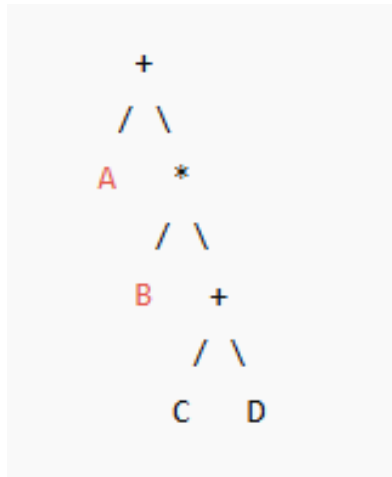


Expression Tree

Example 1: $3 + ((5 + 9) * 2)$

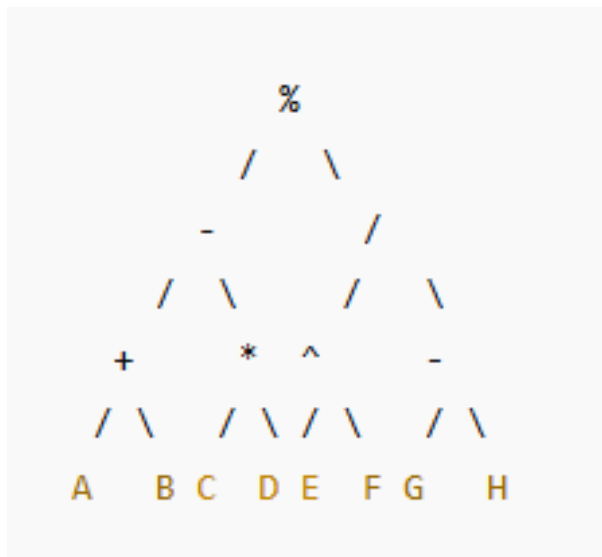


Example 2: $A+B*(C+D)$



Example 3: Construct expression tree of $((A+B)-(C*D))\%((E^F)/(G-H))$

Postfix Expression: $A\ B\ +\ C\ D\ *\ -\ E\ F\ ^\ G\ H\ -\ /\ %$



Applications Of Trees

➤ Uses of Tree Data Structure

Unlike Array and Linked List, which are linear data structures, tree is hierarchical (or non-linear) data structure.

- **Hierarchical Structure:** One reason to use trees might be because you want to store information that naturally forms a hierarchy. For example, the file system on a computer: The DOM model of an HTML page is also tree where we have html tag as root, head and body its children and these tags, then have their own children. DNS System is also example where we have natural hierarchy.
- **Searching Efficiency:** Trees provide an efficient way to search for data. For example, in a binary search tree, searching for a value takes time proportional to the logarithm of the number of elements, which is much faster than searching in a linear data structure like an array or a linked list.
- **Sorting:** Trees can be used to sort data efficiently. For example, in a self-balancing binary search tree, the data is automatically sorted as it is inserted into the tree, making it easy to find the minimum, maximum, and other values in the tree.
- **Dynamic Data:** Trees are dynamic data structures, which means that they can grow and shrink as needed. This makes them well-suited for applications where the data changes frequently, such as in real-time systems.
- **Efficient Insertion and Deletion:** Trees provide efficient algorithms for inserting and deleting data, which is important in many applications where data needs to be added or removed frequently.
- **Easy to Implement:** Trees are relatively easy to implement, especially when compared to other data structures like graphs. This makes them a popular choice for many programming projects.

➤ Other Applications of Tree Data Structure:

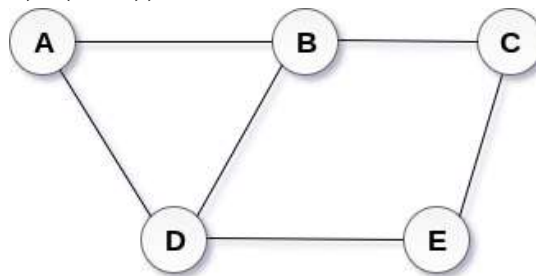
- Store hierarchical data, like folder structure, organization structure, XML/HTML data.
- Binary Search Tree is a tree that allows fast search, insert, delete on a sorted data. It also allows finding closest item
- Heap is a tree data structure which is implemented using arrays and used to implement priority queues.
- B-Tree and B+ Tree : They are used to implement indexing in databases.

- Syntax Tree: Scanning, parsing , generation of code and evaluation of arithmetic expressions in Compiler design.
- K-D Tree: A space partitioning tree used to organize points in K dimensional space.
- Trie : Used to implement dictionaries with prefix lookup.
- Suffix Tree : For quick pattern searching in a fixed text.
- Spanning Trees and shortest path trees are used in routers and bridges respectively in computer networks
- As a workflow for compositing digital images for visual effects.
- Decision trees.
- Organization chart of a large organization.
- In XML parser.
- Machine learning algorithm.
- For indexing in database.
- IN server like DNS (Domain Name Server)
- In Computer Graphics.
- To evaluate an expression.
- In chess game to store defence moves of player.
- In java virtual machine.
- Tree data structures are used to organize and manage files and directories in a file system. Each file and directory is represented as a node in the tree, with parent-child relationships indicating the hierarchical structure of the file system.
- Tree data structures are often used in parsing, such as in compilers and interpreters, to represent the structure of a program or a document.
- Tree data structures, such as binary search trees, are commonly used to implement efficient searching and sorting algorithms.
- Graphics and UI design
- Tree data structures are commonly used in decision-making algorithms in artificial intelligence, such as game-playing algorithms, expert systems, and decision trees.
- Tree data structures can be used to represent the topology of a network and to calculate routing tables for efficient data transmission.

Graph - Definitions and Concepts

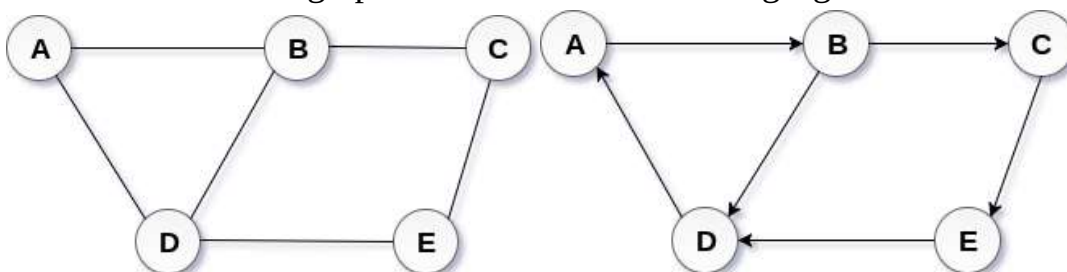
➤ What is Graph?

- A graph is a collection of nodes and edge
- Denoted by $G = (V, E)$.
- **nodes** (vertices, points).
- **Edges** (links, arcs) between pairs of nodes.
- A Graph $G(V, E)$ with 5 vertices (A, B, C, D, E) and six edges ((A,B), (B,C), (C,E), (E,D), (D,B), (D,A)) is shown in the following figure.



• **Directed and Undirected Graph**

- A graph can be directed or undirected. However, in an undirected graph, edges are not associated with the directions with them. An undirected graph is shown in the above figure since its edges are not attached with any of the directions. If an edge exists between vertex A and B then the vertices can be traversed from B to A as well as A to B.
- In a directed graph, edges form an ordered pair. Edges represent a specific path from some vertex A to another vertex B. Node A is called initial node while node B is called terminal node.
- A directed graph is shown in the following figure

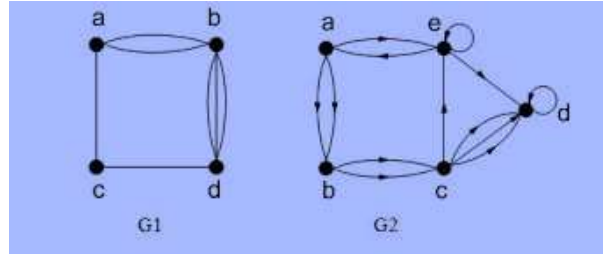


Undirected Graph

Directed Graph

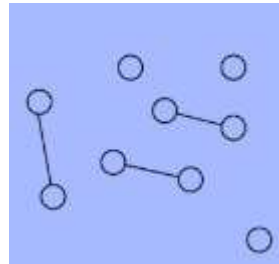
- **Multigraph:**

- A **multigraph** is a **graph** which is permitted to have **multiple** edges (also called parallel edges), that is, edges that have the same end nodes. Thus two vertices may be connected by more than one edge.



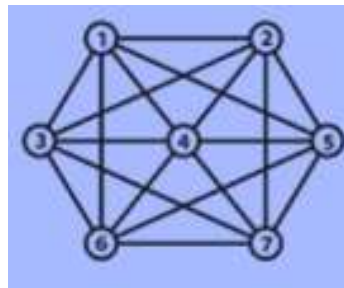
- **Sparse Graph:**

- Sparse graph is a graph in which the number of edges is close to the minimal number of edges. Sparse graph can be a disconnected graph.



- **Dense Graph:**

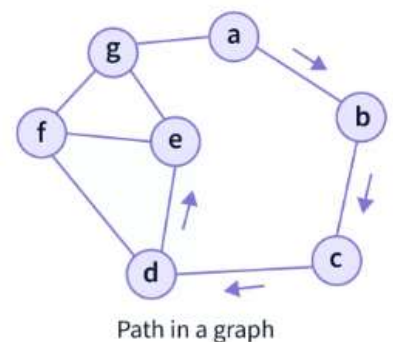
- A Dense graph is a graph in which the number of edges is close to the maximal number of edges.



➤ **Graph Terminology:**

- **Path**

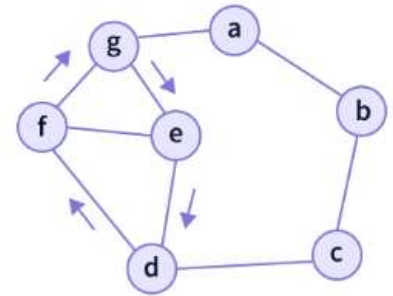
- A path can be defined as the sequence of nodes that are followed in order to reach some terminal node V from the initial node U.
- It can connect to 2 or more nodes. Also, if the path connects all the nodes of a graph in Data



Structure, then it is a connected graph, otherwise it is called a disconnected graph.

• Closed Path

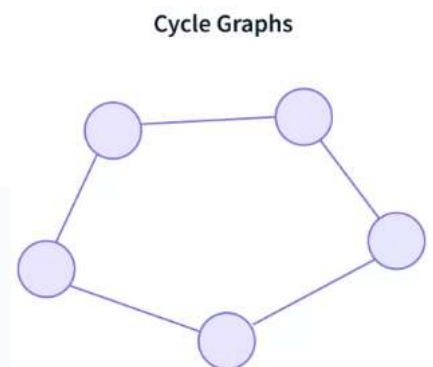
- A path is called as closed path if the initial node is same as terminal(end) node. A path will be closed path if $V_0V_0 = V_nV_n$, where V_0V_0 is the starting node of the graph and V_nV_n is the last node. So, the starting and the terminal nodes are same in a closed graph.



Closed Path in a graph

• Cycle

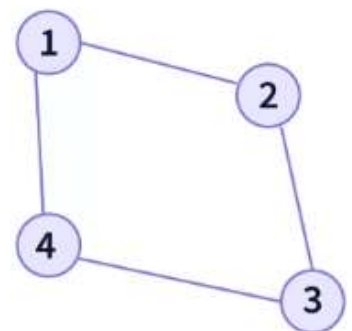
- A cycle can be defined as the path which has no repeated edges or vertices except the first and last vertices.
- A simple graph of 'n' **nodes(vertices)** ($n \geq 3$) and n edges forming a cycle of length 'n' is called as a cycle graph. In a cycle graph, all the vertices are of degree 2.



Cycle Graphs

• Connected Graph

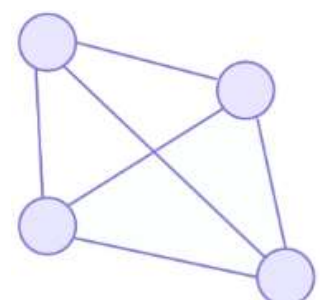
- A connected graph is the one in which some path exists between every two vertices (u, v) in V. There are no isolated nodes in connected graph.
- we can visit from any one vertex to any other vertex. There exists at least one path between every pair of vertices.
- There is not a single vertex in a connected graph, Which is unreachable(or isolated).



Connected Graph

• Complete Graph

- A complete graph is the one in which every node is connected with all other nodes. A complete graph contain $n(n-1)/2$ edges where n is the number of nodes in the graph.
- Here, every vertex has an edge to all other vertices.

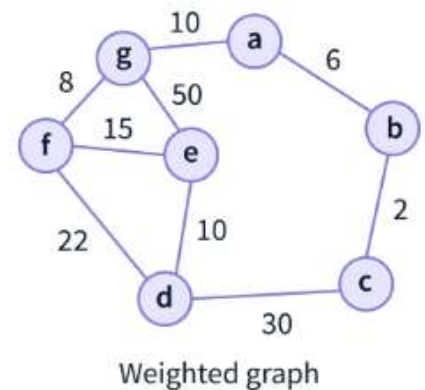


Complete Graph

It is also known as a full graph

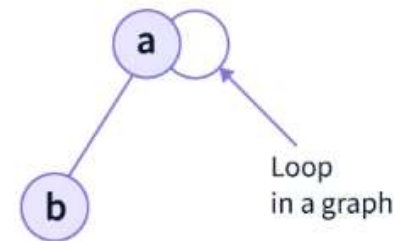
• Weighted Graph

- In a weighted graph, each edge is assigned with some data such as length or weight. The weight of an edge e can be given as $w(e)$ which must be a positive (+) value indicating the cost of traversing the edge.
- The weights may represent for example, any distance, or time, or the number of connections shared between two users in a social network.



• Loop

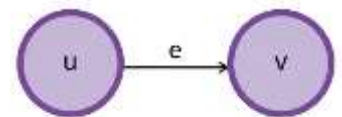
- An edge that is associated with the similar end points can be called as Loop.
- A loop also called a **self-loop** is an **edge** that connects a vertex to itself.



• Adjacent Nodes

- If two nodes u and v are connected via an edge e then the nodes u and v are called as neighbors or adjacent nodes.

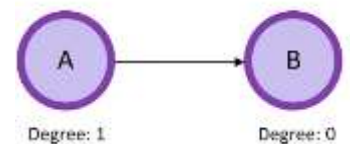
Adjacent Nodes



• Degree of the Node

- A degree of a node is the number of edges that are connected with that node. A node with degree 0 is called as isolated node.
- the degree of a node (or vertex) refers to the number of edges connected to that node.
- **Outdegree** of a vertex is the number of outgoing edges from a vertex in a directed graph. It is always positive, and 0 if no edges exist.
- **Indegree** of a vertex is the number of Incoming edges to a vertex in a directed graph. It is always positive, and 0 if no edges exist.

Degree of the Node



• Acyclic Graph

- An Acyclic graph is a directed graph with no directed cycles. It is also known as Directed Acyclic Graph (DAG).

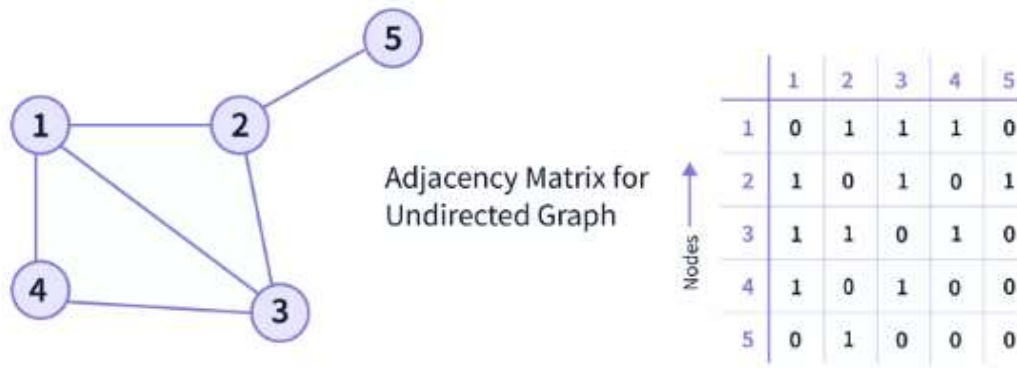
Difference Between Tree & Graph

Feature	Tree	Graph
Definition	A hierarchical data structure with a root node.	A collection of nodes connected by edges (can be arbitrary).
Direction	Always directed (from parent to child).	Can be directed or undirected .
Cycles	No cycles allowed (acyclic).	Can have cycles or be acyclic.
Hierarchy	Has a clear hierarchy (parent-child relationship).	No hierarchy necessarily exists.
Root Node	Has a single root node .	May or may not have a root.
Edge Count	If there are n nodes, there are exactly $n - 1$ edges.	Can have any number of edges.
Connectivity	Always connected.	May be connected or disconnected.
Types	Binary Tree, BST, AVL Tree, etc.	Directed Graph, Undirected Graph, Weighted Graph, etc.
Use Cases	File systems, organization charts, XML/JSON parsing.	Social networks, road maps, recommendation systems.

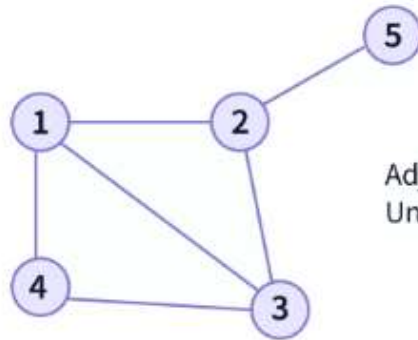
Graph Representation

Two ways to Representation of Graph

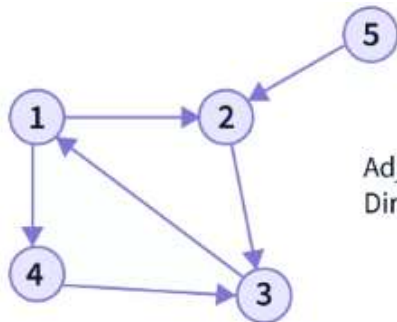
- a graph representation is a technique to store graph into the memory of computer
- The following two are the most commonly used representations of a graph.
 1. Adjacency Matrix
 2. Adjacency List
- **Adjacency Matrix:**
 - An Adjacency Matrix is a 2D array of size $V \times V$ where V is the number of nodes in a graph. It is used to represent a "finite graph", with 0's and 1's. Since, its size is $V \times V$, it is a square matrix.
 - The elements of the matrix indicates whether pairs of vertices are adjacent or not in the graph i.e. **is there any edge connecting a pair of nodes in the graph.**



- Let us take an example to simplify the above statements and understand better.
- The above example shows the adjacency matrix for the **undirected graph**. Let us note some important points:
 1. The adjacent matrix's row or column, consists of the nodes or vertices(that is numbered in red, in the above graph).
 2. Each cell in the above matrix is represented as A_{ij} , where i and j are nodes. The value of A_{ij} is either **1** or **0** depending on whether there is an edge from vertex i to vertex j . If there is a node then, $A_{ij} = 1$, otherwise $A_{ij} = 0$
 3. In the above graph, there is an edge between **node 1 & node 2**, so in the matrix, we have $A[1][2] = 1$ and $A[2][1] = 1$.
 4. If there is **no edge between 2 nodes**, then that cell in the **matrix will contain '0'**. For example, there is no edge from **node 1** to **node 5**, so, in the matrix, $A[1][5] = 0$ and $A[5][1] = 0$.
 5. Adjacency matrix of an undirected graph is **symmetric**. Hence, the above graph is symmetric.
- The adjacency Matrix for a **directed graph** also follows the same conventions, expect for, there is a '1' in the matrix if there is an edge pointing from one node to another, say from node A to node B. But vice versa may not be applicable. That is, in a directed graph, if $A[i][j] = 1$ then $A[j][i]$ may or may not be 1. Adjacency matrix of a directed graph is not symmetric(generally).



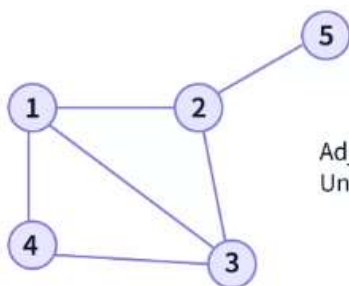
Adjacency Matrix for Undirected Graph



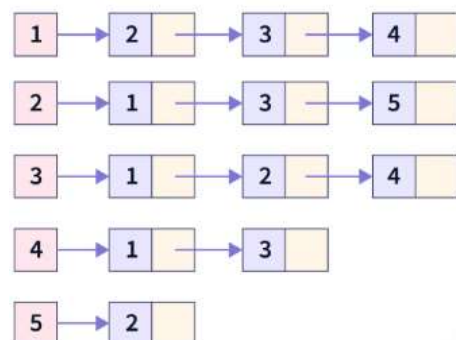
Adjacency Matrix for Directed Graph

		Nodes →				
		1	2	3	4	5
Nodes ↑	1	0	1	0	1	0
	2	0	0	1	0	0
	3	1	0	0	0	0
	4	0	0	1	0	0
	5	0	1	0	0	0

- This example clearly shows that, for **node 1**, we have $A[1][2] = 1$ but $A[2][1] = 0$, because we have a directed edge from **node 1** to **node 2**, but there is no edge from **node 2** to **node 1**.
- Again, we have a node from **node 2** to **node 3**, so in the matrix, $A[2][3] = 1$, but $A[3][2] = 0$, because there is no node from **node 3** to **node 2**.
- **Adjacency List:**
 - An adjacency list represents a graph as an **array of linked lists**.
 - The **index** of the array represents a node.
 - Each element in the **linked list** represents the nodes that are connected to that node by an edge.
 - Let us take an example for easy visualization –



Adjacency List for Undirected Graph



- The graph in our example is undirected and we have represented it using the Adjacency List. Let us look into some important points through this graph:
 1. Here, **1, 2, 3, 4, 5** are the nodes(vertices) and each of them forms an **array of linked list** with all of its adjacent nodes(vertices).
 2. In the graph, the **node 1** has 3 adjacent nodes namely -- **node 2, node 3, node 4**. So, in the list, the node is linked with 2, 3 & 4.
 3. **Node 4** has only 2 adjacent nodes, **node 1 & node 3**, so it is linked to the nodes 1 and 3 only, in the array of linked list.
 4. The last node in the linked list will point to **null**.
- Adjacency List also follows the same rule in case of directed graph, where the nodes will only be linked to the nodes to whom they have a directed edge(or, to the nodes their outgoing edges are pointing to)

Elementary Graph operations

Traversal of a graph means visit each node exactly once.

- Two commonly used graphs traversal techniques are

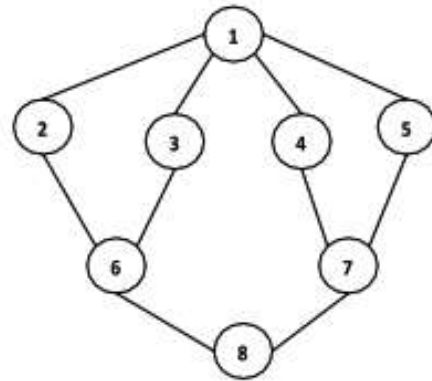
1. Depth First Search (DFS)

2. Breadth First Search (BFS)

➤ **Breadth First Search (BFS):**

- This methods starts from vertex v0
- V0 is marked as visited. All vertices adjacent to v0 are visited next
- Let vertices adjacent to v0 are v1, v2, v3, v4
- v1, v2, v3 and v4 are marked visited.
- All unvisited vertices adjacent to v1, v2, v3, v4 are visited next.
- The method continuous until all vertices are visited
- The algorithm for BFS has to maintain a list of vertices which have been visited but not explored for adjacent vertices. The vertices which have been visited but not explored for adjacent vertices can be stored in queue.
- Initially the queue contains the starting vertex.
- In every iteration, a vertex is removed from the queue and its adjacent vertices which are not visited as yet are added to the queue.
- The algorithm terminates when the queue becomes empty.

- This procedure traverse the graph G in BFS manner. V is a starting vertex to be explored. Q is a queue, visited[] is an array which tells you whether particular vertex is visited or not. W is a adjacent node of vertex V.



Graph G

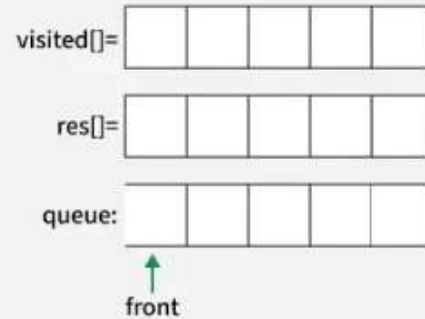
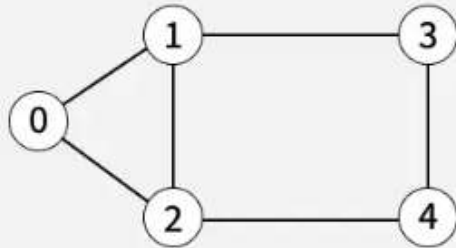
BFS traversal of given graph is:

1 | 2, 3, 4, 5 | 6, 7 | 8

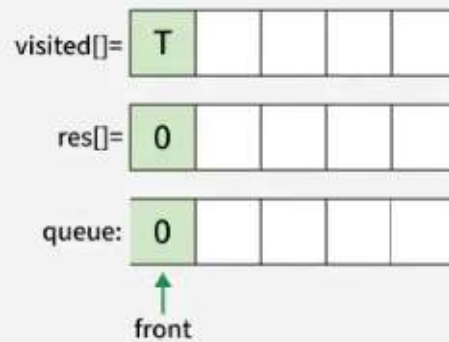
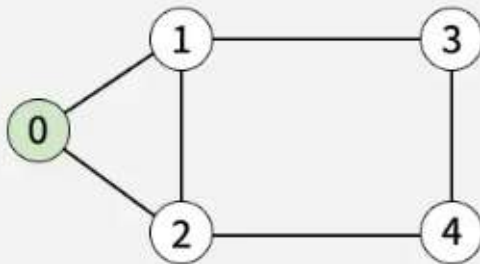
Procedure : BFS (Vertex V)

- Algorithm:**
 1. Initialize Q
 2. [Marks visited of V as 1]
visited [v] ← 1
 3. [Add vertex v to Q]
InsertQueue(V)
 4. [Repeat while Q is not empty]
Repeat while Q is not empty
v ← RemoveFromQueue()
For all vertices W adjacent to v
if
visited[w] is 0
then
visited[w] ← 1
InsertQueue(w)

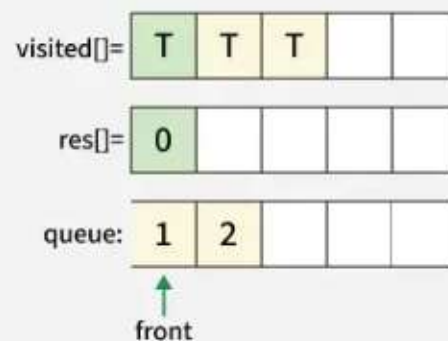
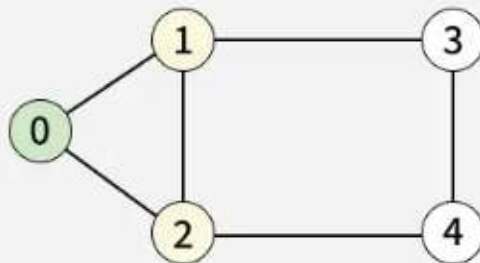
- 01** Maintain a queue, res[], and a Visited array, such that the queue stores nodes to be processed in FIFO order, res[] stores the BFS traversal sequence, and Visited keeps track of visited nodes to prevent reprocessing



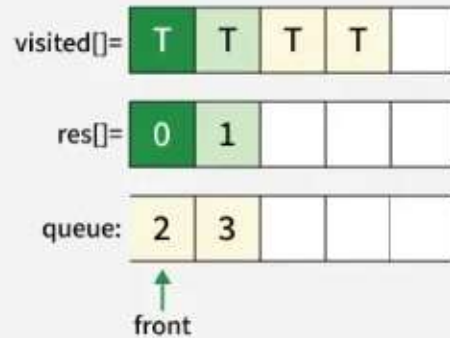
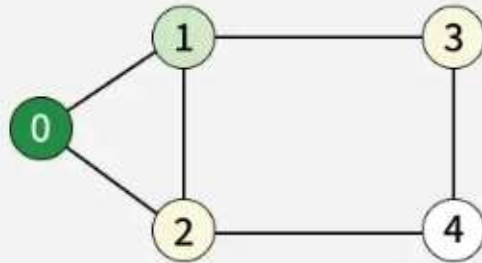
- 02** Start the traversal with node 0, push it into the queue, mark it as visited, and while the queue is not empty, pop the front node, store it in res



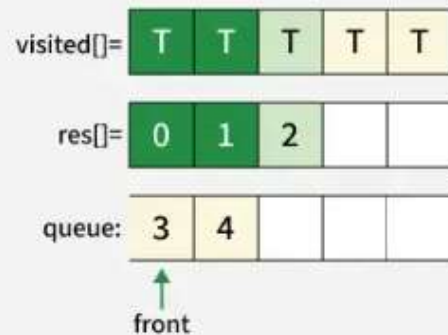
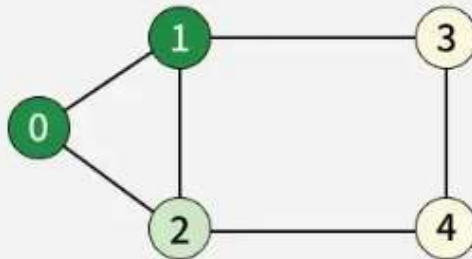
- 03** Remove the front element 0 from the queue, store it in res, visit its unvisited neighbors, mark them as visited, and push them into the queue for further processing.



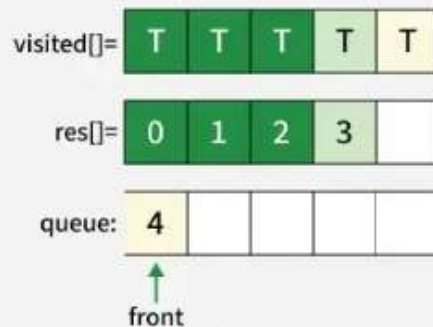
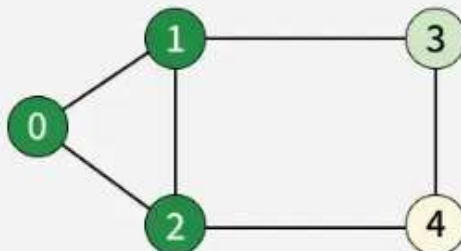
04 Remove the front element 1 from the queue, store it in res, visit its unvisited neighbors, mark them as visited, and push them into the queue for further processing.



05 Remove the front element 2 from the queue, store it in res, visit its unvisited neighbors, mark them as visited, and push them into the queue for further processing.

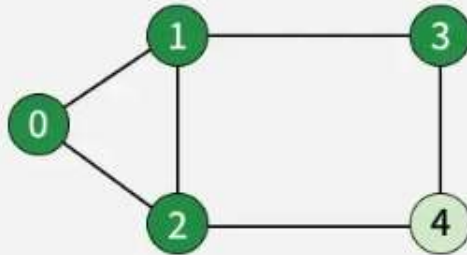


06 Remove the front element 3 from the queue, store it in res, visit its unvisited neighbors, mark them as visited, and push them into the queue for further processing.



All the node of 3 have been visited, proceed to the next node in the queue

07 Remove the front element 4 from the queue, store it in res, visit its unvisited neighbors, mark them as visited, and push them into the queue for further processing.



visited[] =

T	T	T	T	T
---	---	---	---	---

res[] =

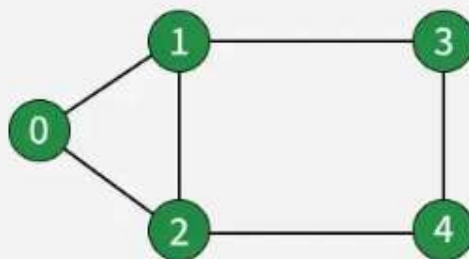
0	1	2	3	4
---	---	---	---	---

queue:

--	--	--	--	--

All the node of 4 have been visited, proceed to the next node in the queue

08 Now that the queue is empty and there are no more nodes to process, we obtain the final BFS traversal.



visited[] =

T	T	T	T	T
---	---	---	---	---

res[] =

0	1	2	3	4
---	---	---	---	---

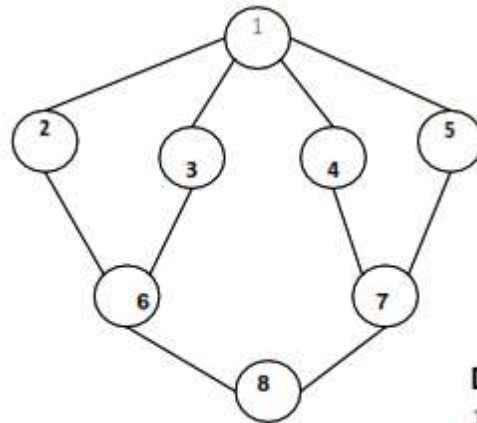
queue:

--	--	--	--	--

final BFS traversal order is 0, 1, 2, 3, 4.

➤ Depth First Search (DFS):

- It is like preorder traversal of tree.
- Traversal can start from any vertex v_i
- v_i is visited and then all vertices adjacent to v_i are traversed recursively using DFS
- Since graph can have cycles, we must avoid re-visiting a node. To do this when we visit a vertex V , we marks it visited as visited should not be selected for traversal.



DFS (G, 1) is given by

- a) Visit (1)
- b) DFS (G, 2)
- DFS (G, 3)
- DFS (G, 4)
- DFS (G, 5)

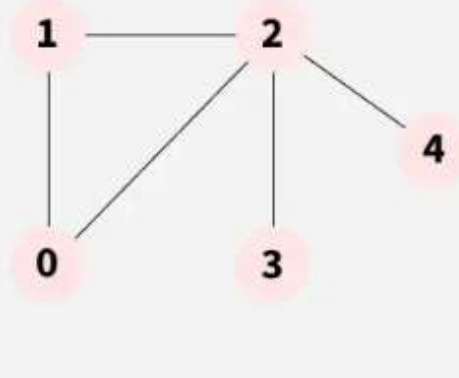
DFS traversal of given graph is:
1, 2, 6, 3, 8, 7, 4, 5

Graph G

- **Procedure : DFS (vertex V)**
- This procedure traverse the graph G in DFS manner. V is a starting vertex to be explored. S is a Stack, visited[] is an array which tells you whether particular vertex is visited or not. W is a adjacent node of vertex V. PUSH and POP are functions to insert and remove from stack respectively.
- **Algorithm:**
 1. [Initialize TOP and Visited]
 - visited[] $\leftarrow$ 0
 - TOP $\leftarrow$ 0
 2. [Push vertex into stack]
 - PUSH (V)
 3. [Repeat while stack is not empty]
 - Repeat step 3 while stack is not empty
 - v $\leftarrow$ POP()
 - if
 - visited[v] is 0
 - then
 - visited [v] $\leftarrow$ 1
 - for all W adjacent to v
 - if
 - visited [w] is 0
 - then
 - PUSH (W)
 - end for
 - end if

01
Step

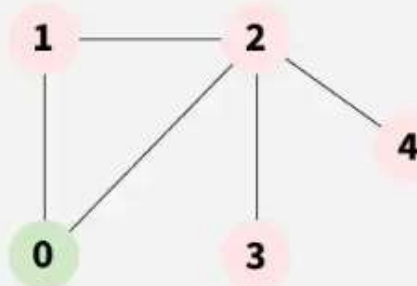
Maintain a visited array tracks which nodes have been explored.



	0	1	2	3	4
visited[] =	F	F	F	F	F
res[] =					

02
Step

Iteration 1: DFSRec(adj, visited, 0), Marks visited[0] = true
Call the dfsRec function, So call stack contain dfsRec

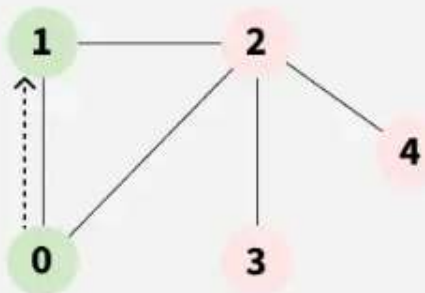


	0	1	2	3	4
visited[] =	T	F	F	F	F
res[] =	0				

dfsRec (0)

03
step

Iteration 2: DFSRec(adj, visited, 1),
Marks visited[1] = true

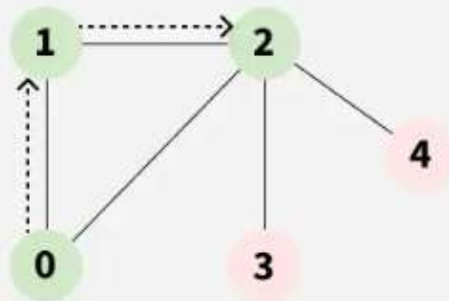


	0	1	2	3	4
visited[] =	T	T	F	F	F
res[] =	0	1			

dfsRec (1)
dfsRec (0)

04
Step

Iteration 3: DFSRec(adj, visited, 2),
Marks visited[2] = true



visited[] =

0	1	2	3	4
T	T	T	F	F

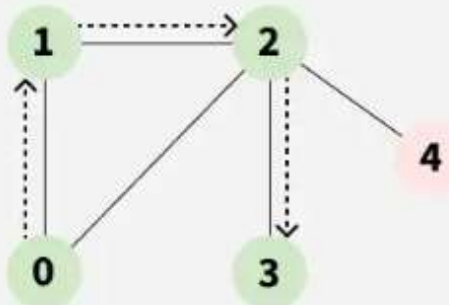
res[] =

0	1	2		
---	---	---	--	--

dfsRec (2)
dfsRec (1)
dfsRec (0)

05
Step

Iteration 4: DFSRec(adj, visited,3),
Marks visited[3] = true



visited[] =

0	1	2	3	4
T	T	T	T	F

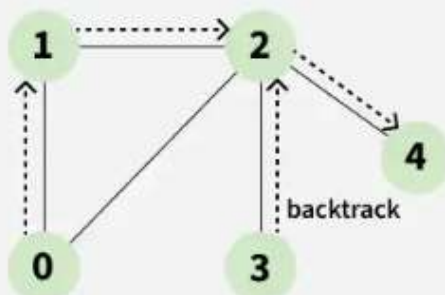
res[] =

0	1	2	3	
---	---	---	---	--

dfsRec (3)
dfsRec (2)
dfsRec (1)
dfsRec (0)

06
Step

Iteration 4: DFSRec(adj, visited,3),
Marks visited[3] = true



visited[] =

0	1	2	3	4
T	T	T	T	T

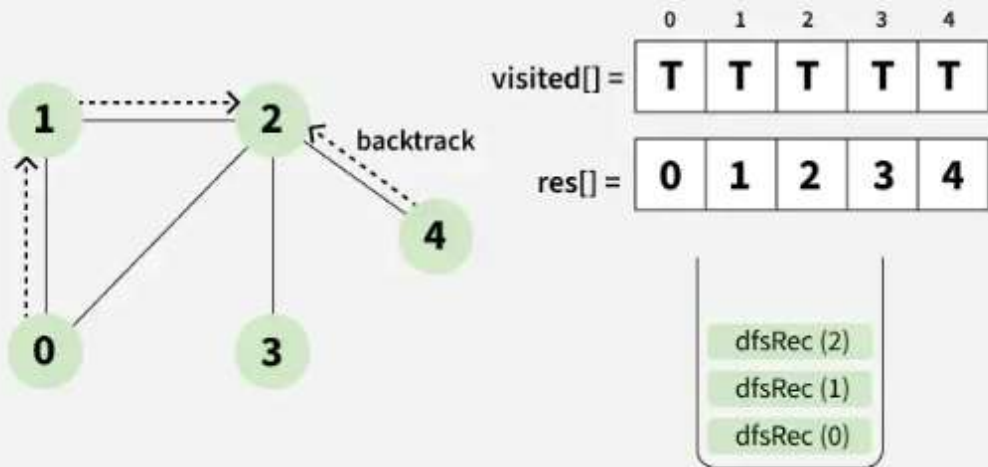
res[] =

0	1	2	3	4
---	---	---	---	---

dfsRec (4)
dfsRec (2)
dfsRec (1)
dfsRec (0)

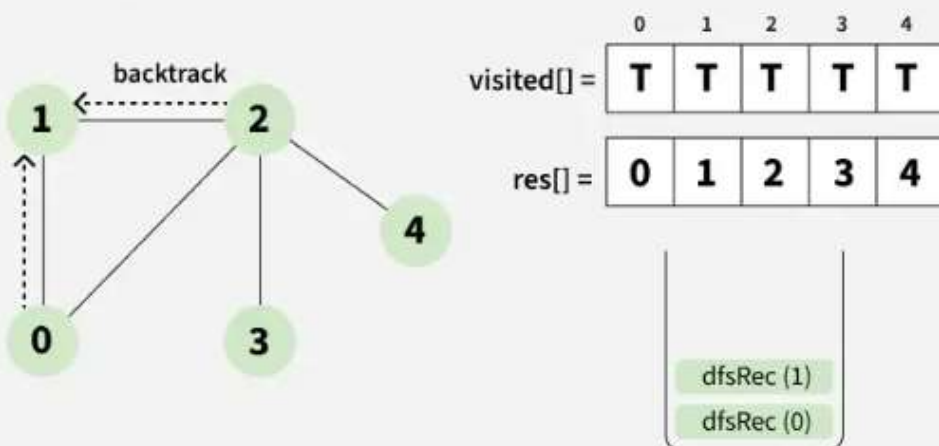
07
Step

Backtracking from 4



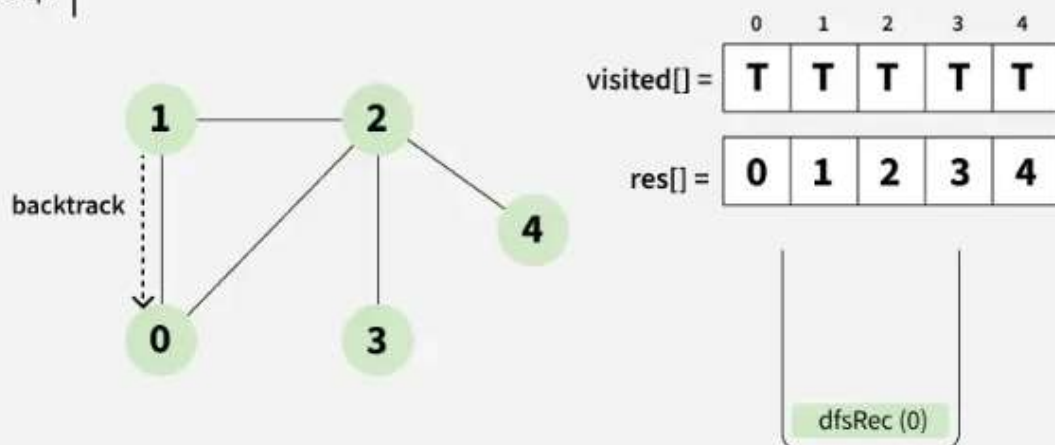
08
Step

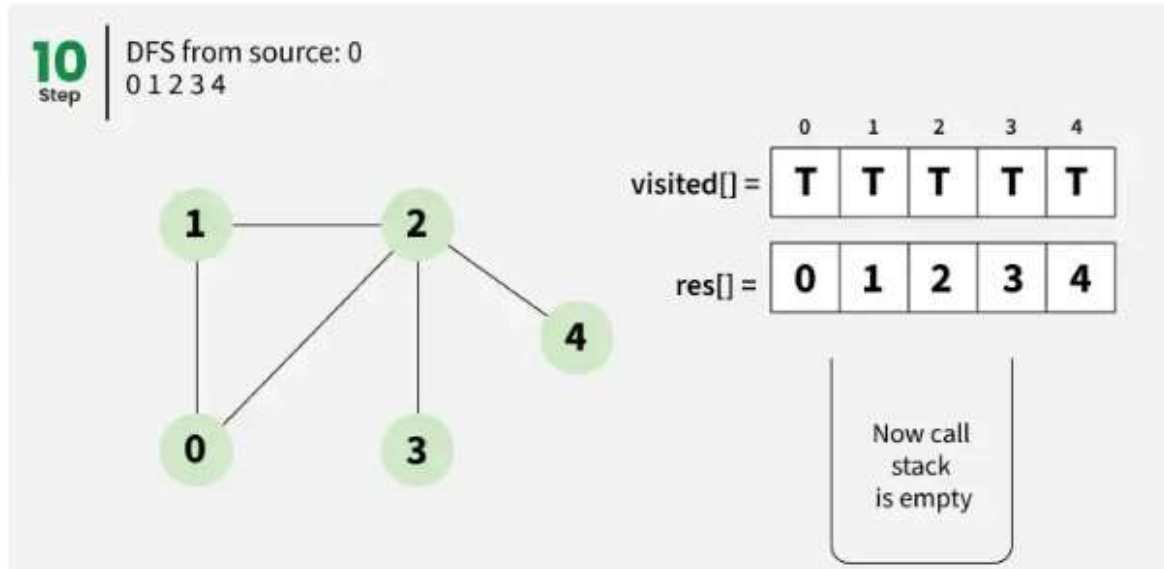
Backtracking from 2



09
Step

Backtracking from 1



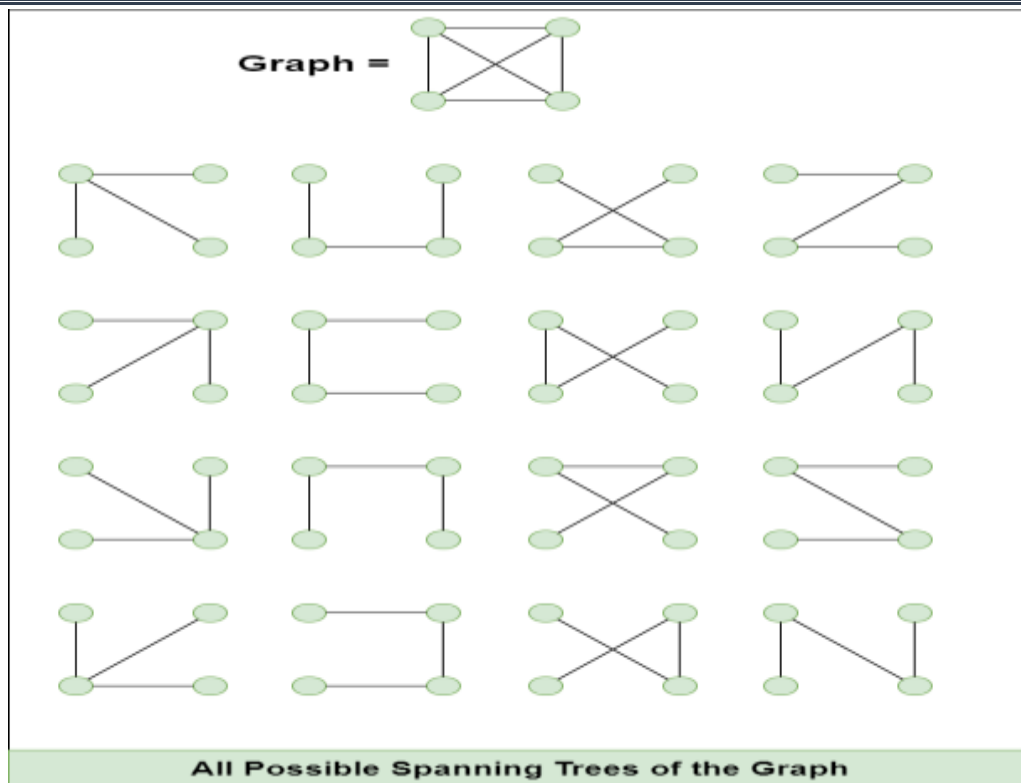


BFS vs. DFS

Feature	BFS (Breadth-First Search)	DFS (Depth-First Search)
Traversal Order	Level by level (breadth-wise)	Depth-wise (go as deep as possible before backtracking)
Data Structure Used	Queue	Stack (or recursion)
Visited Node Order	Explores neighbours first	Explores down a path as far as possible
Shortest Path (Unweighted Graph)	Yes, finds the shortest path	No, does not guarantee shortest path
Space Complexity	High (can store many nodes at a level)	Lower (stores only the path)
Suitable For	Finding shortest path, levels of a tree/graph	Searching deeply for solutions (e.g., puzzles)

Spanning Trees

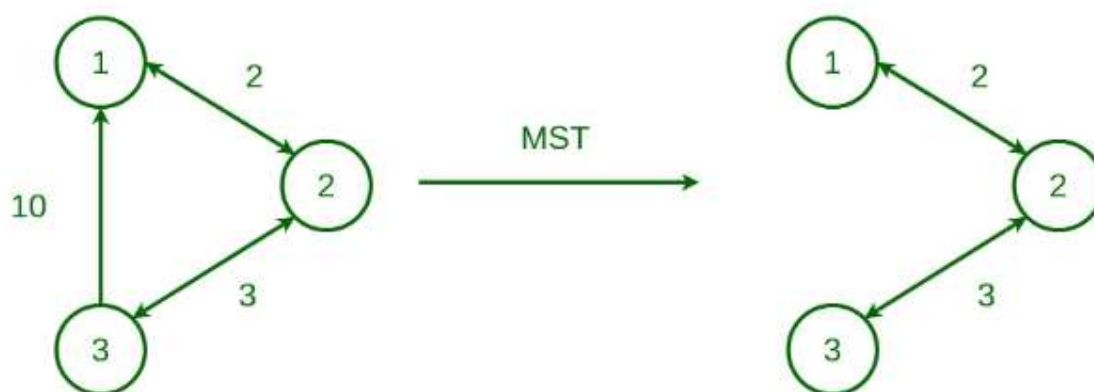
A spanning tree is a subset of Graph G, such that all the vertices are connected Using minimum possible number of edges. Hence, a spanning tree does not have cycles and a graph may have more than one spanning tree.



Minimum Spanning Tree

- The weight of a spanning tree is determined by the sum of weight of all the edge involved in it.
- A minimum spanning tree (MST) is defined as a spanning tree that has the minimum weight among all the possible spanning trees.

Minimum Spanning Tree for Directed Graph

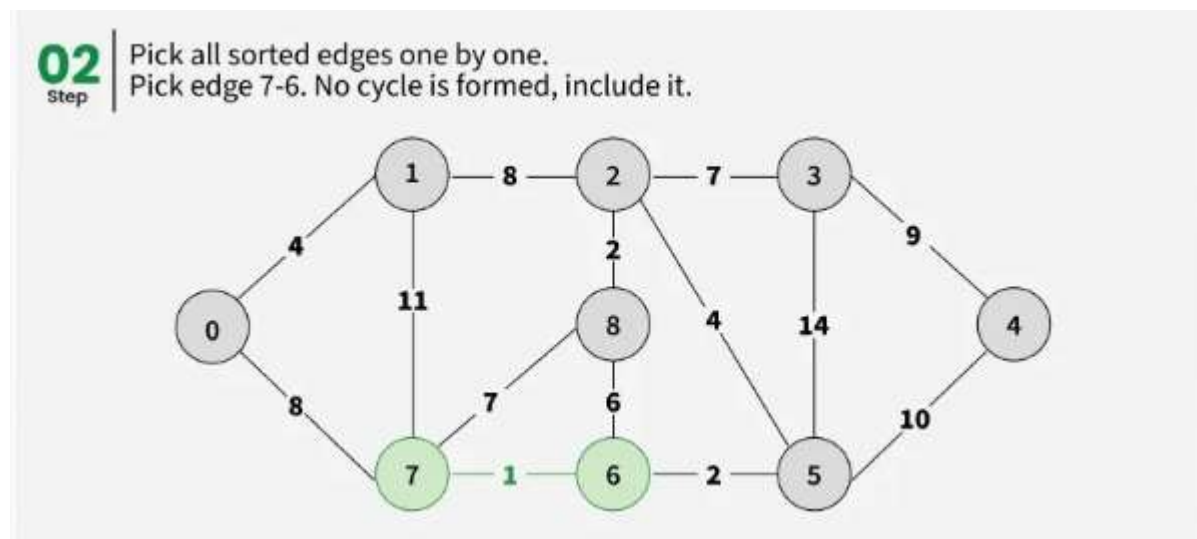
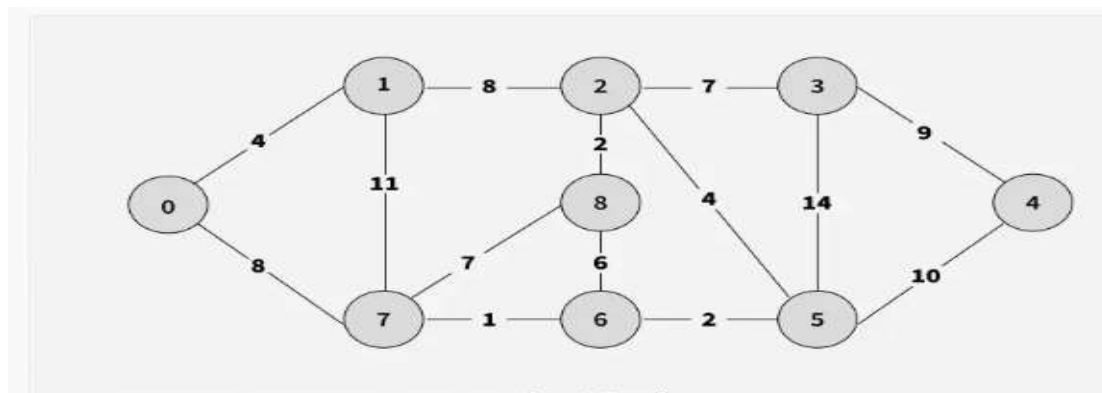


- There are two algorithms to find the minimum spanning tree from a given graph, some of them are listed below:
- **Krushkal's MST Algorithm**
 - **Prim's MST Algorithm**

Krushkal's Algorithm

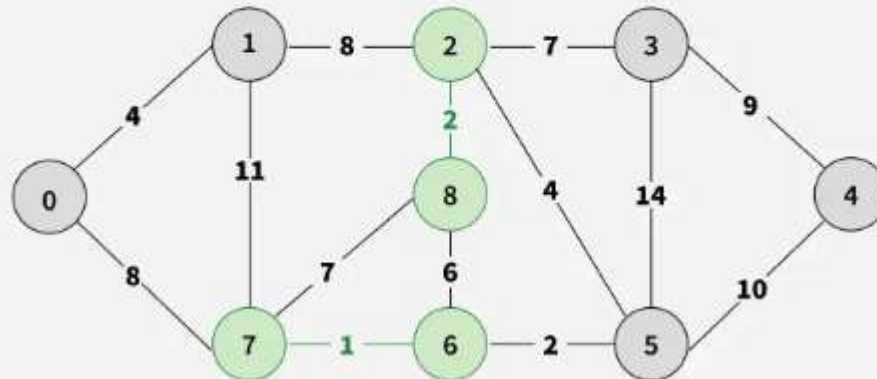
- First, it sorts all the edges of the graph by their weights, Then starts the iterations of finding the spanning tree.
- At each iteration, the algorithm adds the next lowest-weight edge one by one, such that the edges picked until now does not form a cycle.

Example : Find MST for given graph using Krushkal's Algorithm



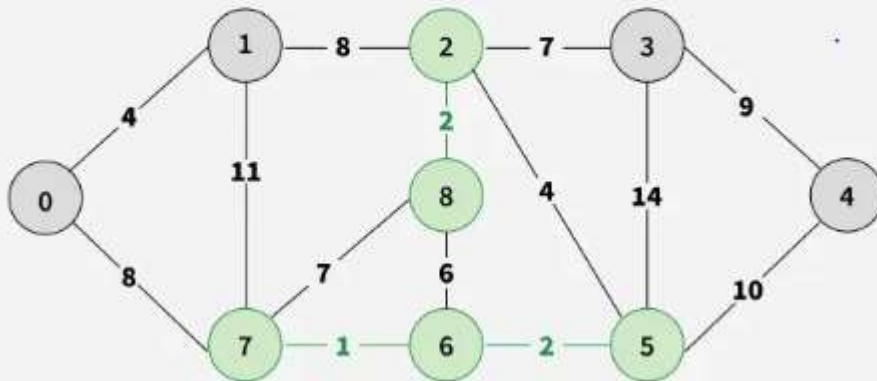
03
Step

Pick edge 8-2. No cycle is formed, include it.



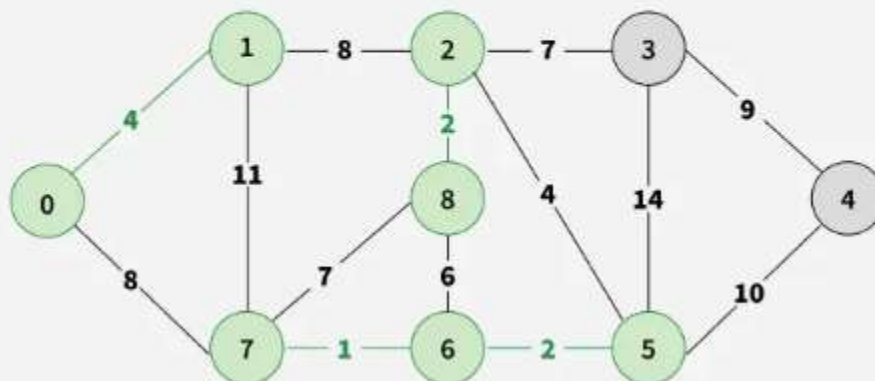
04
Step

Pick edge 6-5. No cycle is formed, include it.



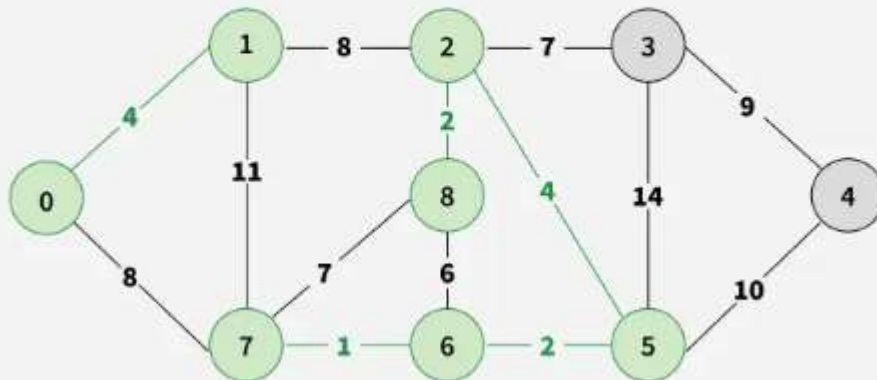
05
Step

Pick edge 0-1. No cycle is formed, include it.



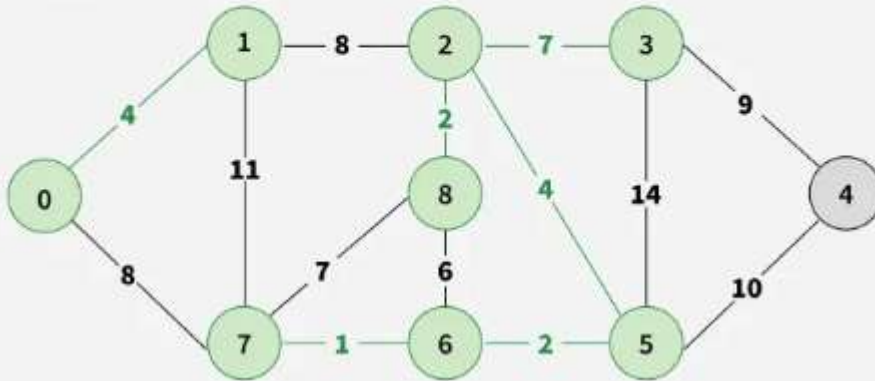
06
Step

Pick edge 2-5. No cycle is formed, include it.



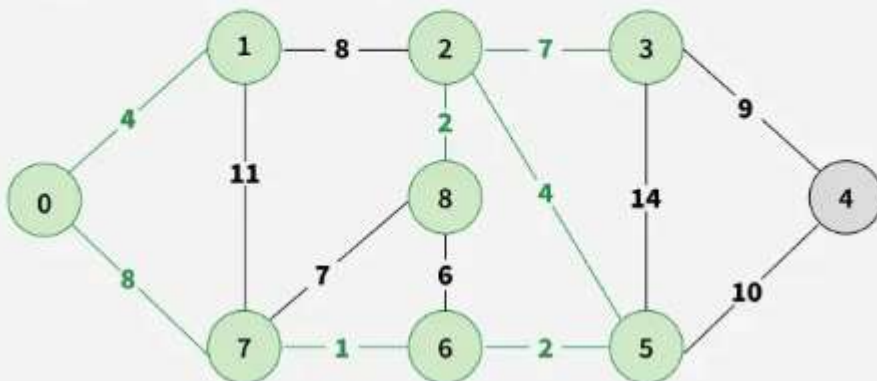
07
Step

Pick edge 8-6. Since including this edge results in the cycle, discard it.
Pick edge 2-3: No cycle is formed, include it.



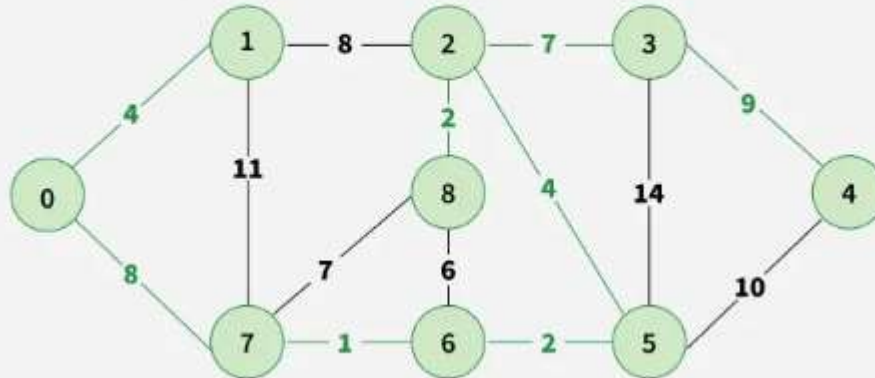
08
Step

Pick edge 7-8. Since including this edge results in the cycle, discard it.
Pick edge 0-7. No cycle is formed, include it.



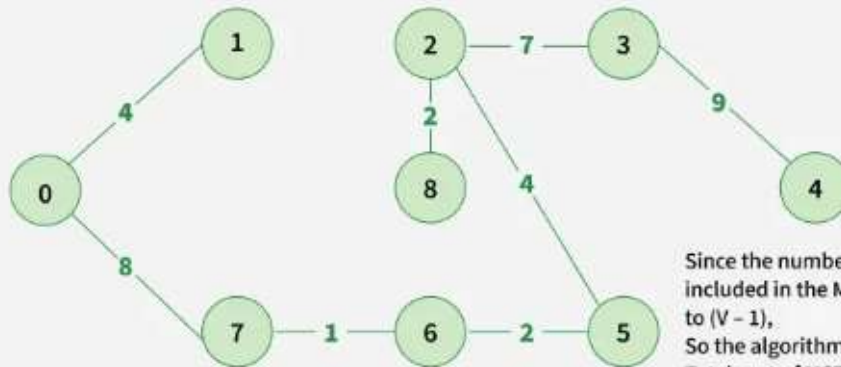
09
Step

Pick edge 1-2. Since including this edge results in the cycle, discard it.
Pick edge 3-4. No cycle is formed, include it.



10
Step

Final obtained graph minimum spanning tree.



Since the number of edges included in the MST equals to $(V - 1)$,
So the algorithm stops here.
Total cost of MST = (37)

Prim's Algorithm

Step 1: Determine an arbitrary vertex as the starting vertex of the MST. We pick 0 in the below diagram.

Step 2: Follow steps 3 to 5 till there are vertices that are not included in the MST (known as fringe vertex).

Step 3: Find edges connecting any tree vertex with the fringe vertices.

Step 4: Find the minimum among these edges.

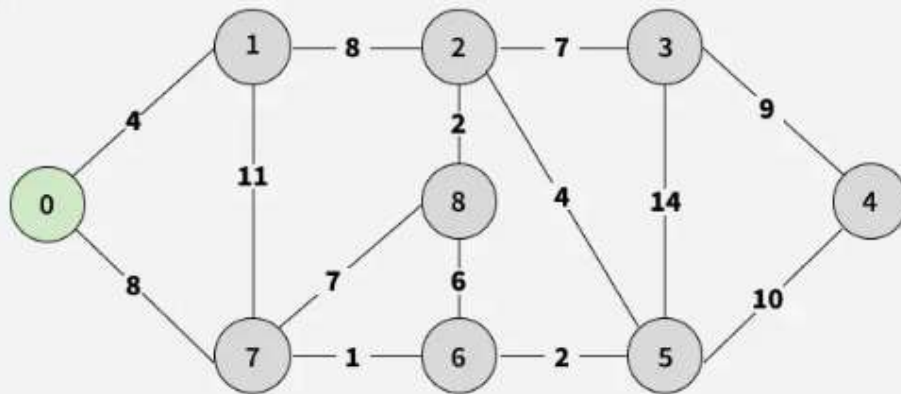
Step 5: Add the chosen edge to the MST. Since we consider only the edges that connect fringe vertices with the rest, we never get a cycle.

Step 6: Return the MST and exit

Example:

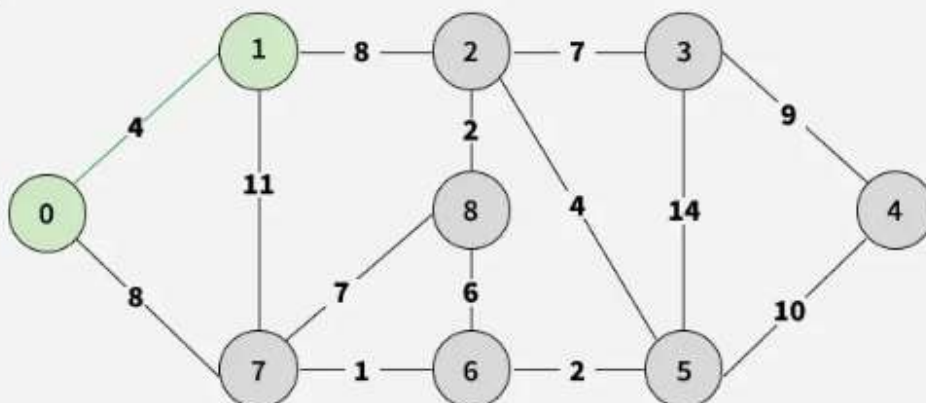
01
Step

Start with edges {0,1} and {0,7}; pick the minimum {0,1} and add vertex 1.



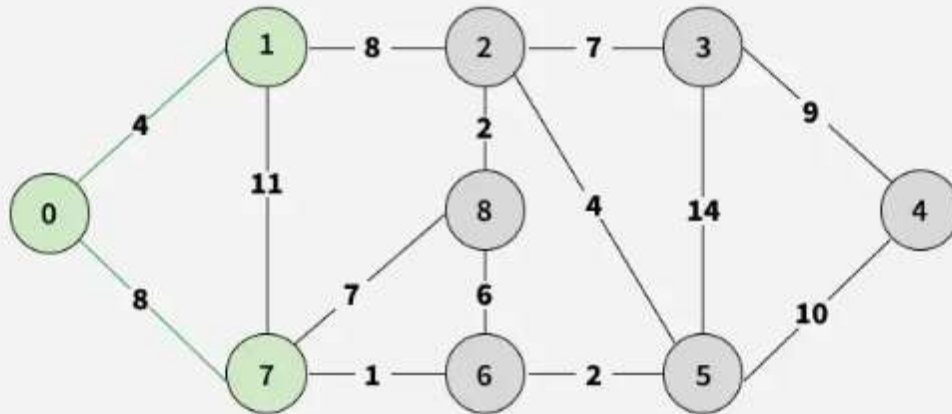
02
Step

Choose between {0,7} and {1,2}; pick {0,7} (or {1,2}), adding vertex 7.



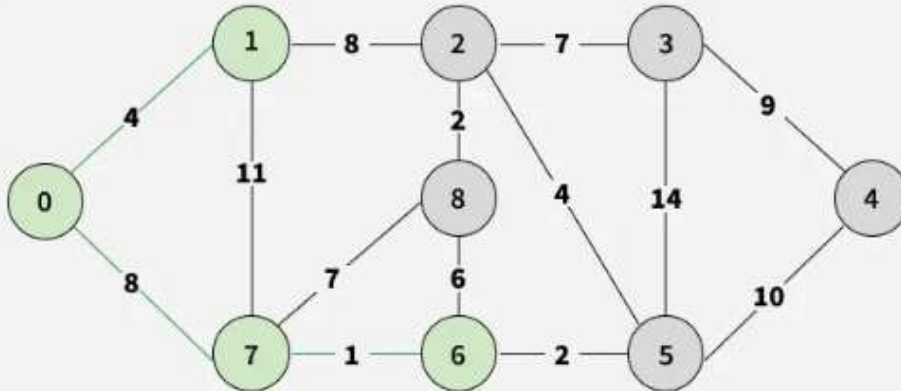
03
Step

Select {7,6} as it has the least weight (1), adding vertex 6.



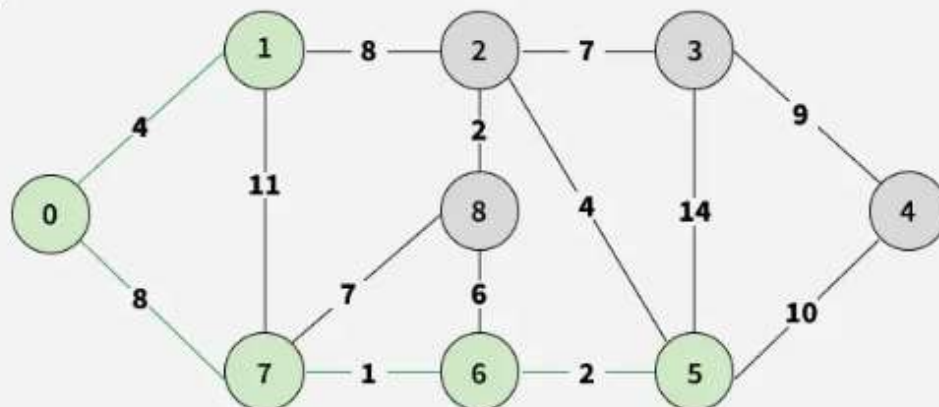
04
Step

Among {7,8}, {6,8}, and {6,5}, pick {6,5} (weight 2) and add vertex 5.



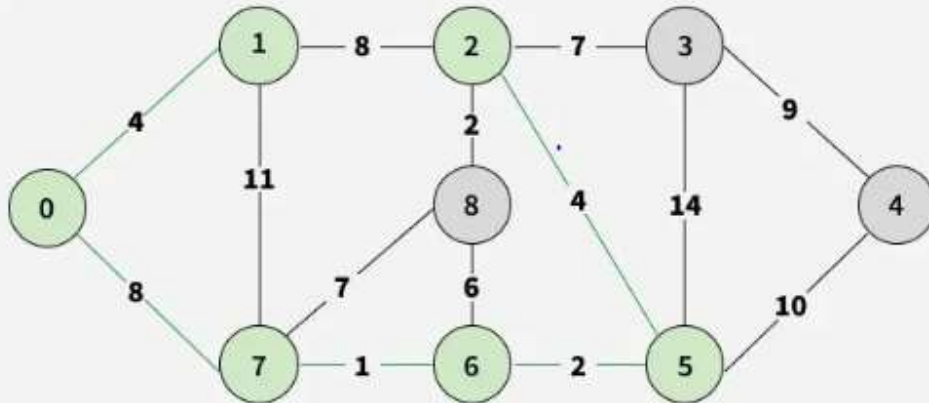
05
Step

Select {5,2} (weight 4), adding vertex 2.



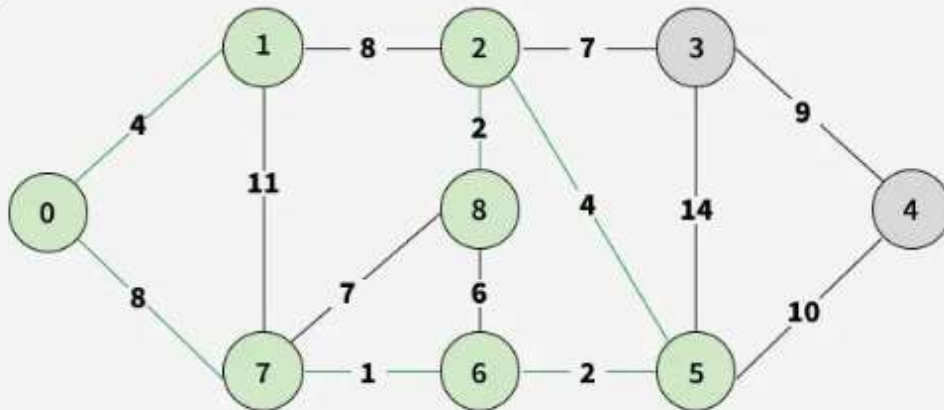
06
Step

Choose {2,8} (weight 2), adding vertex 8.



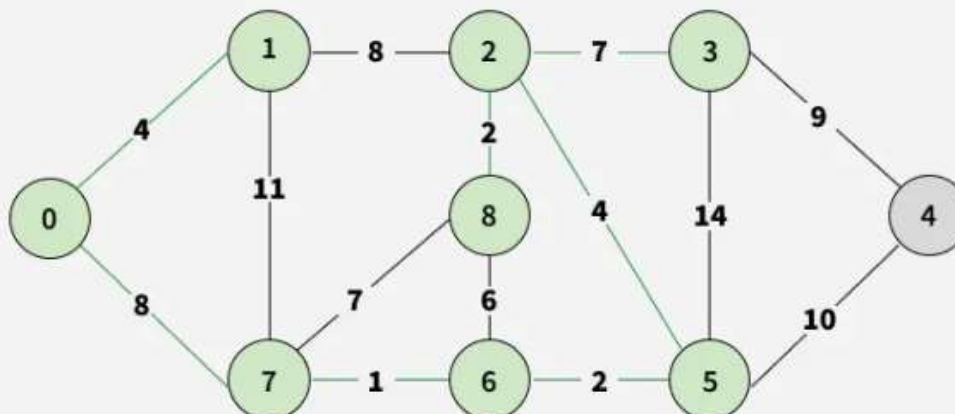
07
Step

Pick {2,3} as the smallest edge, adding vertex 3.



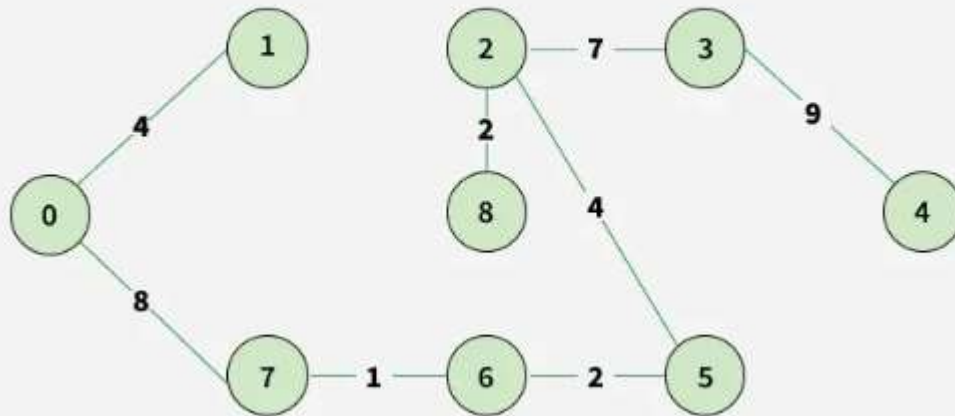
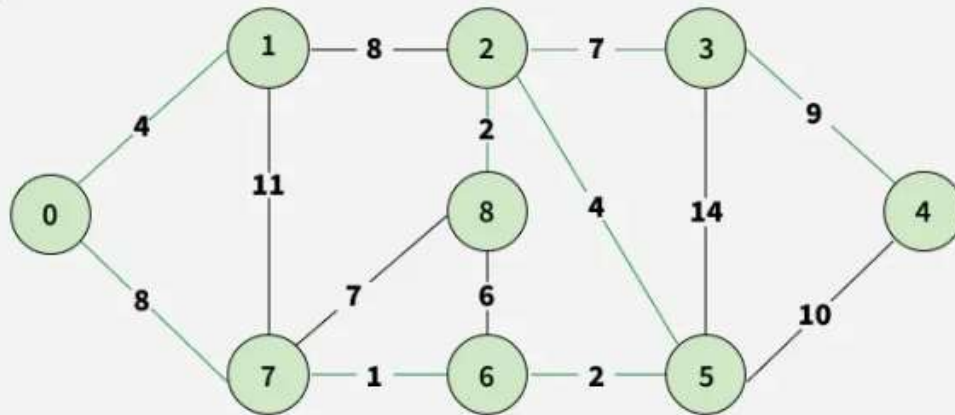
08
Step

Finally, select {3,4}, adding vertex 4.



09
Step

All vertices are included, so the MST is complete.



$$\text{MST} = 4+8+1+2+4+2+7+9 = 37.$$

Shortest Path

- To solve the shortest path problem means to find the shortest possible route or path between two vertices (or nodes) in a Graph.
- In the shortest path problem, a Graph can represent anything from a road network to a communication network, where the vertices can be intersections, cities, or routers, and the edges can be roads, flight paths, or data links.

Dijkstra's Algorithm for Shortest Path

Dijkstra's Algorithm is a shortest path algorithm used to find the minimum distance from a source node to all other nodes in a graph with non-negative edge weights.

Procedure

1. Initialize:

- Distance to source = 0.
- Distance to all other nodes = ∞ (infinity).
- Mark all nodes as unvisited.
- Keep a set of visited nodes.

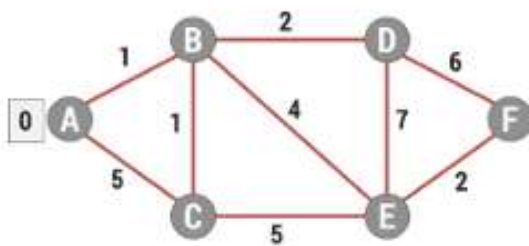
2. Pick the unvisited node with the smallest distance.

3. Update distances to its neighbors.

4. Mark the node as visited.

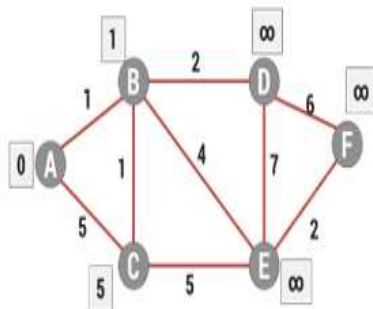
5. Repeat until all nodes are visited.

Example: Find Shortest Path from A to F in given Graph.



	A	B	C	D	E	F
Distance	0	∞	∞	∞	∞	∞
Visited	0	0	0	0	0	0

1st Iteration: Select Vertex A with minimum distance



	A	B	C	D	E	F
Distance	0	1	5	∞	∞	∞
Visited	1	0	0	0	0	0

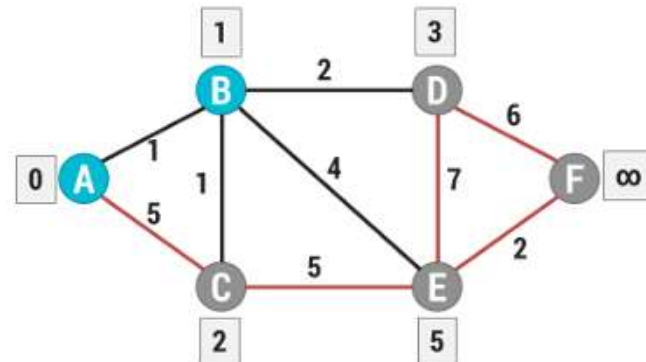
2nd Iteration: Select **Vertex B** with minimum distance

Cost of going to C via B = $\text{dist}[B] + \text{cost}[B][C] = 1 + 1 = 2$

Cost of going to D via B = $\text{dist}[B] + \text{cost}[B][D] = 1 + 2 = 3$

Cost of going to E via B = $\text{dist}[B] + \text{cost}[B][E] = 1 + 4 = 5$

Cost of going to F via B = $\text{dist}[B] + \text{cost}[B][F] = 1 + \infty = \infty$



	A	B	C	D	E	F
Distance	0	1	5	∞	∞	∞
Visited	1	0	0	0	0	0

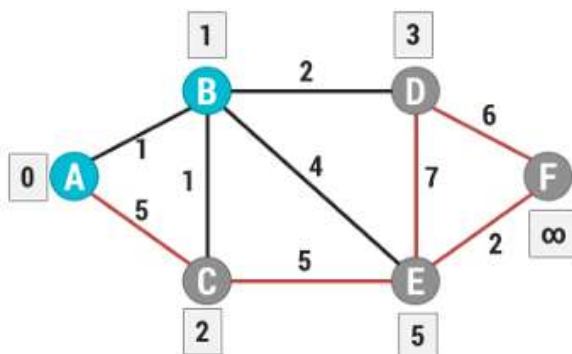
	A	B	C	D	E	F
Distance	0	1	2	3	5	∞
Visited	1	1	0	0	0	0

3rd Iteration: Select **Vertex C** via B with minimum distance

Cost of going to D via C = $\text{dist}[C] + \text{cost}[C][D] = 2 + \infty = \infty$

Cost of going to E via C = $\text{dist}[C] + \text{cost}[C][E] = 2 + 5 = 7$

Cost of going to F via C = $\text{dist}[C] + \text{cost}[C][F] = 2 + \infty = \infty$



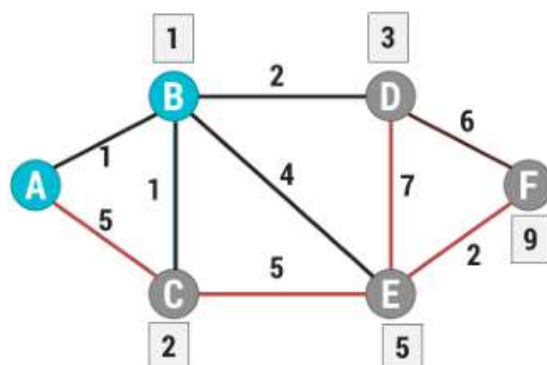
	A	B	C	D	E	F
Distance	0	1	2	3	5	∞
Visited	1	1	0	0	0	0

	A	B	C	D	E	F
Distance	0	1	2	3	5	∞
Visited	1	1	1	0	0	0

4th Iteration: Select **Vertex D** via path A - B with minimum distance

Cost of going to E via D = $\text{dist}[D] + \text{cost}[D][E] = 3 + 7 = 10$

Cost of going to F via D = $\text{dist}[D] + \text{cost}[D][F] = 3 + 6 = 9$



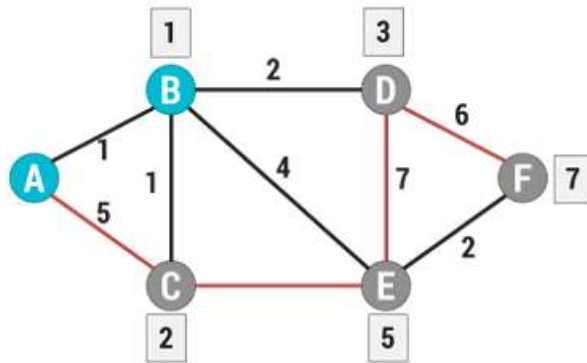
	A	B	C	D	E	F
Distance	0	1	2	3	5	∞
Visited	1	1	1	0	0	0

	A	B	C	D	E	F
Distance	0	1	2	3	5	9
Visited	1	1	1	1	0	0

4th Iteration: Select **Vertex E** via path A – B – E with minimum distance

Cost of going to F via E = $\text{dist}[E] + \text{cost}[E][F] = 5 + 2 = 7$

	A	B	C	D	E	F
Distance	0	1	2	3	5	9
Visited	1	1	1	1	0	0



	A	B	C	D	E	F
Distance	0	1	2	3	5	7
Visited	1	1	1	1	1	0

Shortest Path from A to F is

A → B → E → F = 7

GTU Questions from Unit 3

Sr. No	Questions	CO	Answer	Marks
1	Define the following terms (a) Complete binary tree (b) Leaf node (c) Height of a tree. [NLJIET]	3	Pg 09,03,05	3
2	Define following with respect to Tree: i) M-ary tree ii) Out Degree iii) Leaf [NLJIET]	3	Pg 09,65,05	3
3	Define the following: 1. Sibling 2. Forest 3. Strictly Binary Tree [NLJIET]	3	Pg 03,07,08	3
4	Define the terms below: 1) Complete Binary Tree 2) Forest 3) Abstract data type [NLJIET]	3	Pg 09,07	3
5	Define (1) Forest (2) Leaf Node (3) Tree [NLJIET]	3	Pg 07,05,01	3
6	Define following terms: 1. Full Binary Tree 2. Complete Binary Tree 3. Skewed Binary Tree [NLJIET]	3	Pg 09	3
7	What are binary trees? Mention different types of binary trees with example. [NLJIET]	3	Pg 09	3
8	How can you represent a Binary Tree in memory using array?[NLJIET]	3	Pg 12	3
9	Write algorithms for the inorder, preorder and postorder traversal of the binary tree.[NLJIET]	3	Pg 14	3
10	Construct Binary Tree where the preorder traversal is 1,2,4,5,3,6,8,9,7 & postorder is 4,5,2,8,9,6,7,3,1 [NLJIET]	3	Pg 21	3
11	Construct a binary tree from the traversals given below: Inorder: D, B, A, E, G, C, H, F, I Preorder: A, B, D, C, E, G, F, H, I	3	Pg 18	3
12	Write an algorithm to add a node into a binary search tree. [NLJIET]	3	Pg 26	3
13	Explain binary search tree with suitable example. [NLJIET]	3	Pg 26	3
14	Construct a binary search tree from the following numbers. 38, 13, 51, 10, 12, 40, 84, 25, 89, 37, 66, 95 [NLJIET]	3	Pg 26	3
16	Discuss Threaded Binary Tree. [NLJIET]	3	Pg 22	3
17	Explain AVL tree with suitable example. [NLJIET]	3	Pg 34	3
18	Create 2-3 Tree for the following sequence: 50, 100, 150, 200 [NLJIET]	3	Pg 29	3
19	Explain 2-3 Tree in brief. [NLJIET]	3	Pg 55	3
20	Discuss height balance tree. [NLJIET]	3	Pg 34	3
21	What is B -tree of order m? Draw a B-tree of order 3. [NLJIET]	3	Pg 49	3



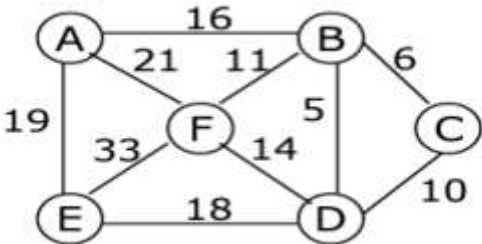
New L.J Institute of Engineering and Technology
Semester: III Subject: Data Structures (BE03000081)
Unit 3: Non Linear Data Structure

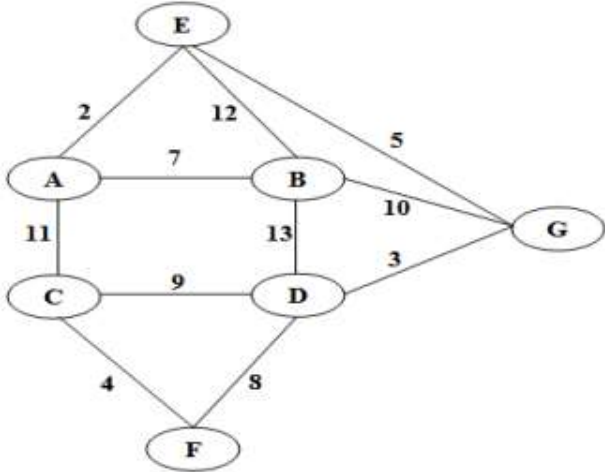
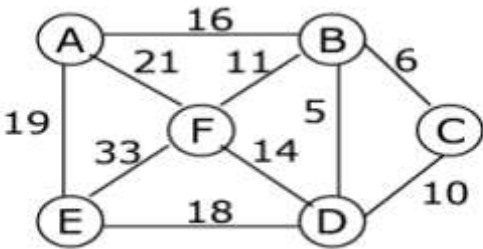
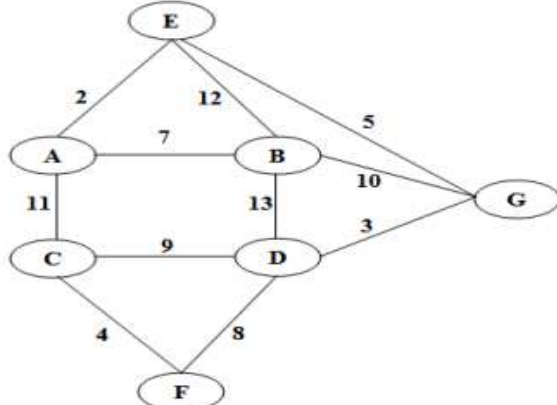
22	Define the terms below: 1) Threaded Tree 2) Acyclic graph 3) Sparse matrix [NLJIET]	3	Pg 22,65	3
23	Define: 1. Connected graph 2. Threaded tree 3. Degree of node [NLJIET]	3	Pg 64,22,65	3
24	Define: 1. Acyclic graph 2. Leaf node 3. Complete binary tree [NLJIET]	3	Pg 22,05,09	3
25	Define following terms 1) Balance factor 2) Hash function 3) Cyclic graph [NLJIET]	3	Pg 34,62	3
26	Define following terms 1) Hash table 2) Graph 3) Complete binary tree [NLJIET]	3	Pg 62,09	3
27	Define Graph and list any 3 uses of graph. [NLJIET]	3	Pg 62	3
28	Define the following terms (a) Directed graph (b) Multigraph (c) Complete graph. [NLJIET]	3	Pg 62,63,64	3
30	Explain Prim's algorithm. [NLJIET]	3	Pg 84	3
31	Write an algorithm for a non recursive (Iterative) pre order traversal of Binary search tree. [NLJIET]	3	Pg 14	4
32	Draw a Binary expression tree for the following and perform preorder traversal: $a * (b + c) + (d * e) / f + g * h$ [NLJIET]	3	Pg 58	4
33	Given Inorder and Preorder traversal, find Postorder traversal. Inorder: Y B K C F A G X E D H Z Preorder: G B Y A C K F X D E Z H [NLJIET]	3	Pg 18	4
34	Represent following in form of an expression tree: $A+B*(C+D)$ [NLJIET]	3	Pg 58	4
35	Construct a binary tree having the following traversal sequences: Preorder traversal A B C D E F G H I Inorder traversal B C A E D G H F I [NLJIET]	3	Pg 18	4
36	Construct a binary tree from the traversals given below: In-order : D, B, A, E, G, C, H, F, I Pre-order : A, B, D, C, E, G, F, H, I [NLJIET]	3	Pg 18	4
37	Construct tree from Following • In order Traversal: D, B, E, A, F, C Pre order Traversal: A, B, D, E, C, F [NLJIET]	3	Pg 18	4
38	Construct a binary tree from the traversals given below: Pre-order : 1, 2, 4, 8, 9, 5, 3, 6, 7 Post-order : 8, 9, 4, 5, 2, 6, 7, 3, 1 [NLJIET]	3	Pg 21	4
39	For following expressions, construct the corresponding binary tree. 1. $A+B/C*D-E$ 2. $((A+B)-(C*D))\%((E^F)/(G-H))$ [NLJIET]	3	Pg 58	4

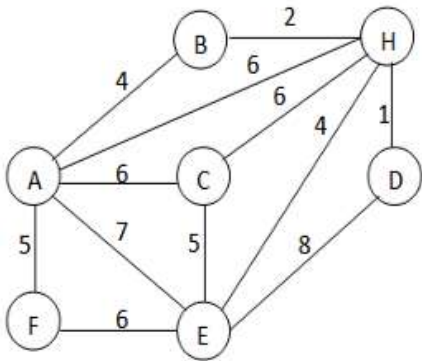
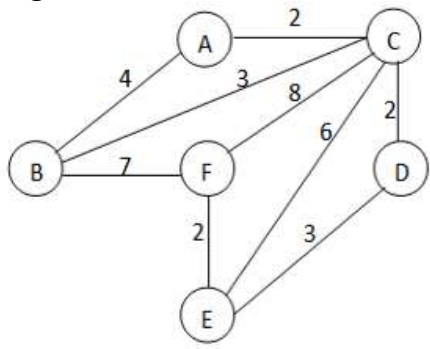


New L.J Institute of Engineering and Technology
Semester: III Subject: Data Structures (BE03000081)
Unit 3: Non Linear Data Structure

40	Construct BST for following sequence and find inorder traversal for the same. 35, 46, 29, 2, 24 ,68, 44, 57, 1, 22, 79, 71 [NLJIET]	3	Pg 26	4
41	Describe Threaded Binary Tree with example. [NLJIET]	3	Pg 22	4
42	Explain rotation rules for AVL tree. [NLJIET]	3	Pg 34	4
43	Write a short on AVL tree. [NLJIET]	3	Pg 34	4
44	Explain AVL tree in detail with suitable example. [NLJIET]	3	Pg 34	4
45	Construct an AVL Tree by inserting numbers from 1 to 8. [NLJIET]	3	Pg 34	4
46	Explain 2-3 tree in brief. [NLJIET]	3	Pg 49	4
47	Define the following: 1. Sibling 2. Forest 3. Complete Binary Tree 4. Complete Graph [NLJIET]	3	Pg 03,07, 09,64	4
48	What is a graph? Explain various representations of graphs. [NLJIET]	3	Pg 66	4
49	Explain BFS and DFS. [NLJIET]	3	Pg 69	4
50	Explain the DFS traversal of the graph with an example. [NLJIET]	3	Pg 73	4
51	Differentiate between depth first search and breadth first search. [NLJIET]	3	Pg 78	4
52	Differentiate: BFS and DFS. [NLJIET]	3	Pg 78	4
53	Discuss Minimal Spanning Tree. [NLJIET]	3	Pg 79	4
54	Explain minimal spanning tree. [NLJIET]	3	Pg 79	4
55	Explain tree traversal in detail with example. [NLJIET]	3	Pg 13	7
56	Construct a binary tree from the traversals given below: Inorder : D, B, E, H, A, I, F, C, G Preorder : A, B, D, E, H, C, F, I, G [NLJIET]	3	Pg 18	7
57	Create Binary Search Tree for following Data and write pre-order traversal, in-order traversal and post-order traversal of the constructed tree. 10 15 28 09 39 31 30 14 07 08 [NLJIET] Construct a Binary Search Tree for the following data. 21, 51, 12, 45, 17, 71, 19, 47, 78. [NLJIET] Write Pre-order, In-order, and Post-order traversal of constructed BST. [NLJIET]	3	Pg 26	7
58	Design an algorithm to insert a given value in the binary search tree. [NLJIET]	3	Pg 26	7
59	Write an algorithm to add a node into a binary search tree. [NLJIET]	3	Pg 26	7
60	Write an algorithm for DELETE operation in a Binary search tree. [NLJIET]	3	Pg 30	7
61	Explain the concept of a binary search tree (BST) and its properties. Discuss how operations such as insertion, deletion, and searching are performed in a BST. [NLJIET]	3	Pg 26	7
62	What is Binary Search Tree? Construct a binary search tree for the following elements 21, 16, 24, 18, 22, 25, 26,	3	Pg 26	7

	27, 29, 33 [NLJIET]			
63	What is a binary search tree? Create a binary search tree for the following data. 14, 10, 17, 12, 10, 11, 20, 12, 18, 25, 20, 8, 22, 11, 23. Explain deleting node 20 in the resultant binary search tree. [NLJIET]	3	Pg 26	7
64	Discuss the key concepts and techniques of AVL trees and 2-3 trees, focusing on their balance mechanisms and the impact on performance. Provide examples of insertions and deletions to illustrate how these trees maintain balance. [NLJIET]	3	Pg 34	7
65	Create an AVL tree for the following sequence of numbers. Also mention name of action taken. 200, 400, 800, 900, 850, 700, 950, 100, 150 [NLJIET]	3	Pg 34	7
66	Construct AVL tree for following sequence: 10, 20, 30, 40, 50, 60, 70, 80 [NLJIET]	3	Pg 34	7
67	Create AVL tree for following Data and write pre-order traversal of the constructed tree. 10 15 28 09 39 31 30 14 07 08 [NLJIET]	3	Pg 34	7
68	Explain the concept of B-tree with suitable example and list its applications. [NLJIET]	3	Pg 49	7
69	Explain insertion and deletion in B-tree with example. [NLJIET]	3	Pg 49	7
70	Insert the following letters into an empty B-tree of order 5: C N G A H E K Q M F W L T Z D P R X Y S [NLJIET]	3	Pg 49	7
71	Explain B+ tree with example. [NLJIET]	3	Pg 49	7
72	How are graphs represented inside a computer's memory? Which method do you prefer and why? [NLJIET]	3	Pg 66	7
73	Explain the concept of a graph and its representations. Compare adjacency matrix and adjacency list representations in terms of space complexity and use cases. [NLJIET]	3	Pg 66	7
74	Discuss algorithm of Breadth First Search (BFS) traversal for a Graph. Explain with an example. [NLJIET]	3	Pg 69	7
75	Explain DFS and BFS with appropriate example. [NLJIET]	3	Pg 69	7
76	Differentiate DFS and BFS in detail. [NLJIET] Compare and contrast Depth First Search (DFS) and Breadth First Search (BFS) in terms of their algorithms, uses, and performance. [NLJIET]	3	Pg 69	7
77	Explain Prim's algorithm with suitable example. [NLJIET]	3	Pg 84	7
78	Construct the minimum spanning tree using prim's algorithm for the following graph. [NLJIET] 	3	Pg 84	7

79	Find the minimum spanning tree of the following graph using prim's method. [NLJIET] 	3	Pg 84	7
80	Explain Kruskal's algorithm with suitable example. [NLJIET]	3	Pg 80	7
81	Construct the minimum spanning tree using Kruskal's algorithm for the following graph. [NLJIET] 	3	Pg 80	7
82	Find the minimum spanning tree of following graph using Kruskal's method. [NLJIET] 	3	Pg 80	7
83	Find the minimum spanning tree of the following graph using Kruskal's algorithm. [NLJIET]	3	Pg 80	7

				
84	Explain prim's and krushkal's algorithm with suitable example. [NLJIET]	3	Pg 80 to 87	7
86	Find the shortest path from A to F using Dijkstra's Algorithm. [NLJIET] 	3	Pg 87	7
87	Explain Dijkstra's shortest path using suitable example. [NLJIET] Explain Dijkstra's shortest path algorithm with appropriate example. [NLJIET]	3	Pg 87	7